

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



ĐỒ ÁN TỐT NGHIỆP KỸ THUẬT MÁY TÍNH
NGÀNH KỸ THUẬT MÁY TÍNH

**Hoạch định chuyển động cho rô-bốt sử
dụng tối ưu lỗi**

Học kỳ 252

Giảng viên hướng dẫn: PGS. TS Lê Hồng Trang
ThS. Trương Quỳnh Chi

STT	Họ và tên	MSSV	Ghi chú
1	Nguyễn Trần Đăng Khoa	2211635	Phân hoạch lỗi; Hiện thực, thực nghiệm
2	Hồ Đăng Khoa	2211588	Cơ sở lý thuyết; Mô hình hóa bài toán

Thành phố Hồ Chí Minh, Tháng 5 năm 2026

Lời Cam Đoan

Chúng tôi xin cam đoan rằng chúng tôi đã thực hiện đề án này, dưới sự hướng dẫn của PGS. TS. Lê Hồng Trang tại Khoa Khoa học và Kỹ thuật Máy tính, Trường Đại học Bách khoa - Đại học Quốc gia Thành phố Hồ Chí Minh.

Chúng tôi đã trích dẫn và ghi nhận đầy đủ tất cả các nguồn tài liệu và tài liệu tham khảo bên ngoài được sử dụng trong đề án.

Nếu có bất kỳ hành vi đạo văn nào, chúng tôi xin chịu hoàn toàn trách nhiệm. Trường Đại học Bách khoa - Đại học Quốc gia TP.HCM sẽ không chịu trách nhiệm cho bất kỳ vi phạm bản quyền nào có thể xảy ra trong quá trình nghiên cứu của chúng tôi.

TP. Hồ Chí Minh, tháng 5 năm 2026

Nhóm sinh viên thực hiện,

Nguyễn Trần Đăng Khoa

Hồ Đăng Khoa

Lời Cảm Ơn

Chúng em xin bày tỏ lòng biết ơn sâu sắc đến PGS. TS. Lê Hồng Trang và Cô Trương Quỳnh Chi vì sự hỗ trợ và hướng dẫn tận tình. Nhờ có kiến thức sâu rộng, những góp ý sâu sắc và sự hỗ trợ không ngừng của các thầy/cô, quá trình nghiên cứu của chúng em đã đạt được nhiều kết quả tốt đẹp.

Bên cạnh đó, chúng em muốn gửi lời tri ân đến gia đình. Niềm tin vững chắc và sự động viên không ngừng nghỉ của gia đình chính là điểm tựa vững chắc nhất cho chúng em. Sự tin tưởng của gia đình luôn là nguồn động lực to lớn giúp chúng em vượt qua mọi khó khăn. Chúng em vô cùng biết ơn tình yêu thương và sự ủng hộ đó.

Tóm tắt

Hoạch định chuyển động cho robot trong môi trường chứa vật cản vốn là một bài toán tối ưu hóa phi tuyến đầy thách thức do tính chất không lồi của không gian cấu hình. Trong khi các phương pháp dựa trên lấy mẫu truyền thống thường gặp hạn chế về tính tối ưu và khả năng xử lý các ràng buộc động lực học phức tạp, bài báo này đề xuất một phương pháp tiếp cận mới dựa trên Đồ thị các Tập Lồi (Graphs of Convex Sets - GCS). Bằng cách phân rã không gian tự do thành các vùng lồi hữu hạn và mô hình hóa quỹ đạo robot sử dụng đường cong Bézier, chúng tôi thiết lập bài toán dưới dạng tối ưu hóa hỗn hợp nguyên. Cuối cùng ta áp dụng kỹ thuật nới lỏng lồi (convex relaxation) chặt chẽ để chuyển đổi bài toán phức tạp này thành bài toán Lập trình nón bậc hai (Second-Order Cone Programming - SOCP) dễ giải. Kết quả thực nghiệm trên các hệ thống có số chiều cao chứng minh phương pháp này vượt trội hơn so với các thuật toán PRM truyền thống, giảm thiểu đáng kể thời gian tính toán trong khi vẫn đảm bảo tìm được quỹ đạo tối ưu toàn cục. Chúng tôi kiểm chứng GCS trên một loạt các môi trường với độ phức tạp tăng dần, từ các vật cản 2D đơn giản và mê cung phức tạp đến các thiết bị bay quadrotor động lực học và tay máy 7 bậc tự do (7-DoF) trong không gian nhiều chiều. Các thực nghiệm số chứng minh rằng GCS đạt hiệu năng vượt trội một cách nhất quán so với các bộ hoạch định dựa trên lấy mẫu được sử dụng rộng rãi (ví dụ: PRM). Cụ thể, trong các nhiệm vụ nhiều chiều, GCS giảm thời gian truy vấn trực tuyến lên đến một bậc so với PRM tiêu chuẩn, đồng thời tìm ra các quỹ đạo tối ưu toàn cục ngắn hơn đáng kể (giảm tới 40% độ dài) so với các bộ hoạch định dựa trên lấy mẫu đã được xử lý hậu kỳ [1].

Mục lục

1	Giới thiệu	1
1.1	Động lực và Thách thức Kỹ thuật	1
1.2	Mục tiêu	3
1.3	Phạm vi nghiên cứu	4
1.4	Cấu trúc luận văn	5
2	Kiến thức nền tảng	6
2.1	Giải tích Lồi và Tối ưu hóa	7
2.1.1	Tập Lồi (Convex Sets)	7
2.1.2	Nón Lồi và Đồng nhất hóa (Convex Cones and Homogenization)	8
2.1.2.1	Nón Phối cảnh và Đồng nhất hóa (Perspective Cones and Homogenization)	8
2.1.3	Hàm Lồi (Convex Functions)	10
2.1.4	Các Bài toán Tối ưu hóa (Optimization Problems)	11
2.1.4.1	Quy hoạch Lồi (Convex Optimization)	11
2.1.4.2	Quy hoạch Nón (Conic Optimization)	12
2.1.4.3	Quy hoạch Nguyên-Hỗn hợp (Mixed-Integer Programs)	12
2.2	Lý thuyết Đồ thị và Bài toán Đường đi Ngắn nhất	13
2.2.1	Cơ sở Lý thuyết Đồ thị	13
2.2.1.1	Các định nghĩa cơ bản	13
2.2.1.2	Tối ưu hóa tổ hợp trên đồ thị	15

2.2.2	Bài toán Đường đi Ngắn nhất	16
2.2.2.1	Định nghĩa và các thuật toán cổ điển . .	16
2.2.2.2	Mô hình dòng mạng cho bài toán đường đi ngắn nhất	17
2.3	Phân hoạch Lỗi	18
2.3.1	Định nghĩa	18
2.3.2	Phân loại Phương pháp Phân hoạch Lỗi	19
2.3.2.1	Phân hoạch Lỗi Chính xác	19
2.3.2.2	Phân hoạch Lỗi Xấp xỉ	20
2.4	Sinh Quỹ đạo Kinodynamic	21
2.4.1	Bài toán Lập kế hoạch Kinodynamic	22
2.4.2	Tham số hóa Quỹ đạo bằng Đường cong Bézier .	23
2.4.2.1	Định nghĩa	23
2.4.2.2	Các tính chất liên quan đến lập kế hoạch quỹ đạo	24
2.4.3	Ràng buộc Độ trơn tại Điểm chuyển tiếp	25
2.5	Bài toán Điều khiển Tối ưu (Optimal Control Problem - OCP)	27
2.6	Phương pháp Bắn Nhiều Lần Trực tiếp	30
3	Các nghiên cứu liên quan	34
3.1	Phương pháp hoạch định dựa trên đồ thị	34
3.2	Phương pháp hoạch định dựa trên lấy mẫu	35
3.3	Tối ưu hóa quỹ đạo trực tiếp	36
3.4	Tối ưu hóa quỹ đạo và Điều khiển tối ưu (Optimal Control)	37
3.4.1	Các phương pháp dựa trên Tối ưu hóa quỹ đạo .	37
3.4.2	Áp dụng Điều khiển tối ưu (OCP) và Multiple Shooting	38
4	Phương pháp đề xuất	39

4.1	Mô hình hoạch định chuyển động	39
4.1.1	Mô hình toán học	40
4.1.2	Tham số hóa quỹ đạo dựa trên tọa độ đường đi	41
4.2	Phân hoạch không gian an toàn	42
4.2.1	Phân hoạch Lỗi Xấp xỉ (ACD) của Đa giác	43
4.2.1.1	Các Khái niệm Cơ bản trong phân hoạch Lỗi	43
4.2.1.2	Ý tưởng và Chi tiết Thuật toán	44
4.2.1.3	Các Thước đo Độ lõm	46
4.2.2	Iterative Regional Inflation by Semi-Definite Programming (IRIS)	47
4.2.2.1	Ý tưởng và Chi tiết Thuật toán	47
4.2.2.2	Đặc tính của Thuật toán	52
4.2.3	Visibility Clique Cover (VCC)	53
4.2.3.1	Ý tưởng và Chi tiết Thuật toán	53
4.3	Hướng tiếp cận 1: Hoạch định chuyển động bằng Graph of Convex Sets (GCS)	55
4.3.1	Đồ thị của các Tập Lỗi (Graphs of Convex Sets)	55
4.3.1.1	Công thức Đường đi Ngắn nhất trong Graph of Convex Sets	55
4.3.1.2	Phân tích Độ phức tạp	59
4.3.1.3	Mô hình chi tiết các đỉnh và cạnh	60
4.3.1.4	Đặc tả hàm chi phí	61
4.3.1.4.a	Độ dài Đường đi Hình học	61
4.3.1.4.b	Chi phí Quỹ đạo Động lực học	62
4.3.1.5	Xây dựng Bài toán Quy hoạch Lỗi Nguyên Hỗn hợp thông qua Nối lỏng Phối cảnh	63
4.3.1.6	Lỗi hóa Bài toán GCS thông qua Kỹ thuật RLT	65

4.3.1.6.a	Cơ sở Lý thuyết: Tính Đối ngẫu giữa Nón Phối cảnh và Bất đẳng thức Hợp lệ	66
4.3.1.6.b	Xây dựng các Ràng buộc Lỗi . . .	67
4.3.1.6.c	Công thức MICP Hoàn chỉnh . . .	68
4.4	Hướng tiếp cận 2: Hoạch định chuyển động sử dụng Optimal Control và Multiple Shooting	70
4.4.1	Bài toán gốc cần giải	71
4.4.2	Phân hoạch không gian tự do và xây dựng đồ thị	72
4.4.3	Biến quyết định của mô hình	73
4.4.3.1	Biến rời rạc: chọn đường đi trên đồ thị .	73
4.4.3.2	Biến liên tục cục bộ trên mỗi vùng . . .	73
4.4.3.3	Biến liên kết tại điểm chuyển tiếp giữa hai vùng	74
4.4.4	Ràng buộc network flow cho phần rời rạc	74
4.4.5	Mô hình động lực học cục bộ theo direct multiple shooting	75
4.4.5.1	Tham số hóa tín hiệu điều khiển trên từng vùng	75
4.4.5.2	Bài toán giá trị đầu cực bộ trên từng vùng	76
4.4.5.3	Ràng buộc defect của multiple shooting	76
4.4.6	Ràng buộc an toàn hình học	77
4.4.6.1	Cách 1: Xấp xỉ qua lưới hữu hạn điểm (hard constraint)	77
4.4.6.2	Cách 2: Hàm cản logarit (log-barrier) . .	78
4.4.7	Ràng buộc ghép nối	79
4.4.7.1	Ghép nối tại điểm chuyển tiếp giữa hai vùng kề nhau	79
4.4.7.2	Điều kiện biên tại nguồn và đích	80

4.4.8	Hàm mục tiêu	80
4.4.9	Mô hình tổng hợp	82
4.4.10	Phân tách cấu trúc bài toán thành bốn lớp	83
4.4.11	Phân loại bài toán	84
4.4.11.1	So sánh với GCS-Bézier	86
5	Thực nghiệm và mô phỏng	88
5.1	Thiết lập thực nghiệm	88
5.1.1	Môi trường phần cứng và phần mềm	89
5.1.2	Chỉ số đánh giá	89
5.1.3	Các kịch bản thử nghiệm	90
5.1.3.1	Kịch bản A — Môi trường 2D đơn giản	91
5.1.3.2	Kịch bản B — Mê cung	91
5.1.3.3	Kịch bản C — Xe nonholonomic kiểu Du- bins	92
5.1.4	Tham số bài toán	92
5.1.5	Phương pháp đối sánh	93
5.2	Kết quả thực nghiệm	93
5.3	Kết quả và Phân tích	94
5.3.1	Phân tích Môi trường 2D đơn giản: Tác động của Phân hoạch lỗi	94
5.3.1.1	Hiệu suất tính toán và Khả năng đáp ứng thời gian thực	98
5.3.1.2	Chất lượng Quỹ đạo và Sự đánh đổi giữa các Mục tiêu Tối ưu	99
5.3.2	Phân tích Thực nghiệm trên Môi trường Mê cung Phức tạp	100
5.3.2.1	Phân tích Hiệu suất Tính toán và Chi phí Tối ưu	100

5.3.2.2	Phân tích Đặc điểm Quỹ đạo và Tính Tối ưu Toàn cục	102
5.4	Mô phỏng	103
6	Kết luận	104
6.1	Kết luận	104
6.2	Hướng nghiên cứu trong tương lai	105
	Tài liệu tham khảo	108
	Appendix A Phụ lục: Chi tiết chiến lược giải bài toán	
	MIOCP/MINLP	112
A.1	Chiến lược giải bài toán MIOCP/MINLP	112

Danh mục hình ảnh

Hình 1.1	Boston Dynamics' Atlas và Amazon's Robin	2
Hình 2.1	Ví dụ về Tập Lỗi	7
Hình 2.2	Ví dụ về Bao Lỗi	8
Hình 2.3	Đồ thị vô hướng và đồ thị có hướng có trọng số	14
Hình 2.4	Ví dụ bài toán đường đi ngắn nhất	16
Hình 2.5	Điều kiện bảo toàn dòng Kirchhoff	18
Hình 2.6	Một vài phương pháp giải bài toán điều khiển tối ưu	28
Hình 4.1	Ví dụ ACD	45
Hình 4.2	Mô tả quá trình đệ quy	45
Hình 4.3	Trường hợp không đảm bảo tính đơn điệu của ACD	46
Hình 4.4	Độ lõm Lai (Hybrid Concavity – H-Concavity)	47
Hình 4.5	Siêu phẳng Phân cách	48
Hình 4.6	Ellipsoid Nội tiếp Cực đại	50
Hình 4.7	Vòng lặp IRIS	51
Hình 4.8	Độ phức tạp thời gian của IRIS	52
Hình 4.9	Tổng quan về Quy trình VCC	53

Hình 4.10	Minh họa quy trình hoạch định quỹ đạo dựa trên khung làm việc GCS	56
Hình 4.11	Minh họa bài toán SPP in GCS	58
Hình 4.12	Quy trình tổng quan OCP+MMS	70
Hình 5.1	So sánh các phương pháp phân hoạch	95
Hình 5.2	So sánh các quỹ đạo chuyển động cho từng phương pháp phân hoạch	97
Hình 5.3	Quỹ đạo chuyển động cho 1 số mê cung	101

Danh mục bảng biểu

Bảng 2.1	So sánh các thuật toán cổ điển cho bài toán đường đi ngắn nhất	17
Bảng 5.1	Stack phần mềm cho hai phương pháp.	89
Bảng 5.2	Tham số chính của các thực nghiệm.	93
Bảng 5.3	So sánh hiệu năng giữa các phương pháp lập kế hoạch quỹ đạo	96
Bảng 5.4	Thống kê kết quả thực nghiệm trên 10 mẫu mê cung ngẫu nhiên	101
Bảng 6.1	Kế hoạch chi tiết Đồ án Tốt nghiệp (15 tuần) . . .	107

Chương 1

Giới thiệu

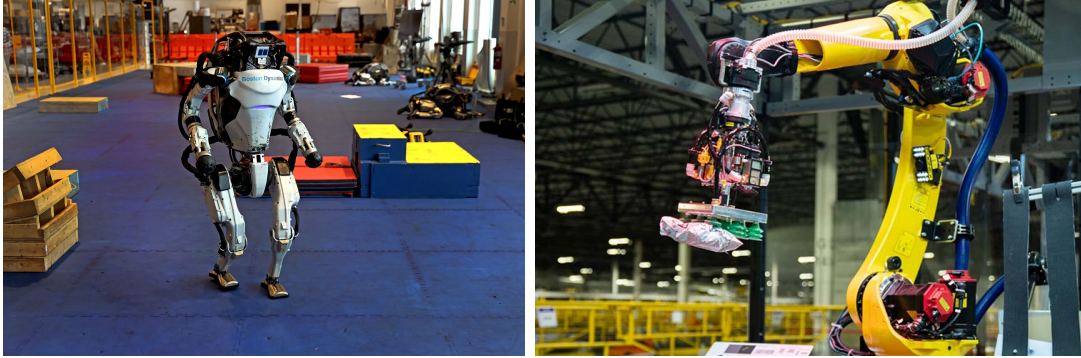
Trong chương 1, tổng quan, mục tiêu và các mục đích của đề tài nghiên cứu được trình bày. Bố cục của báo cáo cũng được giới thiệu.

1.1 Động lực và Thách thức Kỹ thuật

Việc đưa ra các quyết định tối ưu trong thời gian thực là một nền tảng cốt lõi của kỹ thuật hiện đại. Trong nhiều hệ thống phức tạp, đặc biệt là trong lĩnh vực robot tiên tiến, yêu cầu kết hợp chặt chẽ giữa **các quyết định rời rạc** và **điều khiển liên tục** trở nên hết sức rõ ràng.

- **Robot hình người (ví dụ: Atlas của Boston Dynamics):** Việc di chuyển trên địa hình phức tạp, chẳng hạn như các bài toán parkour, đòi hỏi robot phải đồng thời lựa chọn vị trí đặt chân (quyết định rời rạc) và tính toán quỹ đạo chuyển động cho trọng tâm cũng như các chi (điều khiển liên tục). Hiện nay, nhiều phương pháp vẫn phụ thuộc vào việc “lập trình thủ công” các quyết định rời rạc, điều này làm giảm đáng kể mức độ tự chủ của robot khi hoạt động trong những môi trường mới hoặc không xác định [2].
- **Robot công nghiệp (ví dụ: Robin của Amazon):** Hiệu quả của quá trình phân loại hàng hóa phụ thuộc vào việc tối ưu đồng thời thứ tự lựa chọn các thùng chứa (rời rạc) và quỹ đạo chuyển động của

cánh tay robot (liên tục) [3]. Chỉ cần cải thiện 1% tốc độ vận hành thông qua tối ưu hóa tốt hơn cũng có thể tương đương với việc xử lý thêm hàng triệu kiện hàng mỗi năm.



Hình 1.1: ***Bên trái:** Robot Atlas của Boston Dynamics thực hiện parkour, yêu cầu lập kế hoạch bước chân rời rạc và tối ưu quỹ đạo liên tục. **Bên phải:** Cánh tay robot Robin của Amazon phân loại hàng hóa, trong đó việc lựa chọn thùng chứa rời rạc và quỹ đạo chuyển động liên tục cần được tối ưu đồng thời.*

Tuy nhiên, việc giải quyết đồng thời các bài toán kết hợp này vẫn là một thách thức mở. Trong thực tế, các phương pháp hiện nay thường phải dựa vào sự can thiệp thủ công của con người hoặc chấp nhận các nghiệm dưới mức tối ưu.

Về mặt thuật toán, việc lựa chọn phương pháp lập kế hoạch chuyển động buộc các nhà nghiên cứu phải đánh đổi giữa nhiều yếu tố mâu thuẫn nhau, bao gồm số chiều của không gian trạng thái, các ràng buộc động học–động lực học và tính đầy đủ của thuật toán.

- **Tối ưu hóa quỹ đạo:** Các phương pháp này có ưu thế trong việc xử lý động lực học của robot và các bài toán có không gian trạng thái nhiều chiều. Tuy nhiên, khi môi trường chứa nhiều vật cản khiến bài toán trở nên phi lồi, chúng thường dễ rơi vào **cực tiểu cục bộ**, dẫn đến việc không tìm được quỹ đạo tránh và chạm [4].
- **Các phương pháp lập kế hoạch dựa trên lấy mẫu:** Để khắc phục hạn chế trên, cộng đồng robot học thường sử dụng các phương

pháp lấy mẫu như RRT hoặc PRM nhờ vào tính “đầy đủ theo xác suất” (probabilistic completeness). Tuy nhiên, cái giá phải trả là các quỹ đạo thu được thường **kém tối ưu**, và việc áp đặt các **ràng buộc vi phân liên tục** lên các mẫu rời rạc vẫn là một bài toán rất khó [5].

Lập trình lồi hỗn hợp số nguyên (Mixed-Integer Convex Programming – MICP) và đặc biệt là cách tiếp cận **Đồ thị các tập hợp lồi (Graph of Convex Sets - GCS)** đã nổi lên như một hướng tiếp cận đầy hứa hẹn nhằm thu hẹp khoảng cách giữa hai nhóm phương pháp trên. GCS cho phép diễn đạt bài toán chuyển động dưới dạng các tập lồi, kết hợp được tính đầy đủ của các thuật toán dựa trên đồ thị với khả năng đảm bảo **tính tối ưu toàn cục**. Tuy vậy, các cách tiếp cận GCS thuần túy dựa trên hình học thường tồn tại hạn chế trong việc sinh ra quỹ đạo mượt mà và dễ gặp hiện tượng rô-bốt “đi xuyên” (vi phạm) qua biên của các vùng không gian an toàn tại các điểm chuyển tiếp do chưa xử lý triệt để bài toán động lực học theo thời gian.

Mục tiêu của nghiên cứu này là khắc phục các hạn chế trên. Thay vì chỉ tìm quỹ đạo hình học, chúng tôi đề xuất mở rộng GCS thành một bài toán **Điều khiển tối ưu (Optimal Control Problem - OCP)** và giải quyết thông qua phương pháp **Multiple Shooting**. Đồng thời, để đảm bảo an toàn tuyệt đối và ngăn chặn hiện tượng đi xuyên biên, chúng tôi tích hợp thêm **hàm phạt logarit (log-barrier)** vào hàm mục tiêu. Cách tiếp cận này hứa hẹn mang lại một quỹ đạo tối ưu toàn cục, khả thi về mặt động lực học và tuân thủ nghiêm ngặt các giới hạn không gian, bảo đảm hiệu suất và tính an toàn cao cho hệ thống rô-bốt.

1.2 Mục tiêu

Mục tiêu trọng tâm của nghiên cứu này là đề xuất và triển khai một phương pháp tiếp cận mới dựa trên Đồ thị các Tập Lồi (Graphs of Convex Sets -

GCS) nhằm giải quyết bài toán hoạch định chuyển động cho robot trong môi trường chứa vật cản. Nghiên cứu hướng đến việc khắc phục những hạn chế về tính tối ưu và khả năng xử lý các ràng buộc động lực học phức tạp thường gặp ở các phương pháp lấy mẫu truyền thống bằng cách phân hoạch không gian tự do thành các vùng lồi hữu hạn và tham số hóa quỹ đạo sử dụng đường cong Bézier. Luận văn đặt mục tiêu thiết lập bài toán dưới dạng tối ưu hóa hỗn hợp nguyên, sau đó áp dụng kỹ thuật nới lỏng lồi (convex relaxation) chặt chẽ để chuyển đổi thành bài toán lồi chuẩn, nhằm đảm bảo tìm được quỹ đạo tối ưu toàn cục với thời gian tính toán hiệu quả. Cuối cùng, nghiên cứu tập trung chứng minh tính vượt trội của phương pháp GCS so với các thuật toán dựa trên phương pháp lấy mẫu như PRM hay RRT thông qua việc giảm thiểu thời gian truy vấn và rút ngắn độ dài quỹ đạo trên các hệ thống có số chiều cao.

1.3 Phạm vi nghiên cứu

Phạm vi của luận văn bao hàm việc khảo sát và hệ thống hóa các kiến thức nền tảng về giải tích lồi, tối ưu hóa nón, lý thuyết đồ thị và bài toán đường đi ngắn nhất để làm cơ sở xây dựng GCS framework. Nghiên cứu tập trung sâu vào các phương pháp phân hoạch không gian tự do như Phân hoạch lồi xấp xỉ (ACD), kỹ thuật Iterative Regional Inflation by Semi-Definite Programming (IRIS) và Visibility Clique Cover (VCC) để xây dựng đồ thị tập các vùng an toàn. Trong khía cạnh động lực học, luận văn giới hạn việc mô hình hóa các ràng buộc vi phân liên tục bao gồm vận tốc, gia tốc và độ trơn (jerk) thông qua việc tận dụng tính chất bao lồi và các đạo hàm của đường cong Bézier. Các thực nghiệm số và đánh giá hiệu suất được thực hiện trên một loạt các môi trường có độ phức tạp tăng dần, từ các vật cản 2D và mê cung đến các hệ thống động lực học phức tạp như thiết bị bay quadrotor và tay máy 7 bậc tự do (7-DoF). Quá trình đánh giá được thực hiện dựa trên các chỉ số định lượng cụ thể như thời gian giải (solve time),

chi phí quỹ đạo (path cost) và cận sai số tối ưu (optimality gap).

1.4 Cấu trúc luận văn

Luận văn này được tổ chức thành sáu chương, cụ thể như sau:

- **Chương 1** trình bày tổng quan về đề tài, bao gồm động lực nghiên cứu, mục tiêu cũng như phạm vi của luận văn.
- **Chương 2** tập trung giới thiệu các kiến thức nền tảng cần thiết cho việc hiểu và phát triển phương pháp đề xuất, bao gồm tối ưu hóa lồi, tối ưu hóa hỗn hợp số nguyên, lý thuyết đồ thị, bài toán điều khiển tối ưu (OCP) và phương pháp Multiple Shooting.
- **Chương 3** tổng hợp và phân tích các công trình nghiên cứu liên quan nhằm làm rõ bối cảnh nghiên cứu, cũng như những hạn chế còn tồn tại của các phương pháp hiện nay.
- **Chương 4** trình bày phương pháp đề xuất, bao gồm hai hướng tiếp cận: hướng hoạch định theo quỹ đạo hình học với khuôn khổ GCS nguyên bản và hướng mô hình hóa bài toán thành OCP, kết hợp GCS với Multiple Shooting và hàm cản log-barrier.
- **Chương 5** trình bày thiết lập thực nghiệm, các kết quả đánh giá, so sánh hiệu năng giữa hai hướng tiếp cận đề xuất trên nhiều kịch bản, và kết quả mô phỏng trực quan.
- **Chương 6** tổng kết toàn bộ kết quả đạt được của luận văn, nêu ra các hạn chế còn tồn tại và đề xuất các hướng phát triển trong tương lai.

Chương 2

Kiến thức nền tảng

Để hỗ trợ xây dựng và phân tích luận án, chương hiện tại trình bày hệ thống các kiến thức nền tảng từ bốn lĩnh vực toán học liên ngành.

Đầu tiên, *giải tích lồi và tối ưu hóa* cung cấp ngôn ngữ đại số chính xác để mô tả các tập an toàn và các ràng buộc tối ưu hóa. Đặc biệt, kỹ thuật đồng nhất hóa (homogenization) và nón phối cảnh (perspective cone) là nền tảng để chuyển đổi các ràng buộc nhị phân trong bài toán GCS thành dạng lồi giải được hiệu quả bằng quy hoạch nón bậc hai (SOCP). Tiếp theo, *lý thuyết đồ thị* cung cấp cấu trúc tổ hợp cho GCS: bài toán tìm đường đi qua đồ thị các tập lồi được quy về bài toán dòng mạng (network flow), vốn sở hữu tính chất hoàn toàn đơn mô (Total Unimodularity - TU) giúp bài toán rời rạc LP tự động có nghiệm nguyên. Song song với đó, *đường cong Bézier* được sử dụng làm cơ chế tham số hóa quỹ đạo bên trong mỗi tập lồi: tính chất bao lồi của chúng cho phép quy ràng buộc an toàn liên tục về một tập hữu hạn các ràng buộc tuyến tính trên các điểm điều khiển, đồng thời bảo toàn tính lồi của toàn bộ bài toán tối ưu hóa. Cuối cùng, *phân hoạch lồi* (convex decomposition) là bước tiền xử lý chuyển đổi không gian tự do phi lồi thành tập hợp các vùng lồi phủ lấp, từ đó xây dựng nên đồ thị đầu vào cho GCS.

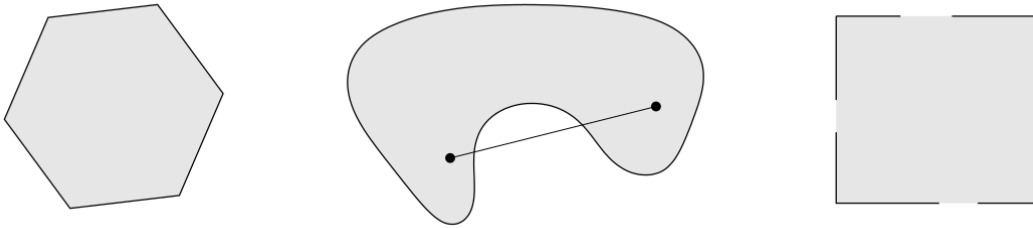
Phần còn lại của chương được tổ chức như sau. Mục 2.1 trình bày giải tích

lỗi, các lớp bài toán tối ưu nón và quy hoạch nguyên-hỗn hợp. Mục 2.2 xây dựng lý thuyết đồ thị và mô hình dòng mạng cho bài toán đường đi ngắn nhất. Mục 2.4 giới thiệu mô hình kinodynamic và các tính chất của đường cong Bézier. Mục 2.3 phân tích các phương pháp phân hoạch lỗi, phân biệt phương pháp chính xác và xấp xỉ. Mục 2.5 phát biểu bài toán Điều khiển Tối ưu dạng liên tục và thảo luận về tính không lỗi của không gian trạng thái trong bài toán tránh vật cản. Cuối cùng, Mục 2.6 trình bày phương pháp Bắn Nhiều Lần Trực tiếp, công cụ rời rạc hóa OCP thành bài toán NLP hữu hạn chiều có cấu trúc thừa khai thác được.

2.1 Giải tích Lỗi và Tối ưu hóa

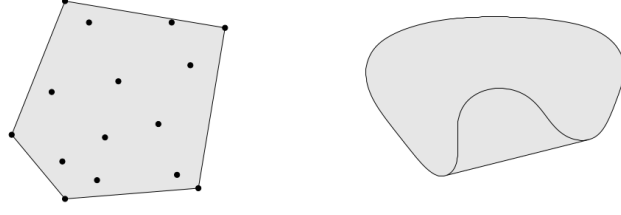
2.1.1 Tập Lỗi (Convex Sets)

Trước khi thảo luận về các bài toán tối ưu hóa, chúng ta cần thiết lập các đối tượng hình học cơ bản được sử dụng trong công trình này. Một tập hợp $C \subseteq \mathbb{R}^n$ được định nghĩa là *lồi* (convex) nếu, với bất kỳ hai điểm nào thuộc tập hợp, đoạn thẳng nối chúng nằm trọn vẹn bên trong tập hợp đó.



Hình 2.1: Các ví dụ về tập lồi và không lồi. Trái: Hình lục giác (bao gồm cả biên) là tập lồi. Giữa: Tập hình quả thận là không lồi, vì đoạn thẳng nối hai điểm đi ra ngoài tập hợp. Phải: Hình vuông chỉ chứa một phần biên không phải là tập lồi.[6]

Gắn liền mật thiết với tính lồi là khái niệm **bao lồi** (convex hull). Bao lồi của một tập hợp các điểm $\{x_1, x_2, \dots, x_m\} \subseteq \mathbb{R}^n$ là tập lồi nhỏ nhất chứa các điểm đó, được định nghĩa một cách hình thức là tập hợp của tất cả các tổ hợp lồi (convex combinations):



Hình 2.2: Bao lồi của một tập hợp điểm (phần được tô đậm).[6]

Trong bối cảnh quy hoạch chuyển động và luận văn này, chúng tôi tập trung chủ yếu vào hai loại tập lồi cụ thể:

- **Đa diện (Polyhedra):** Được định nghĩa là giao của một số hữu hạn các nửa không gian (halfspaces), biểu diễn dưới dạng $\mathcal{P} = \{x \in \mathbb{R}^n \mid Ax \leq b\}$. Khi một đa diện bị chặn (bounded), nó được gọi là một *khối đa diện* (polytope).
- **Ellipsoid:** Được định nghĩa là ảnh của một hình cầu đơn vị qua một phép biến đổi afin (affine transformation). Chúng thường được biểu diễn dưới dạng $\mathcal{E} = \{x \in \mathbb{R}^n \mid (x - c)^\top Q^{-1}(x - c) \leq 1\}$, trong đó Q là một ma trận xác định dương ($Q \succ 0$).

2.1.2 Nón Lồi và Đồng nhất hóa (Convex Cones and Homogenization)

Một tập con $\mathcal{K} \subseteq \mathbb{R}^n$ được gọi là *nón* (cone) nếu nó đóng dưới phép nhân vô hướng dương (tức là, $\theta x \in \mathcal{K}$ với mọi $x \in \mathcal{K}, \theta \geq 0$). Một tập hợp được gọi là *nón lồi* (convex cone) nếu nó vừa là tập lồi vừa là nón. Hai khái niệm quan trọng liên quan đến nón lồi là cốt yếu cho công trình này: nón phối cảnh (được xây dựng thông qua quá trình đồng nhất hóa) và nón đối ngẫu.

2.1.2.1 Nón Phối cảnh và Đồng nhất hóa (Perspective Cones and Homogenization)

Đồng nhất hóa (Homogenization) là một kỹ thuật mạnh mẽ trong giải tích lồi, được sử dụng để biến đổi một tập lồi tổng quát thành một nón trong

không gian có số chiều cao hơn. Quá trình này thực chất là “nâng” (lifts) một tập $\mathcal{S} \subseteq \mathbb{R}^n$ vào không gian $\mathbb{R}^{n+1}$ bằng cách giới thiệu thêm một chiều phụ y . Tập đồng nhất hóa $\tilde{\mathcal{S}}$ được định nghĩa là:

$$\tilde{\mathcal{S}} := \{(x, y) \in \mathbb{R}^{n+1} \mid y > 0, x \in y\mathcal{S}\}. \quad (2.1)$$

Điểm mấu chốt là $\tilde{\mathcal{S}}$ là tập lồi khi và chỉ khi tập gốc $\mathcal{S}$ là lồi. Phép biến đổi này cho phép biểu diễn các tổ hợp lồi bên trong một cấu trúc nón, điều này đặc biệt hữu ích để chuyển đổi các ràng buộc afin sang dạng nón trong tối ưu hóa.

Khi một tập lồi $\mathcal{C}$ được mô tả dưới dạng nón tiêu chuẩn, việc đồng nhất hóa nó về mặt tính toán là rất trực tiếp. Quan sát rằng với $y > 0$, tập $y\mathcal{C}$ thỏa mãn $\{yx \mid Ax + b \in \mathcal{K}\} = \{x' \mid Ax' + by \in \mathcal{K}\}$, ta thu được *nón phối cảnh* (perspective cone):

$$\tilde{\mathcal{C}} = \{(x, y) \in \mathbb{R}^{n+1} \mid y > 0, Ax + by \in \mathcal{K}\}. \quad (2.2)$$

Một ý nghĩa thực tiễn quan trọng nằm ở bao đóng (closure) của tập này, $\text{cl}(\tilde{\mathcal{C}}) = \{(x, y) \mid y \geq 0, Ax + by \in \mathcal{K}\}$. Bao đóng này mở rộng định nghĩa để bao gồm cả biên nơi $y = 0$, điều này rất quan trọng trong việc mô hình hóa các ràng buộc logic trong quy hoạch nguyên-hỗn hợp.

Đối với một tập compact $\mathcal{C}$, khi vô hướng y tiến dần về 0, tập tỉ lệ $y\mathcal{C}$ sẽ co liên tục về gốc tọa độ. Do đó, trong giới hạn khi $y = 0$, điều kiện $(x, 0) \in \text{cl}(\tilde{\mathcal{C}})$ ngụ ý rằng x bắt buộc phải là vector không (giả sử $\mathcal{C}$ chứa gốc tọa độ và bị chặn). Hành vi hình học này hoạt động hiệu quả như một công tắc logic:

- Khi $y > 0$, biến x bị ràng buộc trong phiên bản tỉ lệ của tập $\mathcal{C}$.
- Khi $y = 0$, biến x bị ép về gốc tọa độ ($x = 0$).

Tính chất này được khai thác trực tiếp trong ứng dụng của chúng tôi đối

với Bài toán Đường đi Ngắn nhất trên Đồ thị các Tập Lồi (GCS). Trong mô hình Quy hoạch Lồi Nguyên-Hỗn hợp (MICP), biến kích hoạt nhị phân cho một cạnh đóng vai trò là hệ số tỉ lệ y . Khi một cạnh không hoạt động ($y = 0$), các đóng góp trạng thái liên quan bị ép về 0, và chi phí—được mô hình hóa qua hàm phối cảnh—sẽ tự nhiên triệt tiêu. Hàm phối cảnh $\tilde{\ell}_e$ được định nghĩa để trả về 0 khi cả $y = 0$ và các biến trạng thái bằng 0, đảm bảo rằng các thành phần không hoạt động sẽ không phát sinh chi phí trong hàm mục tiêu tối ưu.

2.1.3 Hàm Lồi (Convex Functions)

Một hàm số $f : \mathcal{D} \rightarrow \mathbb{R}$ là *lồi* (convex) nếu miền xác định $\mathcal{D} \subseteq \mathbb{R}^n$ là một tập lồi và nếu, với mọi $x, y \in \mathcal{D}$ và $\theta \in [0, 1]$, bất đẳng thức sau được thỏa mãn:

$$f(\theta x + (1 - \theta)y) \leq \theta f(x) + (1 - \theta)f(y). \quad (2.3)$$

Về mặt hình học, đoạn thẳng nối bất kỳ hai điểm nào trên đồ thị của hàm số đều nằm phía trên hoặc thuộc đồ thị đó. Một hàm số được gọi là *lồi chặt* (strictly convex) nếu bất đẳng thức này xảy ra dấu nhỏ hơn hẳn với $x \neq y$ và $\theta \in (0, 1)$. Ngược lại, f là *lõm* (concave) nếu $-f$ là lồi.

Tính chất giải tích quan trọng nhất của hàm lồi trong tối ưu hóa là bất kỳ cực tiểu địa phương nào cũng được đảm bảo là cực tiểu toàn cục.

2.1.4 Các Bài toán Tối ưu hóa (Optimization Problems)

2.1.4.1 Quy hoạch Lồi (Convex Optimization)

Một **Chương trình Lồi (Convex Program - CP)** được thiết lập như sau:

$$\text{minimize } f(x) \tag{2.4a}$$

$$\text{subject to } x \in \mathcal{C}, \tag{2.4b}$$

trong đó hàm mục tiêu f và tập chấp nhận được (feasible set) $\mathcal{C}$ đều là lồi. Một điểm x^{opt} là một *ng nghiệm tối ưu* nếu nó thuộc tập chấp nhận được và $f(x^{\text{opt}}) \leq f(x)$ với mọi $x \in \mathcal{C}$.

Mặc dù tính lồi về mặt lý thuyết không đảm bảo tính hiệu quả một cách tuyệt đối (ví dụ, kiểm tra tính không âm của đa thức là NP-khó mặc dù tập hợp này là một nón lồi), nhưng trong phạm vi luận văn này, chúng tôi sử dụng thuật ngữ “bài toán tối ưu lồi” gần như đồng nghĩa với “bài toán tối ưu giải được một cách hiệu quả”. Phần lớn các bài toán CP thực tế có thể được giải một cách đáng tin cậy bằng các phương pháp điểm trong.

2.1.4.2 Quy hoạch Nón (Conic Optimization)

Một **Chương trình Nón (Conic Program - KP)** là một lớp con của CP với hàm mục tiêu tuyến tính và các ràng buộc được định nghĩa bởi một nón lồi $\mathcal{K}$:

$$\text{minimize } c^\top x \tag{2.5a}$$

$$\text{subject to } Ax + b \in \mathcal{K}. \tag{2.5b}$$

Các lớp cụ thể của KP bao gồm:

1. **Quy hoạch Tuyến tính (LP):** $\mathcal{K}$ là orthant không âm.
2. **Quy hoạch Nón Bậc hai (SOCP):** $\mathcal{K}$ là tích của các nón bậc hai.
3. **Quy hoạch Bán xác định (SDP):** $\mathcal{K}$ là nón bán xác định.

Các lớp này đại diện cho một hệ thống phân cấp về khả năng mô hình hóa: $LP \subset SOCP \subset SDP$. Một lớp phổ biến khác là **Quy hoạch Toàn phương (Quadratic Program - QP)**, có hàm mục tiêu bậc hai và các ràng buộc đa diện. Mặc dù mọi bài toán QP đều có thể được thiết lập dưới dạng SOCP, nhưng các thuật toán tập hoạt động (active-set algorithms) chuyên biệt cho QP thường hiệu quả hơn.

2.1.4.3 Quy hoạch Nguyên-Hỗn hợp (Mixed-Integer Programs)

Quy hoạch Nguyên-Hỗn hợp (Mixed-Integer Programs - MIPs)

tổng quát hóa bài toán tối ưu bằng cách phân hoạch các biến thành vector liên tục $x \in \mathbb{R}^n$ và vector nguyên $z \in \mathbb{Z}^m$:

$$\text{minimize} \quad f(x, z) \tag{2.6a}$$

$$\text{subject to} \quad (x, z) \in \mathcal{C}, \tag{2.6b}$$

$$z \in \mathbb{Z}^m. \tag{2.6c}$$

Ràng buộc nguyên làm cho tập chấp nhận được trở nên rời rạc và không lồi, thường khiến các bài toán MIP trở thành NP-hard. Các phương pháp tiêu chuẩn dựa trên tính khả vi không thể áp dụng được.

Để giải quyết MIP, phương pháp **nới lỏng** (relaxation) thường được sử dụng (ví dụ: thay thế $z_j \in \{0, 1\}$ bằng $0 \leq z_j \leq 1$). Giá trị tối ưu của bài toán nới lỏng cung cấp một cận dưới cho nghiệm của MIP. Nếu nghiệm nới lỏng không thỏa mãn tính nguyên, các thuật toán như **Nhánh và Cận** (Branch-and-Bound) sẽ được sử dụng. Phương pháp này phân hoạch không gian tìm kiếm thành các vùng nhỏ hơn (các nhánh) một cách có hệ thống

để loại bỏ các giá trị phân số, từ đó duyệt cây tìm kiếm để tìm ra tối ưu toàn cục.

2.2 Lý thuyết Đồ thị và Bài toán Đường đi Ngắn nhất

2.2.1 Cơ sở Lý thuyết Đồ thị

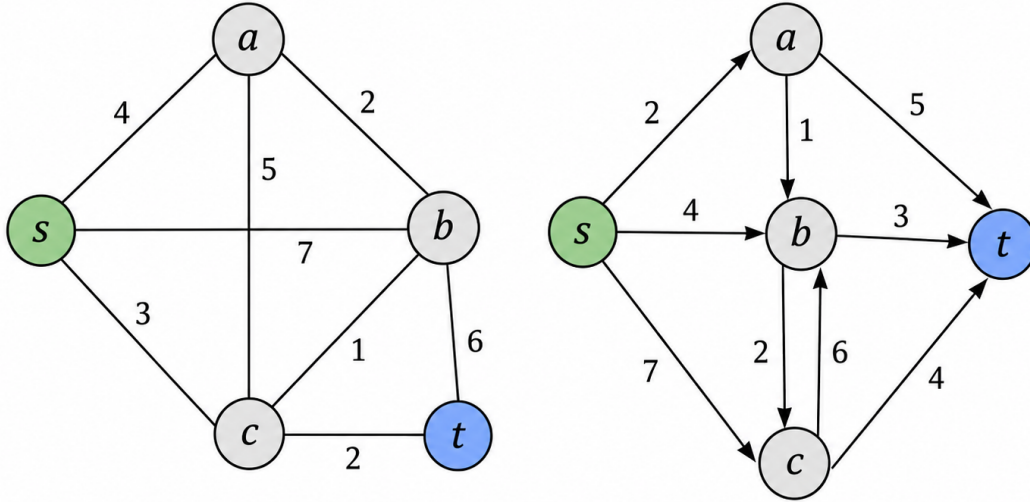
2.2.1.1 Các định nghĩa cơ bản

Lý thuyết đồ thị cung cấp ngôn ngữ toán học để biểu diễn các quan hệ cấu trúc. Dưới đây là hệ thống định nghĩa được sử dụng xuyên suốt chương này.

Định nghĩa 1 (Đồ thị có trọng số). Một *đồ thị có trọng số* là bộ ba $G = (\mathcal{V}, \mathcal{E}, c)$, trong đó $\mathcal{V}$ là tập hữu hạn các *đỉnh* (vertices), $\mathcal{E}$ là tập các *cạnh* (edges), và $c : \mathcal{E} \rightarrow \mathbb{R}$ là *hàm trọng số*. Tùy theo định nghĩa của $\mathcal{E}$, đồ thị được phân loại như sau:

- (i) **Đồ thị vô hướng**: mỗi cạnh là một cặp không có thứ tự, $\{u, w\} \in \mathcal{E}$, $u \neq w$;
- (ii) **Đồ thị có hướng**: mỗi cạnh là một cặp có thứ tự, $(u, w) \in \mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$.

Khi hàm trọng số không liên quan, ta viết gọn $G = (\mathcal{V}, \mathcal{E})$.



Hình 2.3: Minh họa (**trái**) đồ thị vô hướng và (**phải**) đồ thị có hướng có trọng số. Mũi tên thể hiện chiều cạnh; số trên cạnh là giá trị $c(e)$.

Định nghĩa 2 (Bậc đỉnh trong đồ thị có hướng). Với $v \in \mathcal{V}$ trong đồ thị có hướng $G = (\mathcal{V}, \mathcal{E})$, *bậc ra* và *bậc vào* của v được định nghĩa lần lượt là

$$\deg^+(v) = |\{w \in \mathcal{V} \mid (v, w) \in \mathcal{E}\}|, \quad \deg^-(v) = |\{u \in \mathcal{V} \mid (u, v) \in \mathcal{E}\}|. \quad (2.7)$$

Định nghĩa 3 (Đường đi, Chi phí, và Chu trình). Cho $G = (\mathcal{V}, \mathcal{E}, c)$. Một *đường đi* (path) từ v_0 đến v_k là dãy xen kẽ hữu hạn

$$P = (v_0, e_1, v_1, e_2, \dots, e_k, v_k), \quad e_i = (v_{i-1}, v_i) \in \mathcal{E}, \quad (2.8)$$

trong đó tất cả các đỉnh $v_0, v_1, \dots, v_k$ là *phân biệt*. *Chi phí của đường đi* được xác định là

$$c(P) = \sum_{i=1}^k c(e_i). \quad (2.9)$$

Một *chu trình* (cycle) là đường đi thỏa $v_0 = v_k$ với $k \geq 1$.

Định nghĩa 4 (Đồ thị con). Bộ đôi $H = (\mathcal{W}, \mathcal{F})$ là một *đồ thị con* của $G = (\mathcal{V}, \mathcal{E})$ nếu

$$\mathcal{W} \subseteq \mathcal{V}, \quad \mathcal{F} \subseteq \mathcal{E}, \quad (u, w) \in \mathcal{F} \Rightarrow u, w \in \mathcal{W}. \quad (2.10)$$

Các bài toán tối ưu hóa tổ hợp thường được đặt trong khuôn khổ: tìm một đồ thị con $H \in \mathcal{H}$ trong một lớp khả thi $\mathcal{H}$ sao cho tối thiểu hóa một hàm chi phí liên quan.

2.2.1.2 Tối ưu hóa tổ hợp trên đồ thị

Cho đồ thị $G = (\mathcal{V}, \mathcal{E}, c)$ và lớp đồ thị con khả thi $\mathcal{H}$. Bài toán tối ưu hóa tổ hợp tổng quát là

$$\min_{(\mathcal{W}, \mathcal{F}) \in \mathcal{H}} \sum_{v \in \mathcal{W}} c_v + \sum_{e \in \mathcal{F}} c_e. \quad (2.11)$$

Để áp dụng các kỹ thuật lập trình toán học như Branch-and-Bound, bài toán được chuyển sang dạng *Quy hoạch Tuyến tính Nguyên* (Integer Linear Programming, ILP) bằng cách tham số hóa qua *vectơ liên thuộc* (incidence vector)

$$\mathbf{y} = (y_v)_{v \in \mathcal{V}} \oplus (y_e)_{e \in \mathcal{E}} \in \{0, 1\}^{|\mathcal{V}| + |\mathcal{E}|}, \quad y_v = \mathbf{1}[v \in \mathcal{W}], \quad y_e = \mathbf{1}[e \in \mathcal{F}]. \quad (2.12)$$

Về mặt hình học, tập các đồ thị con hợp lệ tương ứng với tập điểm nguyên trong một đa diện $\mathcal{Y} \subseteq [0, 1]^{|\mathcal{V}| + |\mathcal{E}|}$. Bài toán (2.11) được tái biểu diễn là

$$\begin{aligned} \min_{\mathbf{y}} \quad & \mathbf{c}^\top \mathbf{y} \\ \text{s.t.} \quad & \mathbf{y} \in \mathcal{Y} \cap \{0, 1\}^{|\mathcal{V}| + |\mathcal{E}|}, \end{aligned} \quad (2.13)$$

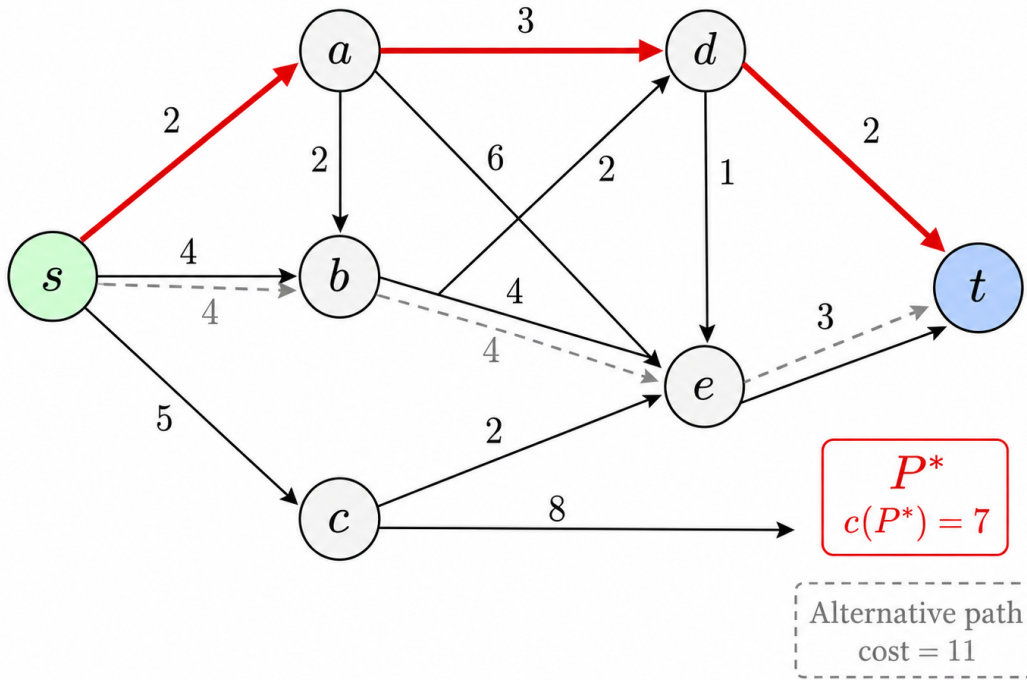
trong đó $\mathbf{c} \in \mathbb{R}^{|\mathcal{V}| + |\mathcal{E}|}$ là vectơ chi phí ghép từ $\{c_v\}_{v \in \mathcal{V}}$ và $\{c_e\}_{e \in \mathcal{E}}$. Với các lớp bài toán đặc biệt—điển hình là bài toán dòng mạng—ma trận ràng buộc của $\mathcal{Y}$ sở hữu tính chất Hoàn toàn Đơn mô, khiến bài toán nói lỏng LP của (2.13) tự động có nghiệm nguyên.

2.2.2 Bài toán Đường đi Ngắn nhất

2.2.2.1 Định nghĩa và các thuật toán cổ điển

Định nghĩa 5 (Bài toán Đường đi Ngắn nhất). Cho đồ thị có hướng có trọng số $G = (\mathcal{V}, \mathcal{E}, c)$, đỉnh nguồn $s \in \mathcal{V}$ và đỉnh đích $t \in \mathcal{V}$. Gọi $\mathcal{P}(s, t)$ là tập tất cả các đường đi từ s đến t theo Định nghĩa 3. *Bài toán Đường đi Ngắn nhất* (Shortest Path Problem, SPP) là

$$P^* = \arg \min_{P \in \mathcal{P}(s, t)} c(P) = \arg \min_{P \in \mathcal{P}(s, t)} \sum_{e \in P} c(e). \quad (2.14)$$



Hình 2.4: Ví dụ bài toán đường đi ngắn nhất trên đồ thị có hướng có trọng số. Đường đậm là đường đi tối ưu P^* ; số trên mỗi cạnh là trọng số $c(e)$.

Các thuật toán cổ điển giải SPP được phân loại theo tính chất của trọng số cạnh; Bảng 2.1 tóm tắt điều kiện áp dụng và độ phức tạp tiệm cận.

Bảng 2.1: So sánh các thuật toán cổ điển cho bài toán đường đi ngắn nhất

Thuật toán	Điều kiện trọng số	Độ phức tạp	Loại
Dijkstra	$c_e \geq 0$	$\mathcal{O}((\mathcal{V} + \mathcal{E}) \log \mathcal{V})$	Đơn nguồn
Bellman–Ford	$c_e \in \mathbb{R}$, NNWC	$\mathcal{O}(\mathcal{V} \mathcal{E})$	Đơn nguồn
Floyd–Warshall	$c_e \in \mathbb{R}$, NNWC	$\mathcal{O}(\mathcal{V} ^3)$	Đa nguồn

2.2.2.2 Mô hình dòng mạng cho bài toán đường đi ngắn nhất

SPP trên đồ thị có hướng $G = (\mathcal{V}, \mathcal{E})$ với $c_{uv} \geq 0$ được mô hình hóa như bài toán *Dòng Chi phí Tối thiểu* với một đơn vị dòng. Định nghĩa biến quyết định nhị phân $x_{uv} \in \{0, 1\}$ cho mỗi $(u, v) \in \mathcal{E}$, trong đó $x_{uv} = 1$ khi và chỉ khi cạnh (u, v) thuộc đường đi tối ưu. Bài toán ILP tương ứng là:

$$\min_{\mathbf{x}} \sum_{(u,v) \in \mathcal{E}} c_{uv} x_{uv} \quad (2.15a)$$

$$\text{s.t.} \quad \sum_{w \in \mathcal{V}^+(u)} x_{uw} - \sum_{w \in \mathcal{V}^-(u)} x_{wu} = \begin{cases} 1 & \text{nếu } u = s, \\ -1 & \text{nếu } u = t, \\ 0 & \text{còn lại,} \end{cases} \quad \forall u \in \mathcal{V} \quad (2.15b)$$

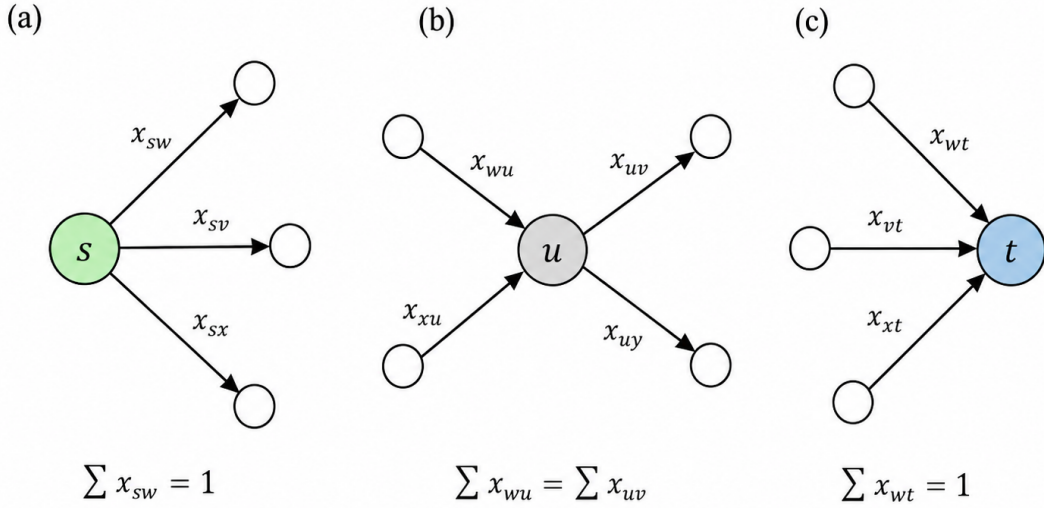
$$\sum_{w \in \mathcal{V}^-(u)} x_{wu} \leq 1, \quad \forall u \in \mathcal{V} \setminus \{s, t\} \quad (2.15c)$$

$$x_{uv} \geq 0, \quad \forall (u, v) \in \mathcal{E}. \quad (2.15d)$$

Ký hiệu $\mathcal{V}^+(u) = \{w \mid (u, w) \in \mathcal{E}\}$ và $\mathcal{V}^-(u) = \{w \mid (w, u) \in \mathcal{E}\}$ lần lượt là tập đỉnh kế tiếp và đỉnh tiền nhiệm của u .

Ràng buộc (2.15b) thể hiện *định luật bảo toàn dòng Kirchhoff*: một đơn vị dòng rời s , đến t , và được bảo toàn tại các đỉnh trung gian. Ràng buộc (2.15c) giới hạn dòng vào mỗi đỉnh trung gian không vượt quá một đơn vị, đảm bảo cấu trúc *đường đi đơn giản* (không có đỉnh lặp). Với $c_{uv} \geq 0$, chu trình luôn không tối ưu nên không cần ràng buộc loại trừ chu trình tường minh.

Ưu điểm then chốt của mô hình (2.15) nằm ở tính chất đại số của ma trận



Hình 2.5: Minh họa điều kiện bảo toàn dòng (Phương trình (2.15b)) tại **(a)** đỉnh nguồn s , **(b)** đỉnh trung gian, và **(c)** đỉnh đích t .

ràng buộc thu được từ (2.15b). Ma trận liên thuộc đỉnh-cung (node-arc incidence matrix) này sở hữu tính chất *Hoàn toàn Đơn mô*. Theo lý thuyết tối ưu hóa tổ hợp, nếu ma trận ràng buộc là TU và vế phải là nguyên, thì mọi nghiệm khả thi cơ sở của bài toán nói lỏng LP đều là nguyên [7]. Do đó, mặc dù chỉ đặt ràng buộc $x_{uv} \geq 0$ trong (2.15d), bài toán LP nói lỏng của (2.15) vẫn cho nghiệm nhị phân hợp lệ, mà không cần sử dụng các phương pháp lập trình nguyên tốn kém.

2.3 Phân hoạch Lỗi

Phần này cung cấp tổng quan về phân hoạch lỗi, một kỹ thuật cơ bản được sử dụng rộng rãi trong hình học tính toán, mô phỏng vật lý, và đặc biệt là lập kế hoạch chuyển động robot.

2.3.1 Định nghĩa

Phân hoạch lỗi (Convex Decomposition) là quá trình chia một hình học phức tạp (ví dụ: đa giác hoặc đa diện không lồi) thành một tập hợp các **vùng con lồi** sao cho hợp của các vùng này khôi phục lại toàn bộ hình ban đầu và các vùng con không chồng lấn nhau, ngoại trừ biên chung.

Về mặt hình thức, với một miền hình học $P \subset \mathbb{R}^n$, một *phân hoạch lỗi* của P là tập hợp $\{P_1, P_2, \dots, P_k\}$ sao cho:

$$P = \bigcup_{i=1}^k P_i, \quad P_i \cap P_j = \partial P_i \cap \partial P_j \text{ với mọi } i \neq j,$$

và mỗi P_i là một tập lỗi. Mục tiêu là tìm phân hoạch này sao cho tối thiểu hóa số lượng các vùng lỗi, giảm thiểu phần chồng chéo, hoặc tối ưu chất lượng hình học của các vùng lỗi được tạo ra (tránh các vùng quá “mảnh” hoặc quá “dài”).

2.3.2 Phân loại Phương pháp Phân hoạch Lỗi

Trong lĩnh vực hình học tính toán và hoạch định chuyển động, các phương pháp phân hoạch lỗi thường được chia thành hai loại chính: **Phân hoạch Lỗi Chính xác (Exact Convex Decomposition – ECD)** và **Phân hoạch Lỗi Xấp xỉ (Approximate Convex Decomposition – ACD)**. Sự khác biệt giữa hai loại này phản ánh sự đánh đổi cơ bản giữa *tính chính xác hình học* và *hiệu quả tính toán*.

2.3.2.1 Phân hoạch Lỗi Chính xác

Phân hoạch Lỗi Chính xác (ECD) là quá trình chia một hình dạng không lỗi thành các **thành phần con hoàn toàn lỗi**, không chồng lấn, sao cho hợp của chúng tái tạo lại chính xác hình dạng ban đầu. Mục tiêu thông thường là tối thiểu hóa số lượng vùng lỗi.

Tuy nhiên, ECD gặp hai thách thức lớn:

- **Độ phức tạp tính toán cao:** Bài toán tìm phân hoạch lỗi tối thiểu đã được chứng minh là NP-hard đối với đa giác có lỗ, nên không tồn tại thuật toán hiệu quả cho các trường hợp tổng quát.
- **Tính khả dụng hạn chế:** Ngay cả khi tính toán được, ECD thường

tạo ra số lượng vùng rất lớn, khiến việc lưu trữ, xử lý và tối ưu hóa tiếp theo trở nên tốn kém.

Do đó, các phương pháp ECD chủ yếu mang tính lý thuyết và dùng làm chuẩn so sánh, trong khi hầu hết các ứng dụng thực tế trong robot và mô phỏng đều dựa vào các phương pháp xấp xỉ.

2.3.2.2 Phân hoạch Lỗi Xấp xỉ

Trước những giới hạn của ECD, các phương pháp **Phân hoạch Lỗi Xấp xỉ (ACD)** được phát triển để đạt sự cân bằng giữa độ chính xác và hiệu quả. ACD cho phép các thành phần không hoàn toàn lỗi, mà chỉ “gần lỗi” trong một mức dung sai lồi τ do người dùng xác định, hoặc chỉ bao phủ một phần không gian tự do. Triết lý cốt lõi của ACD là chấp nhận một **mức độ lồi nhỏ có kiểm soát** để đổi lấy:

- **Hiệu quả tính toán cao hơn:** Các thuật toán ACD, chẳng hạn như phương pháp của Lien và Amato [8], chạy nhanh hơn đáng kể so với ECD.
- **Giảm số lượng vùng:** Cho phép biểu diễn hình dạng với ít vùng lồi hơn, đơn giản hơn và dễ xử lý hơn.
- **Kiểm soát mức độ chi tiết:** Một số thuật toán ACD, chẳng hạn như phương pháp của Lien và Amato, cho phép người dùng điều chỉnh tham số lồi τ để cân bằng giữa độ chính xác và số lượng vùng lồi. Đối với phương pháp Visibility Clique Cover (VCC) [9], người dùng có thể chọn chỉ số bao phủ thể hiện mức độ bao phủ của các vùng lồi so với không gian tự do ban đầu.

Do những ưu điểm trên, ACD được ưu tiên áp dụng trong hầu hết các hệ thống robot thực tế, bao gồm cả khuôn khổ được đề xuất trong luận văn này. Cụ thể, phương pháp ACD tạo ra một tập hợp các vùng lồi phủ không

gian tự do, từ đó xây dựng đồ thị $\mathcal{G}$ cho bài toán GCS. Chất lượng của phân hoạch ảnh hưởng trực tiếp đến số lượng nút trong đồ thị và do đó đến độ phức tạp của bài toán tối ưu hóa – một phân hoạch tốt cần tạo ra đủ ít vùng để giữ cho bài toán gọn nhẹ, đồng thời đủ chi tiết để robot có thể đi qua các hành lang hẹp trong môi trường.

2.4 Sinh Quỹ đạo Kinodynamic

Lập kế hoạch chuyển động hình học (geometric motion planning) tìm kiếm một đường đi không va chạm trong không gian cấu hình của robot mà không xét đến các đặc tính vật lý của hệ thống. Tuy nhiên, trong thực tế, robot là một hệ cơ học mà trạng thái của nó chỉ có thể tiến triển theo thời gian theo các phương trình vi phân xác định được dẫn dắt bởi các đầu vào cơ cấu chấp hành. Một đường đi không va chạm về mặt hình học vẫn có thể không khả thi về mặt vật lý nếu nó đòi hỏi, chẳng hạn, sự thay đổi tức thời của vận tốc hoặc các lực vượt quá giới hạn bảo hòa của cơ cấu chấp hành. Lập kế hoạch chuyển động kinodynamic mở rộng bài toán hình học bằng cách yêu cầu quỹ đạo được lập kế hoạch phải đồng thời tránh vật cản, thỏa mãn các ràng buộc vi phân do động lực học hệ thống áp đặt, và tối ưu hóa một tiêu chí hiệu năng do người dùng chỉ định.

Mục này xây dựng bài toán lập kế hoạch kinodynamic dưới dạng toán học chặt chẽ (Mục 2.4.1), giới thiệu tham số hóa quỹ đạo đa thức bằng đường cong Bézier như một kỹ thuật để đưa bài toán về dạng hữu hạn chiều (Mục 2.4.2), và trình bày các ràng buộc liên tục đảm bảo chuyển động trơn qua các điểm chuyển tiếp giữa các đoạn quỹ đạo (Mục 2.4.3).

2.4.1 Bài toán Lập kế hoạch Kinodynamic

Xét một hệ thống robot có trạng thái $x(t) \in \mathcal{X} \subseteq \mathbb{R}^{n_x}$ và đầu vào điều khiển $u(t) \in \mathcal{U} \subseteq \mathbb{R}^{n_u}$ tiến triển theo phương trình vi phân thường

$$\dot{x}(t) = f(x(t), u(t)), \quad t \in [0, T], \quad (2.16)$$

trong đó $f: \mathbb{R}^{n_x} \times \mathbb{R}^{n_u} \rightarrow \mathbb{R}^{n_x}$ là ánh xạ động lực học của hệ và $T > 0$ là độ dài chân trời lập kế hoạch, có thể được cố định trước hoặc được coi là biến tối ưu. Vector trạng thái thường mã hóa các vị trí và vận tốc suy rộng của robot, và tùy theo mô hình động lực học có thể bao gồm thêm các đạo hàm bậc cao hơn.

Một quỹ đạo kinodynamic là một cặp $(x(\cdot), u(\cdot))$ xác định trên $[0, T]$ thỏa mãn phương trình động lực học (2.16) cùng với các yêu cầu sau.

1. **Điều kiện biên.** Quỹ đạo phải xuất phát từ trạng thái đầu được chỉ định và đến được vùng đích mong muốn:

$$x(0) = x_{\text{start}}, \quad x(T) \in \mathcal{X}_{\text{goal}}, \quad (2.17)$$

trong đó $\mathcal{X}_{\text{goal}} \subseteq \mathcal{X}$ là tập đích.

2. **Ràng buộc an toàn.** Quỹ đạo trạng thái phải nằm trong vùng không có vật cản $\mathcal{X}_{\text{free}} \subseteq \mathcal{X}$ tại mọi thời điểm:

$$x(t) \in \mathcal{X}_{\text{free}}, \quad \forall t \in [0, T]. \quad (2.18)$$

3. **Giới hạn cơ cấu chấp hành và động học.** Đầu vào điều khiển phải nằm trong tập khả thi của cơ cấu chấp hành:

$$u(t) \in \mathcal{U}, \quad \forall t \in [0, T]. \quad (2.19)$$

Trong nhiều bài toán thực tế, $\mathcal{U}$ được đặc trưng bởi các giới hạn tường minh lên vận tốc, gia tốc và độ giạt dọc theo quỹ đạo, ví dụ $\|\dot{x}(t)\| \leq v_{\max}, \|\ddot{x}(t)\| \leq a_{\max}$

Mục tiêu là tìm một quỹ đạo tối thiểu hóa phiếm hàm chi phí có dạng

$$J(x(\cdot), u(\cdot)) = \int_0^T L(x(t), u(t)) dt + E(x(T)), \quad (2.20)$$

trong đó L là running cost phạt, chẳng hạn, nỗ lực điều khiển hoặc độ lệch so với quỹ đạo tham chiếu, và E là terminal cost.

Có hai khó khăn cấu trúc phân biệt bài toán này với bài toán hình học thuần túy. Thứ nhất, ràng buộc an toàn (2.18) phải được thỏa mãn liên tục trên toàn bộ khoảng $[0, T]$, không chỉ tại một tập hữu hạn các điểm rời rạc, và $\mathcal{X}_{\text{free}}$ nói chung là phi lồi do sự hiện diện của vật cản. Thứ hai, phiếm hàm chi phí (2.20) và ràng buộc động lực học (2.16) có tính chất vô hạn chiều, vì các biến quyết định $x(\cdot)$ và $u(\cdot)$ là các hàm theo thời gian. Để giải bài toán này bằng phương pháp số, cần phải đưa nó về dạng bài toán hữu hạn chiều bằng cách chọn một tham số hóa quỹ đạo phù hợp.

2.4.2 Tham số hóa Quỹ đạo bằng Đường cong Bézier

Một hướng tiếp cận tự nhiên để tham số hóa hữu hạn chiều là biểu diễn quỹ đạo dưới dạng đa thức với các hệ số đóng vai trò là biến quyết định. Trong số các biểu diễn đa thức, đường cong Bézier đặc biệt phù hợp với lập kế hoạch kinodynamic vì các tính chất hình học và giải tích của chúng cho phép áp đặt các ràng buộc an toàn, độ trơn và tính khả thi động học trực tiếp dưới dạng các điều kiện đại số trên các hệ số.

2.4.2.1 Định nghĩa

Một đường cong Bézier bậc d trên khoảng tham số $[0, 1]$ được xác định bởi $d+1$ điểm điều khiển $\gamma_0, \dots, \gamma_d \in \mathbb{R}^n$ thông qua các đa thức cơ sở Bernstein.

Đa thức Bernstein thứ k bậc d được định nghĩa là

$$\beta_{k,d}(s) := \binom{d}{k} s^k (1-s)^{d-k}, \quad k = 0, \dots, d, \quad s \in [0, 1]. \quad (2.21)$$

Theo định lý nhị thức, các đa thức này không âm và phân hoạch đơn vị: $\sum_{k=0}^d \beta_{k,d}(s) = 1$ với mọi $s \in [0, 1]$. Do đó, với mỗi s cố định, các giá trị $\{\beta_{k,d}(s)\}_{k=0}^d$ tạo thành một tập hệ số tổ hợp lồi. Đường cong Bézier được định nghĩa là trung bình có trọng số tương ứng của các điểm điều khiển:

$$\gamma(s) := \sum_{k=0}^d \beta_{k,d}(s) \gamma_k, \quad s \in [0, 1]. \quad (2.22)$$

2.4.2.2 Các tính chất liên quan đến lập kế hoạch quỹ đạo

Ba tính chất của đường cong Bézier được khai thác trực tiếp trong lập kế hoạch quỹ đạo kinodynamic.

Nội suy điểm đầu cuối. Đường cong đi qua chính xác điểm điều khiển đầu tiên và điểm điều khiển cuối cùng:

$$\gamma(0) = \gamma_0, \quad \gamma(1) = \gamma_d. \quad (2.23)$$

Tính chất này cho phép áp đặt các điều kiện biên về vị trí và, thông qua công thức đạo hàm bên dưới, cả về vận tốc, dưới dạng các ràng buộc đẳng thức tuyến tính trên các điểm điều khiển.

Tính chất bao lồi. Toàn bộ đường cong nằm bên trong bao lồi của các điểm điều khiển của nó:

$$\gamma(s) \in \text{conv}(\{\gamma_k\}_{k=0}^d), \quad \forall s \in [0, 1]. \quad (2.24)$$

Suy ra rằng nếu tất cả $d+1$ điểm điều khiển nằm trong một tập lồi $\mathcal{S}$, thì $\gamma(s) \in \mathcal{S}$ với mọi s . Nguyên tắc bao hàm này chuyển đổi ràng buộc an toàn liên tục (2.18) thành một tập hữu hạn các ràng buộc thuộc tập trên các

điểm điều khiển, với điều kiện vùng an toàn có thể được xấp xỉ hoặc phân hoạch thành các tập lồi.

Công thức đạo hàm (Hodograph). Đạo hàm của γ theo tham số s cũng là một đường cong Bézier bậc $d-1$, với các điểm điều khiển được tạo thành từ các sai phân hữu hạn bậc một của các điểm điều khiển ban đầu:

$$\dot{\gamma}(s) = \sum_{k=0}^{d-1} \beta_{k,d-1}(s) \dot{\gamma}_k, \quad \dot{\gamma}_k = d(\gamma_{k+1} - \gamma_k). \quad (2.25)$$

Áp dụng đệ quy (2.25) cho phép tính gia tốc và các đạo hàm bậc cao hơn dưới dạng đường cong Bézier có bậc giảm dần, với các điểm điều khiển được biểu diễn dưới dạng tổ hợp tuyến tính của các γ_k ban đầu. Do đó, các ràng buộc lên vận tốc và gia tốc dọc theo quỹ đạo — chẳng hạn các giới hạn động học trong (2.19) — quy về các ràng buộc bất đẳng thức tuyến tính trên các sai phân điểm điều khiển, và có thể được áp đặt đồng thời với tính chất bao lồi.

2.4.3 Ràng buộc Độ trơn tại Điểm chuyển tiếp

Với các quỹ đạo trải dài qua nhiều vùng không gian làm việc có tính chất động học khác nhau, một đoạn Bézier duy nhất bậc vừa phải có thể không đủ linh hoạt. Biện pháp thực tế là sử dụng quỹ đạo *Bézier từng đoạn* (piecewise Bézier) gồm M đoạn ghép nối đầu-đuôi. Gọi đoạn thứ i là đường cong Bézier bậc d được xác định bởi các điểm điều khiển $a_0^{(i)}, \dots, a_d^{(i)}$, với $i = 1, \dots, M$. Việc ghép nối các đoạn đảm bảo đường đi liên thông, nhưng cần thêm các điều kiện đại số tại mỗi điểm nối để đảm bảo tính khả vi của quỹ đạo tổng thể.

Liên tục C^0 (vị trí). Điểm cuối của đoạn i trùng với điểm đầu của đoạn $i+1$:

$$a_d^{(i)} = a_0^{(i+1)}. \quad (2.26)$$

Liên tục C^1 (vận tốc). Vận tốc ra của đoạn i khớp với vận tốc vào của

đoạn $i + 1$, điều này theo (2.25) được dịch thành đẳng thức của các sai phân bậc một tại biên:

$$a_d^{(i)} - a_{d-1}^{(i)} = a_1^{(i+1)} - a_0^{(i+1)}. \quad (2.27)$$

Liên tục C^2 (gia tốc). Gia tốc ra của đoạn i khớp với gia tốc vào của đoạn $i + 1$, được biểu diễn qua các sai phân bậc hai tại biên:

$$a_d^{(i)} - 2a_{d-1}^{(i)} + a_{d-2}^{(i)} = a_2^{(i+1)} - 2a_1^{(i+1)} + a_0^{(i+1)}. \quad (2.28)$$

Các điều kiện (2.26)–(2.28) là các ràng buộc đẳng thức tuyến tính trên các điểm điều khiển và do đó bảo toàn cấu trúc đại số của bài toán tối ưu. Việc yêu cầu liên tục C^1 hay C^2 phụ thuộc vào bậc của mô hình động lực học robot: mô hình tích phân đôi (double-integrator) chỉ cần C^1 , trong khi mô hình có đầu vào mô-men xoắn tường minh thường đòi hỏi C^2 để tránh các lệnh lực xung tại điểm nối giữa các đoạn.

Sau khi đặt tham số hóa Bézier từng đoạn, bài toán lập kế hoạch kinodynamic quy về bài toán tối ưu hữu hạn chiều: các biến quyết định là các điểm điều khiển $\{a_k^{(i)}\}$ và, khi áp dụng, chân trời T ; hàm mục tiêu là phiếm hàm chi phí (2.20) được biểu diễn qua các điểm điều khiển; và các ràng buộc bao gồm điều kiện biên (2.17), các ràng buộc an toàn suy từ tính chất bao lồi (2.24), các giới hạn động học được áp đặt qua công thức đạo hàm (2.25), và các điều kiện tại điểm nối (2.26)–(2.28).

Cấu trúc này — hàm mục tiêu tích phân, ràng buộc vi phân trên quỹ đạo, điều kiện biên cố định, và các ràng buộc bất đẳng thức trên đường — chính xác là cấu trúc của một Bài toán Điều khiển Tối ưu (Optimal Control Problem, OCP). Mục tiếp theo hệ thống hóa mối liên hệ này và phát triển khung OCP làm nền tảng cho phương pháp tối ưu hóa quỹ đạo được áp dụng trong luận văn này.

2.5 Bài toán Điều khiển Tối ưu (Optimal Control Problem - OCP)

Optimal Control Problem (OCP) là bài toán lựa chọn đồng thời quỹ đạo trạng thái và tín hiệu điều khiển theo thời gian sao cho một tiêu chuẩn tối ưu nào đó được cực tiểu, trong khi quỹ đạo vẫn phải tuân theo mô hình động lực học và các điều kiện biên của hệ. Ở dạng liên tục theo thời gian, trạng thái được ký hiệu bởi

$$x(t) \in \mathbb{R}^{n_x},$$

và điều khiển được ký hiệu bởi

$$u(t) \in \mathbb{R}^{n_u},$$

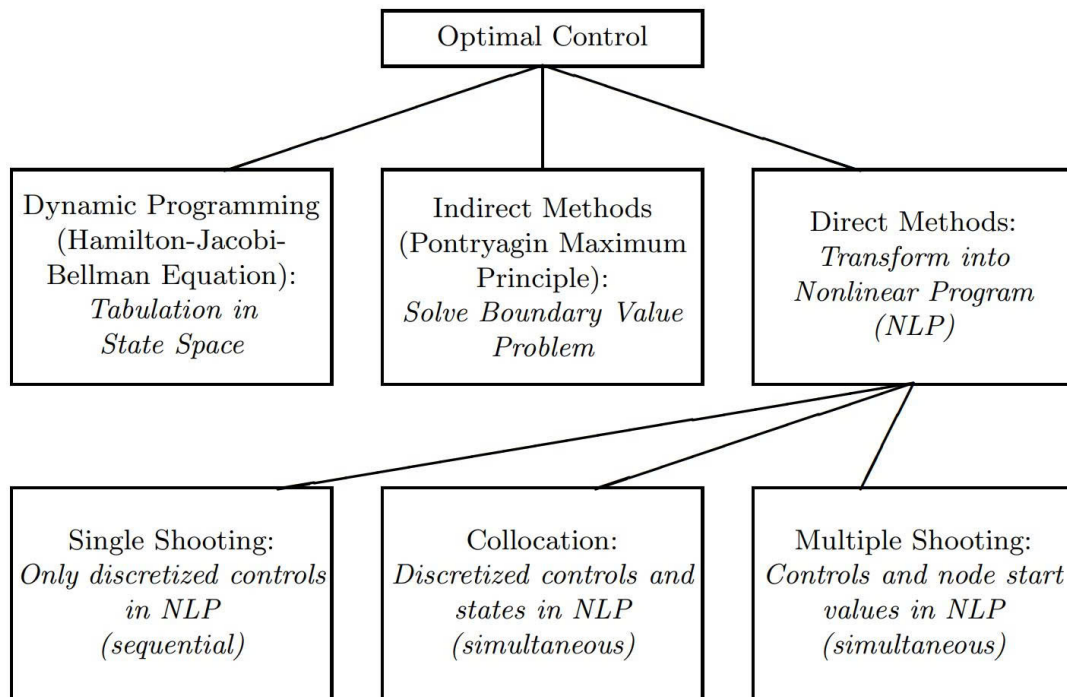
với $t \in [0, T]$ là biến thời gian. Một OCP tổng quát có thể được viết dưới dạng

$$\begin{aligned} & \underset{x(\cdot), u(\cdot), T}{\text{minimize}} && \int_0^T L(x(t), u(t)) dt + E(x(T)) \\ \text{subject to} &&& x(0) - x_0 = 0, && (\text{giá trị đầu cố định}) \\ &&& \dot{x}(t) - f(x(t), u(t)) = 0, && t \in [0, T], && (\text{mô hình ODE}) \\ &&& h(x(t), u(t)) \geq 0, && t \in [0, T], && (\text{ràng buộc theo đường}) \\ &&& r(x(T)) = 0. && (\text{ràng buộc cuối}) \end{aligned} \tag{2.29}$$

Trong công thức trên, $L(x(t), u(t))$ là running cost, $E(x(T))$ là terminal cost, $f(x(t), u(t))$ mô tả động lực học của hệ, $h(x(t), u(t))$ biểu diễn các path constraints, và $r(x(T))$ biểu diễn các terminal constraints. Tùy vào bài toán, độ dài horizon T có thể được cố định trước hoặc được xem như một biến tối ưu.

Điểm quan trọng cần nhấn mạnh là OCP trong (2.29) là một bài toán tối ưu vô hạn chiều. Lý do là các biến quyết định không phải chỉ là các vector hữu hạn chiều, mà là các hàm theo thời gian $x(\cdot)$ và $u(\cdot)$. Vì vậy, một máy tính không thể trực tiếp tối ưu trên toàn bộ không gian các hàm này theo nghĩa liên tục. Muốn giải OCP bằng phương pháp số, cần phải chuyển bài toán liên tục theo thời gian thành một bài toán tối ưu hữu hạn chiều.

Có hai hướng tiếp cận chính để xử lý OCP bằng số. Hướng thứ nhất là các phương pháp gián tiếp, trong đó ta trước hết khai thác các điều kiện cần của tính tối ưu trong miền liên tục, sau đó thu được một bài toán giá trị biên cần giải bằng số. Cách tiếp cận này thường được mô tả là “tối ưu hóa trước, rồi rời rạc hóa sau”. Hướng thứ hai là các phương pháp trực tiếp, trong đó ta trước hết tham số hóa quỹ đạo trạng thái, quỹ đạo điều khiển, hoặc cả hai, rồi biến OCP thành một bài toán quy hoạch phi tuyến hữu hạn chiều. Cách tiếp cận này thường được mô tả là “rời rạc hóa trước, rồi tối ưu hóa sau”.



Hình 2.6: Ba phương pháp cơ bản để giải quyết các bài toán điều khiển tối ưu, (a) lập trình động, (b) phương pháp gián tiếp và (c) phương pháp trực tiếp [4].

Trong các bài toán robotics và motion planning có nhiều ràng buộc, các phương pháp trực tiếp thường là lựa chọn thực dụng hơn. Nguyên nhân là các ràng buộc bất đẳng thức, ràng buộc trạng thái, ràng buộc điều khiển và các thay đổi active set có thể được xử lý trực tiếp trong khung Nonlinear Programming (NLP). Sau khi rời rạc hóa, bài toán OCP được viết về dạng tổng quát

$$\min_w a(w) \quad \text{subject to} \quad b(w) = 0, \quad c(w) \geq 0, \quad (2.30)$$

trong đó w là vector hữu hạn chiều chứa các biến tối ưu, $a(w)$ là hàm mục tiêu hữu hạn chiều, $b(w) = 0$ là các ràng buộc đẳng thức, và $c(w) \geq 0$ là các ràng buộc bất đẳng thức.

Từ góc nhìn này, vai trò của một direct method là xây dựng ánh xạ từ bài toán liên tục (2.29) sang bài toán NLP (2.30). Sự khác nhau giữa các direct methods nằm ở cách chúng xử lý quỹ đạo trạng thái. Trong direct single shooting, trạng thái được sinh ra bằng tích phân tiến từ trạng thái đầu, nên các biến tối ưu chủ yếu là tham số điều khiển. Trong direct collocation, cả trạng thái và điều khiển tại các nút hoặc collocation points đều được đưa vào làm biến tối ưu, còn động lực học được áp bằng các phương trình đại số xấp xỉ. Direct multiple shooting nằm giữa hai cách nhìn này: nó chia horizon thành nhiều đoạn, tích phân động lực học cục bộ trên từng đoạn, đồng thời vẫn giữ các trạng thái đầu đoạn như biến tối ưu độc lập.

Chính vì vậy, multiple shooting có thể được xem là một cách phiên mã OCP vừa giữ được trực giác mô phỏng của shooting method, vừa tận dụng được ưu điểm của simultaneous optimization. Đây là bước trung gian quan trọng để đi từ OCP liên tục theo thời gian đến một NLP hữu hạn chiều có cấu trúc tốt hơn cho solver.

2.6 Phương pháp Bắn Nhiều Lần Trực tiếp

Phương pháp Bắn Nhiều Lần Trực tiếp (Direct Multiple Shooting – DMS) bắt đầu bằng cách chia horizon thời gian $[0, T]$ thành nhiều shooting intervals:

$$0 = t_0 < t_1 < \cdots < t_N = T. \quad (2.31)$$

Trên mỗi khoảng $[t_i, t_{i+1}]$, điều khiển được tham số hóa cục bộ. Ví dụ đơn giản nhất là điều khiển hằng từng đoạn:

$$u(t) = q_i, \quad t \in [t_i, t_{i+1}], \quad (2.32)$$

trong đó q_i là tham số điều khiển của đoạn thứ i .

Khác với phương pháp bắn đơn (single shooting), phương pháp DMS không chỉ dùng một trạng thái đầu duy nhất để tích phân xuyên suốt toàn bộ horizon. Thay vào đó, tại mỗi shooting node t_i , ta giới thiệu một biến trạng thái độc lập

$$s_i \approx x(t_i), \quad (2.33)$$

được gọi là *shooting state* hay *shooting node*. Mỗi đoạn được tích phân độc lập từ trạng thái đầu cục bộ s_i ; cụ thể, trên đoạn $[t_i, t_{i+1}]$, ta giải bài toán giá trị đầu cục bộ

$$\begin{cases} \dot{x}_i(t) = f(x_i(t), q_i), & t \in [t_i, t_{i+1}], \\ x_i(t_i) = s_i. \end{cases} \quad (2.34)$$

Nghiệm của bài toán giá trị đầu này tạo ra một đoạn quỹ đạo cục bộ

$$x_i(t; s_i, q_i),$$

trong đó ký hiệu sau dấu chấm phẩy nhấn mạnh rằng đoạn quỹ đạo phụ

thuộc vào trạng thái đầu đoạn s_i và tham số điều khiển q_i . Trạng thái cuối thu được sau khi tích phân đoạn thứ i được viết dưới dạng ánh xạ điểm cuối (endpoint map):

$$F_i(s_i, q_i) := x_i(t_{i+1}; s_i, q_i). \quad (2.35)$$

Nếu chỉ tạo ra các đoạn cục bộ như trên thì ta chưa có một quỹ đạo toàn cục hợp lệ, vì các đoạn có thể bị đứt tại các shooting node. Do đó, phương pháp DMS áp thêm các ràng buộc nối khớp, còn gọi là ràng buộc liên tục (continuity constraints) hay ràng buộc sai lệch (defect constraints):

$$s_{i+1} - F_i(s_i, q_i) = 0, \quad i = 0, \dots, N-1. \quad (2.36)$$

Với cách xây dựng này, vector biến tối ưu trong phương pháp DMS có dạng

$$w = (s_0, q_0, s_1, q_1, \dots, s_{N-1}, q_{N-1}, s_N). \quad (2.37)$$

Trong đó, các s_i là các trạng thái tại các shooting node, còn các q_i là các tham số điều khiển cục bộ. Bài toán OCP sau đó được biến thành một bài toán quy hoạch phi tuyến (NLP) có dạng

$$\begin{aligned} & \underset{s, q}{\text{minimize}} && \sum_{i=0}^{N-1} l_i(s_i, q_i) + E(s_N) \\ & \text{subject to} && s_0 - x_0 = 0, \quad (\text{giá trị đầu}) \\ & && s_{i+1} - F_i(s_i, q_i) = 0, \quad i = 0, \dots, N-1, \quad (\text{defect / continuity}) \\ & && \text{các ràng buộc trạng thái, điều khiển, và điều kiện cuối nếu có.} \end{aligned} \quad (2.38)$$

Ở đây $l_i(s_i, q_i)$ là chi phí cục bộ trên đoạn $[t_i, t_{i+1}]$, thường thu được bằng cách xấp xỉ tích phân của chi phí chạy (running cost) trên đoạn đó:

$$l_i(s_i, q_i) \approx \int_{t_i}^{t_{i+1}} L(x_i(t; s_i, q_i), q_i) dt. \quad (2.39)$$

Một điểm quan trọng của phương pháp DMS là các trạng thái s_i được phép là biến tối ưu độc lập trong quá trình lặp của solver. Nói cách khác, ở các bước lặp trung gian của NLP, các đoạn quỹ đạo chưa nhất thiết phải nối liên tục với nhau; chỉ tại nghiệm cuối cùng, khi các defect constraints được thỏa mãn, ta mới thu được một quỹ đạo hợp lệ của hệ động lực học. Vì vậy, DMS thuộc nhóm *phương pháp đồng thời* (simultaneous methods): mô phỏng và tối ưu hóa được thực hiện đồng thời trong cùng một bài toán NLP.

Trong thực tế, ánh xạ điểm cuối $F_i(s_i, q_i)$ hiếm khi có biểu thức giải tích đóng; nó thường được tính bằng một bộ tích phân số, chẳng hạn như Runge–Kutta. Nếu chuẩn hóa mỗi shooting interval về biến thời gian cục bộ $\tau \in [0, 1]$ với độ dài đoạn $\Delta_i = t_{i+1} - t_i$, động lực học cục bộ có thể được viết lại thành

$$\frac{dx_i}{d\tau}(\tau) = \Delta_i f(x_i(\tau), u_i(\tau)), \quad \tau \in [0, 1], \quad (2.40)$$

với

$$x_i(0) = s_i. \quad (2.41)$$

Khi đó endpoint map là

$$F_i(s_i, q_i, \Delta_i) = x_i(1). \quad (2.42)$$

Ví dụ, với một sơ đồ Runge–Kutta tổng quát, bước tích phân từ t_i đến t_{i+1} có thể được viết dưới dạng

$$x_{i+1} = x_i + \sum_{j=1}^s b_j K_j, \quad (2.43)$$

trong đó

$$K_j = h f \left(x_i + \sum_{m=1}^{j-1} a_{jm} K_m, u_i \right), \quad j = 1, \dots, s. \quad (2.44)$$

Ở đây h là bước thời gian, còn a_{jm} và b_j là các hệ số của sơ đồ Runge–Kutta. Trong phương pháp DMS, Runge–Kutta chỉ đóng vai trò là công cụ tính ánh xạ điểm cuối trên từng đoạn; chính các ràng buộc defect mới là thành phần buộc các ánh xạ điểm cuối ghép lại thành một quỹ đạo động lực học hợp lệ.

Tóm lại, phương pháp Bắn Nhiều Lần Trực tiếp là cầu nối giữa OCP liên tục theo thời gian và NLP hữu hạn chiều. Phương pháp chia quỹ đạo thành nhiều đoạn động lực học cục bộ, tích phân từng đoạn bằng một bộ tích phân số, rồi ghép các đoạn lại bằng defect constraints. Nhờ vậy, bài toán vẫn giữ được cấu trúc động lực học của OCP gốc, nhưng được viết dưới dạng hữu hạn chiều đủ rõ ràng để giải bằng các bộ giải quy hoạch phi tuyến (nonlinear programming solvers).

Chương 3

Các nghiên cứu liên quan

Chương này khảo sát và phân tích hệ thống các công trình nghiên cứu hiện hành nhằm xác định vị thế kỹ thuật của đề tài trong bối cảnh chung của lĩnh vực hoạch định chuyển động. Chúng tôi thực hiện so sánh đối chứng giữa các hướng tiếp cận dựa trên đồ thị rời rạc, các thuật toán dựa trên lấy mẫu (SBP) và kỹ thuật tối ưu hóa quỹ đạo trực tiếp để làm nổi bật những hạn chế về tính tối ưu và chi phí tính toán. Thông qua đó, chương này luận chứng cho sự cần thiết của một khung lý thuyết thống nhất như GCS, vốn kết hợp được khả năng khám phá không gian của các thuật toán đồ thị với độ chính xác của tối ưu hóa lồi.

3.1 Phương pháp hoạch định dựa trên đồ thị

Các phương pháp hoạch định dựa trên đồ thị (Graph-based planning), tiêu biểu là thuật toán A^* và Dijkstra, đóng vai trò nền tảng trong khoa học robot nhờ các đặc tính về tính tối ưu và tính đầy đủ theo độ phân giải (resolution completeness) trong không gian trạng thái được rời rạc hóa [10]. Trong các kiến trúc phổ biến, hệ thống thường sử dụng bản đồ lưới chiếm chỗ (occupancy grid map) làm cơ sở dữ liệu để thuật toán A^* thực hiện tìm kiếm quỹ đạo thông qua việc tối ưu hóa hàm chi phí kết hợp

với hàm heuristic. Để hỗ trợ thuật toán hoạch định, hệ thống nhận thức thường tích hợp cảm biến LiDAR và khối đo lường quán tính (IMU). Một quy trình phổ biến là sử dụng thuật toán SLAM (như Cartographer) để xây dựng bản đồ và định vị đồng thời [11]. Đối với các hệ thống hoạt động trong không gian 3D nhưng thực hiện hoạch định trong không gian trạng thái $SE(2)$, dữ liệu đám mây điểm (3D point cloud) từ cảm biến LiDAR thường được chiếu xuống mặt phẳng ngang nhằm đơn giản hóa mô hình môi trường trong khi vẫn duy trì các thông tin vật cản thiết yếu.

Tuy nhiên, sự hạn chế về tính liên tục của các phương pháp rời rạc thường dẫn đến các quỹ đạo thiếu độ trơn (zigzag), đòi hỏi các bước hậu xử lý phức tạp để thỏa mãn các ràng buộc động lực học của robot. Để giảm thiểu chi phí tính toán cho các hệ thống có cấu hình động học cao chiều, một chiến lược phổ biến là phân rã bài toán thành hai giai đoạn: hoạch định toàn cục (Global Planning) trong không gian tác vụ để xác định quỹ đạo thô, và thực thi cục bộ (Local Execution) để giải quyết các ràng buộc vị phân và duy trì sự ổn định của hệ thống [11]. Mặc dù cải thiện hiệu suất thời gian thực, sự tách biệt này thường tạo ra khoảng cách lớn giữa hình học và động lực học, dẫn đến các nghiệm dưới mức tối ưu hoặc mất tính khả thi trong các môi trường hẹp.

3.2 Phương pháp hoạch định dựa trên lấy mẫu

Để vượt qua “lời nguyền đa chiều” (curse of dimensionality) mà các phương pháp đồ thị truyền thống gặp phải, các thuật toán dựa trên lấy mẫu (SBP) như Probabilistic Roadmap Method (PRM) và Rapidly-exploring Random Trees (RRT) đã trở thành công cụ mạnh mẽ trong việc xử lý không gian cấu hình (C -space) số chiều cao [12]. Thay vì xây dựng tường minh các rào cản vật lý, SBP xấp xỉ tính kết nối của không gian tự do (C_{free}) thông

qua việc lấy mẫu ngẫu nhiên các cấu hình và kết nối chúng thành cấu trúc đồ thị hoặc cây. Trong khi PRM là bộ lập hoạch đa truy vấn dựa trên giai đoạn học và truy vấn bản đồ lộ trình, thì RRT lại tập trung vào đơn truy vấn bằng cách mở rộng cây từ điểm bắt đầu hướng tới mục tiêu thông qua cơ chế “độ chệch Voronoi” (Voronoi bias), hỗ trợ khám phá nhanh chóng các khu vực chưa xác định [12].

Mặc dù sở hữu tính đầy đủ theo xác suất (probabilistic completeness), các phương pháp SBP nguyên bản thường tạo ra quỹ đạo dạng “răng cưa” và thiếu tính tối ưu về mặt hình học. Biến thể RRT* đã giới thiệu các thủ tục tái kết nối (rewire) để đạt được tính tối ưu tiệm cận (asymptotic optimality), song tốc độ hội tụ của nó thường chậm và đòi hỏi tài nguyên tính toán lớn cho các bước làm mịn [13]. Thách thức lớn nhất đối với SBP vẫn nằm ở việc tích hợp các ràng buộc vi phân liên tục (kinodynamic constraints), do việc giải bài toán biên (BVP) để kết nối các mẫu rời rạc trong các hệ thống phi holonom là cực kỳ phức tạp và nhạy cảm với các hàm metric khoảng cách [13].

3.3 Tối ưu hóa quỹ đạo trực tiếp

Các phương pháp tối ưu hóa quỹ đạo trực tiếp đóng vai trò quyết định trong việc tạo ra các chuyển động mượt mà và khả thi về mặt vật lý thông qua việc chuyển đổi bài toán điều khiển tối ưu liên tục thành bài toán lập trình phi tuyến (Nonlinear Programming - NLP) [14]. Phương pháp bắn trực tiếp (Direct Shooting Method), đặc biệt là kỹ thuật bắn nhiều điểm (Direct Multiple Shooting), cho phép chia quỹ đạo thành nhiều phân đoạn độc lập để tính toán song song, đồng thời đảm bảo tính liên tục của trạng thái tại các điểm nút khi hội tụ [15]. Khác với SBP, DTO cho phép nhúng trực tiếp các phương trình động lực học của hệ thống vào quá trình tối ưu hóa dưới dạng các ràng buộc toán học, từ đó xử lý hiệu quả các hệ thống có ràng buộc vi phân phức tạp.

Trong các nghiên cứu hiện đại, kỹ thuật tối ưu hóa trực tiếp thường được sử dụng như một giai đoạn hậu xử lý để tinh chỉnh các quỹ đạo sơ bộ từ SBP, nhằm đạt được sự cân bằng giữa khả năng khám phá không gian và tính tối ưu về năng lượng hoặc thời gian. Tuy nhiên, để tận dụng triệt để ưu điểm của cả hai trường phái, các bộ lập hoạch dựa trên Lập trình lồi số nguyên hỗn hợp (Mixed-Integer Convex Programming - MICP) đã được đề xuất [16]. Hướng tiếp cận này hứa hẹn kết hợp tính đầy đủ của các thuật toán dựa trên lấy mẫu với khả năng xử lý chính xác các ràng buộc động học và động lực học của robot từ các phương pháp tối ưu hóa quỹ đạo, thiết lập một khung lý thuyết thống nhất cho các bài toán hoạch định chuyển động hiện đại.

3.4 Tối ưu hóa quỹ đạo và Điều khiển tối ưu (Optimal Control)

3.4.1 Các phương pháp dựa trên Tối ưu hóa quỹ đạo

Tối ưu hóa quỹ đạo (Trajectory Optimization) là nhóm phương pháp mạnh mẽ để sinh ra các quỹ đạo mượt mà và khả thi về mặt động lực học. Các phương pháp phổ biến như CHOMP [17] hay TrajOpt [18] mô hình hóa việc tránh va chạm dưới dạng các hàm phạt (penalty functions) và sử dụng quy hoạch phi tuyến (SQP) để tìm nghiệm cục bộ. Mặc dù cho kết quả tốt trong các môi trường đơn giản, các thuật toán này phụ thuộc rất lớn vào giá trị khởi tạo. Nếu điểm khởi tạo nằm trong một cực tiểu cục bộ (ví dụ: bị bao quanh bởi vật cản hình chữ U), thuật toán thường thất bại trong việc tìm ra đường đi hợp lệ.

3.4.2 Áp dụng Điều khiển tối ưu (OCP) và Multiple Shooting

Để khắc phục tính phi lồi tự nhiên của không gian có vật cản, bài toán hoạch định chuyển động thường được phát biểu dưới dạng Bài toán Điều khiển Tối ưu (Optimal Control Problem). Tuy nhiên, để giải quyết chúng trong thời gian thực, các kỹ thuật rời rạc hóa phải được áp dụng một cách cẩn thận.

Phương pháp Single Shooting (mô phỏng tiến từ trạng thái ban đầu) rất đơn giản nhưng cực kỳ nhạy cảm với sai số số học và dễ dẫn đến sự bất ổn định phi tuyến, đặc biệt với các hệ thống robot có thời gian di chuyển dài [15]. Ngược lại, phương pháp Multiple Shooting [4] đã chứng minh được tính ưu việt bằng cách chia nhỏ quỹ đạo thành các đoạn ngắn và giải quyết đồng thời. Khả năng tích hợp Multiple Shooting với các bộ giải quy hoạch phi tuyến hiệu năng cao (như IPOPT hay SNOPT) đã giúp phương pháp này trở thành tiêu chuẩn vàng trong điều khiển tối ưu. Mặc dù vậy, khi áp dụng thuần túy vào môi trường có nhiều vật cản, Multiple Shooting vẫn không thể vượt qua hoàn toàn rào cản về cực tiểu cục bộ nếu không có sự phân chia không gian an toàn một cách có hệ thống.

Sự kết hợp giữa lý thuyết đồ thị (như GCS) để tìm một đường hầm (tube) không gian an toàn lồi, sau đó áp dụng Multiple Shooting trong không gian đó, đã được đề xuất trong các công trình gần đây nhằm tận dụng tối đa ưu điểm của cả hai: sự đảm bảo về an toàn hình học toàn cục và sự chính xác, khả thi về mặt động lực học của hệ thống liên tục.

Chương 4

Phương pháp đề xuất

Trong chương này, chúng tôi trình bày chi tiết về hai hướng tiếp cận để giải quyết bài toán hoạch định chuyển động cho rô-bốt. Đầu tiên là mô hình bài toán, sau đó là chi tiết phương pháp lập kế hoạch chuyển động bằng Đồ thị các tập hợp lồi (Graph of Convex Sets - GCS), và cuối cùng là phương pháp cải tiến sử dụng bài toán Điều khiển Tối ưu (Optimal Control Problem - OCP) kết hợp với Multiple Shooting (MMS) và hàm cản logarit (log-barrier).

4.1 Mô hình hoạch định chuyển động

Trong phần này, chúng tôi trình bày mô hình toán học của bài toán hoạch định chuyển động cho hệ thống robot tự hành với các ràng buộc vi phân liên tục. Mục tiêu là xây dựng một khung lý thuyết vững chắc để phát triển các thuật toán lập kế hoạch hiệu quả, đồng thời đảm bảo tính khả thi về mặt động lực học và tối ưu hóa hiệu suất vận hành trong môi trường phức tạp chứa vật cản.

4.1.1 Mô hình toán học

Ta xem xét một quỹ đạo mượt trong $\mathbb{R}^d$. Một quỹ đạo có thể được biểu diễn dưới dạng hàm số liên tục:

$$\mathbf{x}(t) : [0, T] \rightarrow \mathbb{R}^d, \quad (4.1)$$

trong đó T là thời gian hoàn thành quỹ đạo. Quỹ đạo này phải thỏa mãn điều kiện khả vi đến bậc K , tức là:

$$\frac{d^k \mathbf{x}(t)}{dt^k} \text{ tồn tại và liên tục với } k = 1, 2, \dots, K, \quad (4.2)$$

với đạo hàm bậc 0 là chính hàm số $\mathbf{x}(t)$.

Hàm mục tiêu cần được tối ưu hóa có thể là tổng chi phí năng lượng, thời gian hoàn thành, hoặc một hàm chi phí tổng quát khác. Cụ thể, ta định nghĩa hàm mục tiêu dưới dạng tích phân:

$$\psi(\mathbf{x}(t), T) := a\Phi(T) + \sum_{m=1}^M b_m \int_0^T \left\| \frac{d^m \mathbf{x}(t)}{dt^m} \right\|^2 dt, \quad (4.3)$$

trong đó $\Phi(T)$ là hàm chi phí liên quan đến thời gian, m là bậc đạo hàm đại diện cho đặc tính động lực học cần tối ưu hóa (ví dụ: vận tốc, gia tốc), và a, b_m là các hệ số trọng số điều chỉnh tầm quan trọng tương đối của các thành phần trong hàm mục tiêu.

Tổng quát, bài toán hoạch định chuyển động có thể được mô tả như sau:

$$\begin{aligned} & \min_{\mathbf{x}(t), T} \quad \psi(\mathbf{x}(t), T) \\ & \text{subject to} \quad \mathbf{x}(0) = \mathbf{x}_{\text{start}}, \quad \mathbf{x}(T) = \mathbf{x}_{\text{goal}}, \\ & \quad \mathbf{x}(t) \in \mathcal{C}_{\text{free}}, \quad \forall t \in [0, T], \\ & \quad \frac{d^k \mathbf{x}(t)}{dt^k} \in \mathcal{D}_k, \quad k = 1, 2, \dots, K, \quad \forall t \in [0, T], \\ & \quad T_{\min} \leq T \leq T_{\max}. \end{aligned} \quad (4.4)$$

Trong đó, ràng buộc đầu và cuối đảm bảo quỹ đạo bắt đầu và kết thúc tại các vị trí xác định. Ràng buộc không gian trạng thái yêu cầu quỹ đạo phải nằm trong không gian tự do $\mathcal{C}_{\text{free}}$. Ràng buộc động học giới hạn các đạo hàm bậc k của quỹ đạo trong các tập hợp khả thi $\mathcal{D}_k$, phản ánh các giới hạn vật lý của hệ thống robot. Cuối cùng, ràng buộc về thời gian đảm bảo rằng thời gian hoàn thành quỹ đạo nằm trong khoảng cho phép $[T_{\min}, T_{\max}]$. Quan sát hàm mục tiêu và ràng buộc, khi tổng thời gian thực hiện quỹ đạo T được coi là một biến quyết định (decision variable) để tối ưu hóa hiệu suất, cấu trúc của hàm mục tiêu tích phân $\int_0^T \|\mathbf{x}^{(m)}(t)\|^2 dt$ trở nên cực kỳ phức tạp. Do giới hạn tích phân thay đổi theo T , mối liên hệ giữa các biến trạng thái và thời gian bị ràng buộc một cách phi tuyến tính.

4.1.2 Tham số hóa quỹ đạo dựa trên tọa độ đường đi

Để giải quyết sự phức tạp của bài toán tối ưu hóa đồng thời hình dạng quỹ đạo và phân bổ thời gian, một phương pháp tiếp cận hiệu quả là thực hiện tham số hóa quỹ đạo thông qua một tọa độ đường đi vô hướng $s \in [0, 1]$ ([19], [20]). Việc sử dụng biến độc lập s thay cho biến thời gian t cho phép phân rã quỹ đạo thành hai thành phần biệt lập: đặc tính hình học không gian và quy luật tiến triển theo thời gian [21]. Trong cấu trúc này, quỹ đạo $\mathbf{x}(t)$ được tái cấu trúc thông qua cặp hàm số bao gồm hàm hình dạng $q(s) : [0, 1] \rightarrow \mathbb{R}^d$ xác định hình dáng hình học trong không gian cấu hình, và hàm thời gian $h(s) : [0, 1] \rightarrow \mathbb{R}$ xác định thời điểm mà robot đi qua vị trí tương ứng với tọa độ s . Khi đó, quỹ đạo thực tế được biểu diễn dưới dạng hàm hợp $\mathbf{x}(t) = q(h^{-1}(t))$, trong đó mối liên hệ giữa thời gian và tọa độ đường đi được thiết lập bởi phương trình $t = h(s)$.

Việc tách biệt này mang lại lợi ích đáng kể trong việc thiết kế các quỹ đạo mượt mà bằng cách áp đặt các điều kiện về tính khả vi lên hàm $q(s)$ và $h(s)$ một cách độc lập. Tuy nhiên, phương pháp tham số hóa này làm nảy sinh những thách thức nhất định khi thiết lập các ràng buộc động học cấp

cao. Trong khi các hàm chi phí lỗi và các ràng buộc về vận tốc vẫn duy trì được tính lồi khi biểu diễn qua $q(s)$ và $h(s)$, thì các ràng buộc liên quan đến đạo hàm bậc hai (như gia tốc) hoặc cao hơn thường mất đi tính lồi vốn có [1]. Điều này gây khó khăn cho các thuật toán tối ưu hóa lồi truyền thống trong việc tìm kiếm nghiệm tối ưu toàn cục cho các bài toán có ràng buộc động lực học phức tạp.

Để khắc phục hạn chế về tính phi lồi của các đạo hàm cấp cao trong khung tham số hóa $q(s)$ và $h(s)$, một chiến lược thực tiễn thường được áp dụng là sử dụng kỹ thuật điều chuẩn (regularization). Thay vì giải quyết trực tiếp các ràng buộc phi lồi khắt khe, hệ thống sẽ bổ sung các thành phần phạt (penalty) tỉ lệ thuận với độ lớn của các đạo hàm bậc cao như $q''(s)$ và $h''(s)$ vào hàm mục tiêu. Đồng thời, hàm thời gian $h(s)$ được ràng buộc chặt chẽ để đảm bảo đạo hàm $h'(s)$ không tiệm cận giá trị không, nhằm duy trì tính đơn điệu tăng của thời gian và đảm bảo tính khả nghịch của phép biến đổi [1]. Cách tiếp cận này cho phép xấp xỉ các ràng buộc kinodynamic trong một không gian tìm kiếm ổn định hơn, giúp bộ lập kế hoạch hội tụ nhanh chóng về các quỹ đạo khả thi về mặt vật lý ngay cả trong các bài toán có số chiều lớn.

4.2 Phân hoạch không gian an toàn

Thay vì tiếp cận bài toán tránh vật cản theo cách truyền thống là mô tả các ràng buộc không lồi dựa trên bề mặt vật cản, ta chuyển đổi không gian cấu hình tự do (collision-free configuration space) thành hợp của một tập hợp hữu hạn các vùng lồi bị chặn (bounded convex regions). Trong mô hình này, các vùng lồi có thể chồng lấn lên nhau, và bài toán hoạch định chuyển động được phát biểu lại dưới dạng một chương trình quy hoạch lồi nguyên hỗn hợp (Mixed-Integer Convex Program - MICP). Cụ thể, các biến nguyên được sử dụng để gán từng đoạn quỹ đạo đa thức (polynomial trajectory segments) vào các vùng lồi an toàn cụ thể này. Phương pháp

này mang lại lợi thế lớn về mặt tính toán vì số lượng biến nguyên chỉ phụ thuộc vào số lượng vùng an toàn được tạo ra (sử dụng thuật toán IRIS) thay vì phụ thuộc vào số lượng mặt của vật cản, giúp giải quyết hiệu quả các môi trường phức tạp với hàng trăm vật cản. Hơn nữa, việc ràng buộc quỹ đạo nằm trọn trong các tập lỗi đảm bảo an toàn tuyệt đối cho toàn bộ hành trình liên tục.

Để giải quyết thách thức về tính toán của phân hoạch tối ưu, một số các phương pháp đã được phát triển. Dưới đây là một số phương pháp được dùng trong bài báo cáo này:

4.2.1 Phân hoạch Lỗi Xấp xỉ (ACD) của Đa giác

Phương pháp Approximate Convex Decomposition (ACD) giới thiệu một cách tiếp cận khác biệt cơ bản để quản lý sự phức tạp. Thay vì yêu cầu sự lồi hoàn hảo, nó cho phép các thành phần "lồi xấp xỉ" trong một dung sai do người dùng xác định, τ . [8] Điều này thường dẫn đến các phân hoạch có ít thành phần hơn nhiều và phù hợp hơn với các đặc điểm cấu trúc nổi bật của đối tượng.

4.2.1.1 Các Khái niệm Cơ bản trong phân hoạch Lồi

Phần này trình bày các khái niệm hình học cơ bản được sử dụng trong quá trình phân hoạch đa giác không lồi thành các vùng lồi. Các định nghĩa này đóng vai trò nền tảng cho các phương pháp đo độ lồi và các thuật toán phân hoạch ở các phần sau.

1. **Bao lồi (Convex Hull – H_P):** Bao lồi của một đa giác P là **tập hợp lồi nhỏ nhất chứa toàn bộ đa giác** đó. Trực quan, có thể hình dung bao lồi như một sợi dây chun được kéo căng bao quanh đa giác. Một đa giác P được gọi là **lồi** nếu và chỉ nếu nó trùng với bao lồi của chính nó, tức là:

$$P = H_P.$$

2. **Notches:** Là các đỉnh của đa giác P **không nằm trên bao lồi** H_P . Về mặt hình học, đây là những đỉnh có **góc trong lớn hơn** 180° . Các Notches biểu hiện các đặc trưng lõm (concave features) trên biên của đa giác.

3. **Cầu (Bridges):** Là các cạnh của bao lồi ∂H_P nối hai đỉnh không liên kề của biên ngoài ∂P_0 . Về mặt toán học, tập hợp các cầu được định nghĩa:

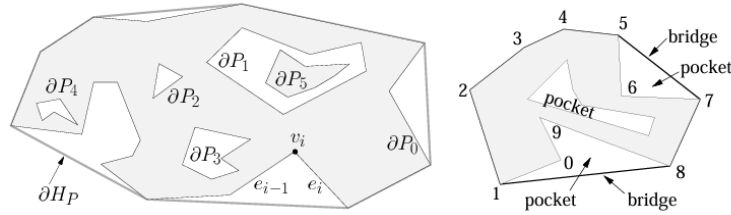
$$\text{BRIDGES}(P) = \partial H_P \setminus \partial P.$$

Mỗi cầu “bắc qua” một vùng lõm của đa giác.

4. **Túi (Pockets):** Là các **chuỗi cạnh liên tiếp** của biên đa giác ∂P không thuộc về vỏ lồi ∂H_P . Về mặt toán học:

$$\text{POCKETS}(P) = \partial P \setminus \partial H_P.$$

Mỗi túi tương ứng với một “vịnh lõm” (concave bay) trên đường biên của đa giác.



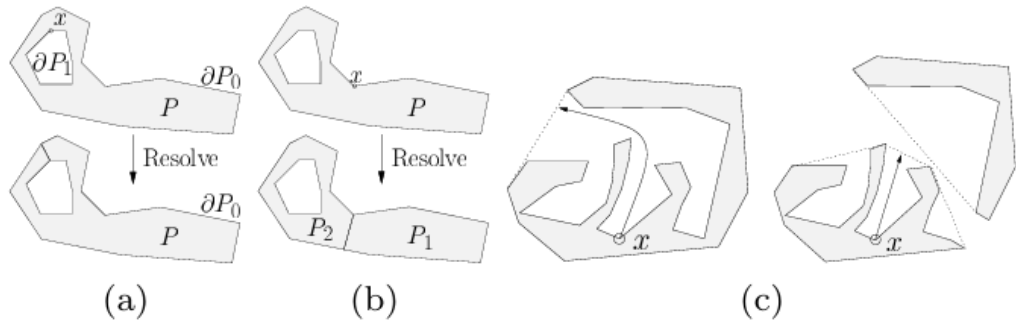
4.2.1.2 Ý tưởng và Chi tiết Thuật toán

Thuật toán là một chiến lược chia để trị đệ quy[8]:

1. Đo độ lõm: Xác định đặc điểm không lồi (một notch hay đỉnh lõm) quan trọng nhất trong đa giác. Đây là điểm có độ lõm lớn nhất. Tùy theo từng trường hợp mà ta có thể sử dụng các thước đo độ lõm khác

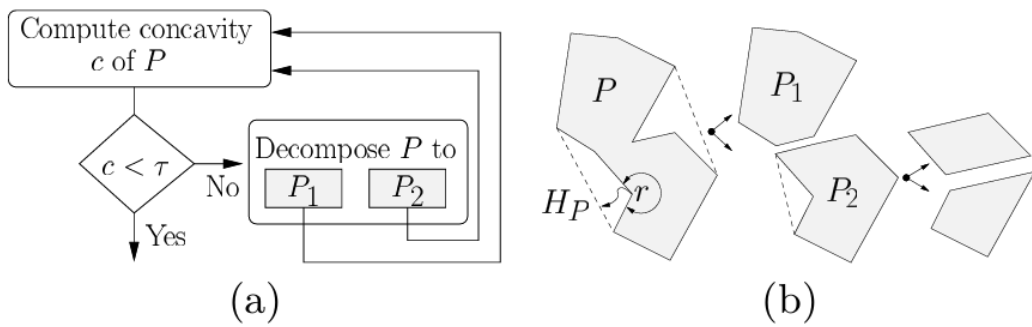
nhau, cách để tính những thước đo độ lõm này sẽ được giới thiệu rõ hơn ở phần 4.2.1.3

2. Kiểm tra Dung sai: Nếu độ lõm tối đa đo được nhỏ hơn τ , quá trình kết thúc đối với thành phần đó.
3. Giải quyết Đỉnh lõm: Nếu độ lõm vượt quá τ , một đường cắt được thêm vào để giải quyết đỉnh lõm, chia đa giác thành hai thành phần mới (hoặc hợp nhất một lỗ).



Hình 4.1: (a) Nếu $x \in \partial P_i > 0$, hàm **Resolve** **gộp** biên ∂P_i vào đa giác P_0 . (b) Nếu $x \in \partial P_0$, hàm **Resolve** **tách** đa giác P thành hai đa giác P_1 và P_2 . (c) **Độ lõm** tại điểm x thay đổi sau khi đa giác được phân hoạch.

4. Độ quy: Quá trình được áp dụng đệ quy cho các thành phần mới.



Hình 4.2: Quá trình đệ quy tiếp tục cho đến khi đạt đến giới hạn độ lõm đầu vào của người dùng τ .

4.2.1.3 Các Thước đo Độ lõm

Chất lượng của phân hoạch phụ thuộc rất nhiều vào **cách đo độ lõm** của các đỉnh trong đa giác không lồi. Các phương pháp đo độ lõm phổ biến bao gồm:

1. **Độ lõm Đường thẳng (Straight-Line Concavity – SL-Concavity):**

Là khoảng cách Euclid từ một đỉnh trong *túi* (pocket) đến *cầu nối* (bridge) liên kết của nó — thường là một cạnh thuộc bao lồi của đa giác. Phương pháp này rất nhanh, nhưng **không đảm bảo tính đơn điệu**, có thể khiến một số đặc trưng lõm quan trọng bị bỏ sót.



Hình 4.3: Giả sử r là điểm lõm có độ lõm lớn nhất. Sau khi giải quyết (resolve) r , độ lõm của s tăng lên. Nếu $\text{concave}(r) < \tau$, vùng s sẽ không bao giờ được xử lý, ngay cả khi $\text{concave}(s)$ lớn hơn τ .

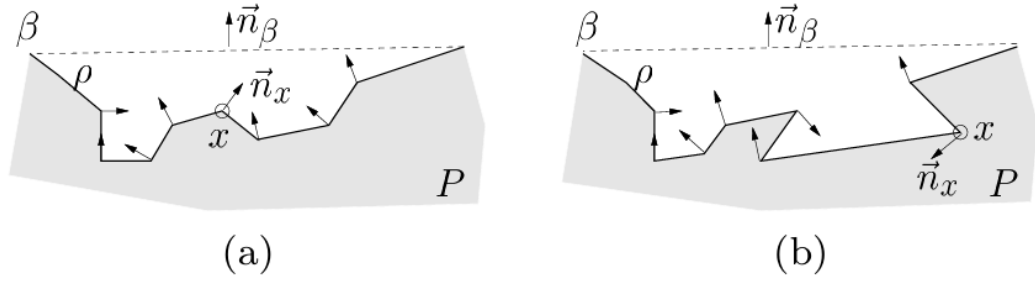
2. **Độ lõm Đường đi Ngắn nhất (Shortest Path Concavity – SP-Concavity):**

Là **chiều dài đường đi ngắn nhất** từ một đỉnh trong túi đến cầu nối tương ứng, với điều kiện đường đi đó phải nằm hoàn toàn trong túi. Phương pháp này chậm hơn SL-Concavity nhưng **đảm bảo tính đơn điệu**: độ lõm của các đỉnh giảm dần sau mỗi lần phân hoạch, giúp đảm bảo rằng tất cả các đặc điểm lõm quan trọng cuối cùng sẽ được xử lý.

3. **Độ lõm Lai (Hybrid Concavity – H-Concavity):**

Một giải pháp trung gian thực tiễn — sử dụng SL-Concavity nhanh làm mặc định và chỉ chuyển sang SP-Concavity khi **phát hiện khả năng không đơn điệu**. Bằng cách thêm bước kiểm tra *nguy cơ tiềm ẩn* bằng cách so sánh n_β và n_i ; nếu tồn tại $n_i \cdot n_\beta < 0$, biên túi bị đảo hướng — dấu

hiệu của túi phức tạp mà SL-Concavity có thể thất bại.



Hình 4.4: Độ lõm SL có thể xử lý được túi trong (a) một cách chính xác vì không có hướng chuẩn nào của các đỉnh trong túi ngược với hướng chuẩn của cầu. Tuy nhiên, túi trong (b) có thể dẫn đến độ lõm giảm không đơn điệu.

4.2.2 Iterative Regional Inflation by Semi-Definite Programming (IRIS)

Thuật toán Iterative Regional Inflation by Semidefinite Programming (IRIS) thay vì cố gắng bao phủ toàn bộ không gian tự do cùng một lúc, nó đặt ra câu hỏi: "Với một điểm 'mầm' (seed) ban đầu, vùng lồi lớn nhất của không gian tự do mà tôi có thể tìm thấy xung quanh nó là gì?". Tức sau khi chạy xong giải thuật output của ta là một vùng lồi lớn nhất có thể của thuật toán từ 1 seed point cho trước.

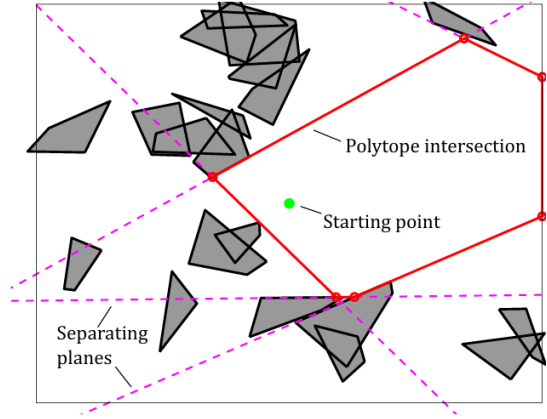
4.2.2.1 Ý tưởng và Chi tiết Thuật toán

IRIS là một quy trình tối ưu hóa lặp đi lặp lại gồm 4 bước:

1. **Khởi tạo:** Thuật toán sẽ tạo ra một hình elip ban đầu là một hình cầu rất nhỏ có tâm chính là điểm mầm (seed point).

2. Tìm Siêu phẳng Phân cách (QP):

Đối với ellipsoid hiện tại, thuật toán tìm điểm gần nhất trên mỗi chướng ngại vật. Sau đó, nó xây dựng các siêu phẳng phân cách ellipsoid khỏi các chướng ngại vật này. Mỗi siêu phẳng tiếp tuyến với



Hình 4.5: Siêu phẳng Phân cách

một phiên bản mở rộng của ellipsoid và đi qua điểm gần nhất trên chướng ngại vật tương ứng. Việc tìm điểm gần nhất trên mỗi chướng ngại vật lỗi được xây dựng như một bài toán Quy hoạch Toàn phương (Quadratic Program - QP). Để giữ cho bài toán tối ưu hóa tiếp theo có quy mô nhỏ, các siêu phẳng thừa (những siêu phẳng không xác định biên của vùng lỗi kết quả) sẽ được loại bỏ.

Bước 2.1. Tìm điểm gần nhất trên obstacle đến hình elip hiện tại
Biến đổi không gian (Từ không gian Ellipsoid sang không gian Quả cầu đơn vị)

Việc tính toán khoảng cách đến một ellipsoid là phức tạp. Tuy nhiên, tính khoảng cách đến một quả cầu đơn vị tại gốc tọa độ thì lại rất đơn giản. Ý tưởng ở đây là biến đổi toàn bộ không gian sao cho ellipsoid trở thành một quả cầu đơn vị. Cụ thể, một ellipsoid E được định nghĩa là ảnh của một quả cầu đơn vị $\tilde{E}$ qua phép biến đổi affine $x = C\tilde{x} + d$, trong đó C là ma trận xác định hình dạng và hướng của ellipsoid, còn d là tâm của nó. Khi đó, ta có thể sử dụng phép biến đổi ngược $\tilde{x} = C^{-1}(x - d)$ để ánh xạ mọi điểm trong không gian ban đầu trở lại "không gian quả cầu đơn vị". Sau phép biến đổi này, ellipsoid E trở thành quả cầu đơn vị $\tilde{E}$ tại gốc tọa độ, và đồng thời mỗi chướng ngại vật l_j trong không gian ban đầu cũng bị biến đổi (méo đi) thành một đối tượng mới $\tilde{l}_j$ trong không gian mới.

Bước 2.2: Tìm điểm gần nhất bằng Quy hoạch Toàn phương (QP)

Bây giờ, bài toán tìm điểm trên chướng ngại vật l_j gần ellipsoid E nhất tương đương với việc tìm điểm trên chướng ngại vật đã biến đổi $\tilde{l}_j$ gần gốc tọa độ nhất.

Bài toán này được xây dựng dưới dạng một bài toán *Quy hoạch Toàn phương* (Quadratic Program - QP):

$$\begin{aligned} & \underset{\tilde{x} \in \mathbb{R}^n, w \in \mathbb{R}^m}{\text{minimize}} && \|\tilde{x}\|^2 \\ & \text{subject to} && \begin{cases} [\tilde{v}_{j,1} \ \tilde{v}_{j,2} \ \cdots \ \tilde{v}_{j,m}] w = \tilde{x}, \\ \sum_{i=1}^m w_i = 1, \\ w_i \geq 0, \quad i = 1, \dots, m. \end{cases} \end{aligned}$$

Về bản chất, ta đang tìm một điểm $\tilde{x}$ là **tổ hợp lồi** của các đỉnh đã biến đổi $\tilde{v}_{j,k}$ sao cho khoảng cách của $\tilde{x}$ đến gốc tọa độ là nhỏ nhất.

Bước 2.3. Xác định mặt phẳng tiếp tuyến của ellipsoid tại điểm tiếp xúc

Sau khi tìm được điểm x^* gần nhất, ta xây dựng mặt phẳng tiếp tuyến với ellipsoid tại điểm đó. Ellipsoid được mô tả bằng biểu thức nghịch đảo:

$$E = \{x \mid (x - d)^\top C^{-1} C^{-\top} (x - d) \leq 1\}.$$

Vector pháp tuyến của ellipsoid tại x^* được xác định bởi gradient:

$$a_j = \nabla_x [(x - d)^\top C^{-1} C^{-\top} (x - d)]_{x=x^*} = 2C^{-1} C^{-\top} (x^* - d).$$

Vì mặt phẳng tiếp tuyến đi qua x^* , ta có:

$$b_j = a_j^\top x^*.$$

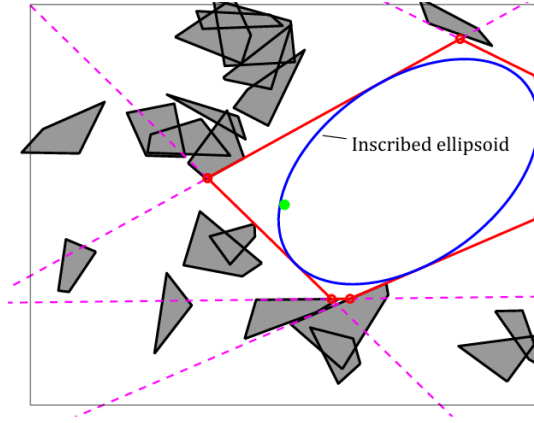
Khi đó, phương trình mặt phẳng tiếp tuyến là:

$$a_j^\top x = b_j,$$

được thêm vào như một **ràng buộc tuyến tính** trong tập khả thi của thuật toán IRIS. Mặt phẳng này đảm bảo ellipsoid mở rộng tối đa nhưng không giao với chướng ngại vật.

Bước 2.4. Lặp lại với tất cả các obstacles trong không gian. Ta sẽ có được một tập các siêu phẳng để tạo thành một vùng lồi đa giác.

3. **Tìm Ellipsoid Nội tiếp Cực đại (SDP):** Giao của các nửa không gian được xác định bởi các siêu phẳng tạo thành một đa diện lồi (polytope). Thuật toán sau đó tìm ellipsoid có thể tích lớn nhất có thể nội tiếp trong đa diện này.



Hình 4.6: Ellipsoid Nội tiếp Cực đại

Bài toán này tìm **ellipsoid có thể tích lớn nhất** nằm gọn trong một đa diện lồi P :

$$P = \{x \in \mathbb{R}^n \mid Ax \leq b\}. \quad (4.5)$$

Ellipsoid được định nghĩa là $\mathcal{E}$, với d là tâm và ma trận C xác định

hình dạng:

$$\mathcal{E} = \{C\tilde{x} + d \mid \|\tilde{x}\|_2 \leq 1\}. \quad (4.6)$$

Thể tích của ellipsoid tỉ lệ với $\det(C)$.

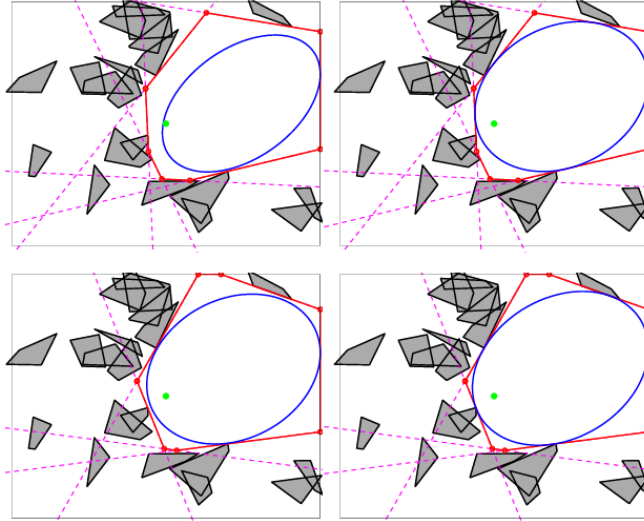
Để tìm ellipsoid lớn nhất nội tiếp trong P , ta giải bài toán tối ưu lồi sau:

$$\begin{aligned} & \underset{C, d}{\text{maximize}} && \log \det C \\ & \text{subject to} && \|a_i^\top C\|_2 + a_i^\top d \leq b_i, \quad \forall i = 1, \dots, N, \\ & && C \succ 0. \end{aligned} \quad (4.7)$$

Mục tiêu: Cực đại hoá $\log \det C$ để tối đa hoá thể tích ellipsoid.

Ràng buộc: Đảm bảo mọi điểm của ellipsoid đều nằm trong đa diện P .

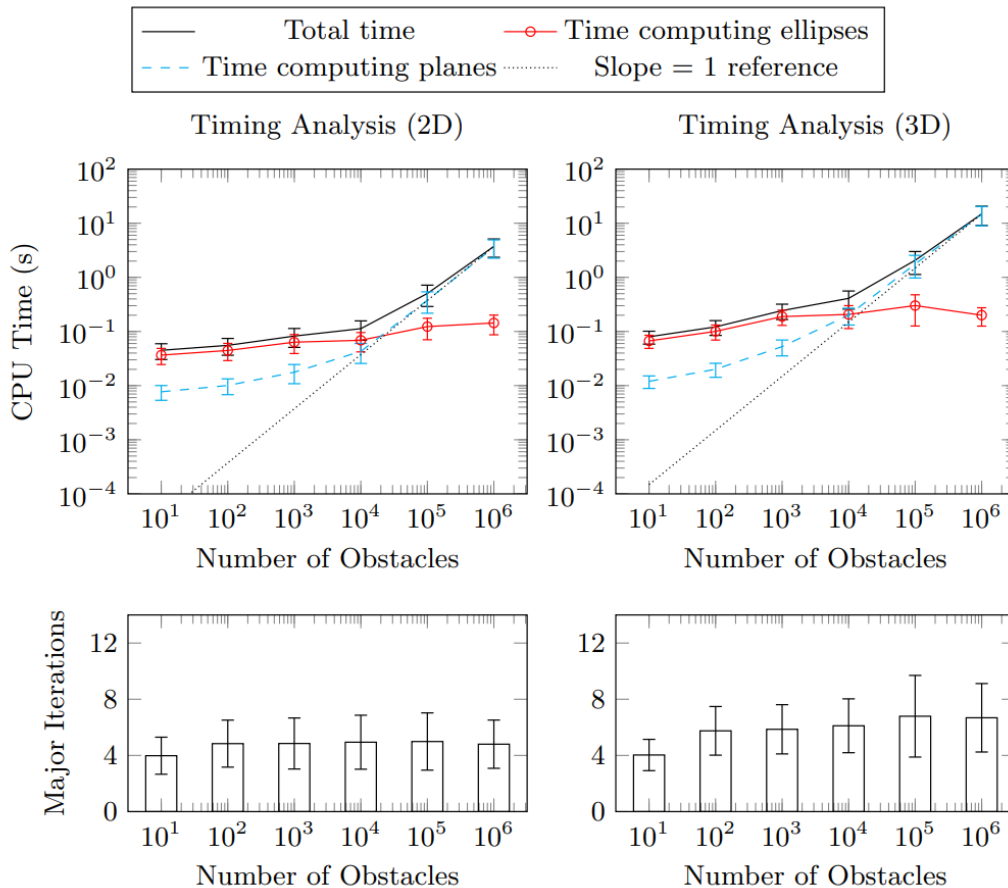
4. **Lặp lại:** Ellipsoid mới, lớn hơn này được sử dụng làm điểm khởi đầu cho vòng lặp tiếp theo. Quá trình này hội tụ khi thể tích của ellipsoid ngừng tăng một cách đáng kể, trả về một đa diện lồi lớn và ellipsoid nội tiếp của nó.



Hình 4.7: Lặp lại đến khi hội tụ

4.2.2.2 Đặc tính của Thuật toán

- Loại thuật toán: IRIS là một phương pháp xấp xỉ để bao phủ toàn bộ không gian tự do. Một lần chạy duy nhất chỉ tạo ra một vùng lõi. Để có được một lớp phủ hoàn chỉnh, cần phải chạy thuật toán nhiều lần với các điểm mầm khác nhau.
- Độ phức tạp thời gian: IRIS thường hội tụ qua 4-8 vòng lặp. Thời gian tính toán của mỗi vòng lặp bằng thời gian tìm các mặt phẳng phân cách cộng với thời gian tính toán ellipses nội tiếp. Trong khi thời gian giải ellipses (SDP) gần như không đổi (constant) nhờ vào việc loại bỏ các siêu phẳng thừa thì thời gian tính các siêu phẳng phân cách lại tăng tuyến tính theo số lượng các obstacles.



Hình 4.8: Độ phức tạp thời gian [22]

- Xử lý Vật cản không lõi:

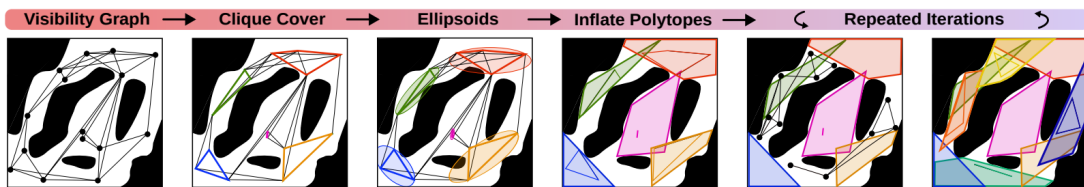
Chướng ngại vật trong không gian work space: Thuật toán IRIS ban đầu giả định các chướng ngại vật là lỗi, đây cũng là một nhược điểm lớn nhất của thuật toán này. Để đảm bảo các obstacles là lỗi, ta thường phải tiền xử lý bằng cách bao lỗi các chướng ngại vật không lỗi[22]. Ngoài ra cách tiếp cận tiêu chuẩn cho các chướng ngại vật không lỗi là phân hoạch chúng thành một tập hợp các mảnh lỗi trước khi chạy IRIS (tức tăng số lượng obstacles của không gian lên). Điều này khả thi vì IRIS có khả năng mở rộng hiệu quả với số lượng lớn chướng ngại vật.

Chướng ngại vật trong không gian cấu hình (Configuration Space): Đối với các robot có động học phức tạp, nơi các chướng ngại vật trong không gian cấu hình được định nghĩa một cách ẩn ý và không lỗi, các phần mở rộng như IRIS-NP [23] và C-IRIS [24] được sử dụng.

4.2.3 Visibility Clique Cover (VCC)

Thuật toán Visibility Clique Cover (VCC) giải quyết trực tiếp một hạn chế chính của IRIS: nhu cầu về các điểm mẫu tốt.[9] Việc lấy mẫu ngẫu nhiên là không hiệu quả. VCC tự động hóa quá trình này bằng cách trước tiên tìm hiểu cấu trúc toàn cục của không gian tự do và sau đó đặt các điểm mẫu một cách chiến lược vào các khu vực có khả năng mở rộng lớn.

4.2.3.1 Ý tưởng và Chi tiết Thuật toán



Hình 4.9: Tổng quan về Quy trình VCC[9]

Quy trình VCC như đã thể hiện ở hình 4.9 bao gồm bốn bước chính:

1. Lấy mẫu & Xây dựng Đồ thị Khả kiến (Visibility Graphs): Thuật

toán lấy mẫu n điểm ngẫu nhiên trong không gian cấu hình tự do (C_{free}). Các điểm này tạo thành các đỉnh của một đồ thị khả kiến (visibility graph)¹. Một cạnh tồn tại giữa hai đỉnh nếu đoạn thẳng nối hai điểm tương ứng hoàn toàn nằm trong không gian tự do (tức là không va chạm).

2. Tìm Clique Cover: Thuật toán tìm một tập hợp các clique lớn (đồ thị con đầy đủ) trong đồ thị khả kiến. Ý tưởng cốt lõi là một tập hợp các điểm mà tất cả đều "nhìn thấy" nhau có khả năng nằm trong cùng một vùng lỗi². Điều này được thực hiện một cách tham lam bằng cách giải lặp đi lặp lại bài toán MAXCLIQUE (tìm clique lớn nhất), được xây dựng dưới dạng một bài toán Integer Linear Programming - ILP.
3. Khởi tạo Ellipsoid: Đối với mỗi clique được tìm thấy, thuật toán tính toán ellipsoid có thể tích nhỏ nhất bao quanh tất cả các điểm trong clique đó. Ellipsoid này cung cấp một tâm (điểm mẫu) và các trục chính (hình dạng ban đầu) cho bước lặp đầy tiếp theo.
4. Lấp đầy thành Đa diện: Tâm và hình dạng của mỗi ellipsoid được sử dụng để khởi tạo một vòng lặp duy nhất của thuật toán lấp đầy tương tự IRIS. Điều này hiệu quả hơn IRIS tiêu chuẩn vì ellipsoid ban đầu đã được "thông báo" về hình học cục bộ, thường loại bỏ sự cần thiết của nhiều vòng lặp tốn kém về mặt tính toán.[9]
5. Lấp lại và Hội tụ: Quá trình này được lặp lại cho đến khi đạt được độ phủ đủ bằng cách lấy mẫu mới từ không gian trống còn lại và lặp lại các bước trước đó.

¹ *Visibility graph* là đồ thị trong đó các đỉnh biểu diễn vị trí (hoặc đỉnh chướng ngại vật), và có cạnh nối giữa hai đỉnh nếu đoạn thẳng nối chúng nằm hoàn toàn trong không gian tự do.

² Nếu hai điểm có thể nhìn thấy nhau, điều đó ngụ ý rằng con đường thẳng nhất giữa chúng là an toàn. Đây là khối xây dựng cơ bản để hiểu và xấp xỉ các vùng lỗi trong.

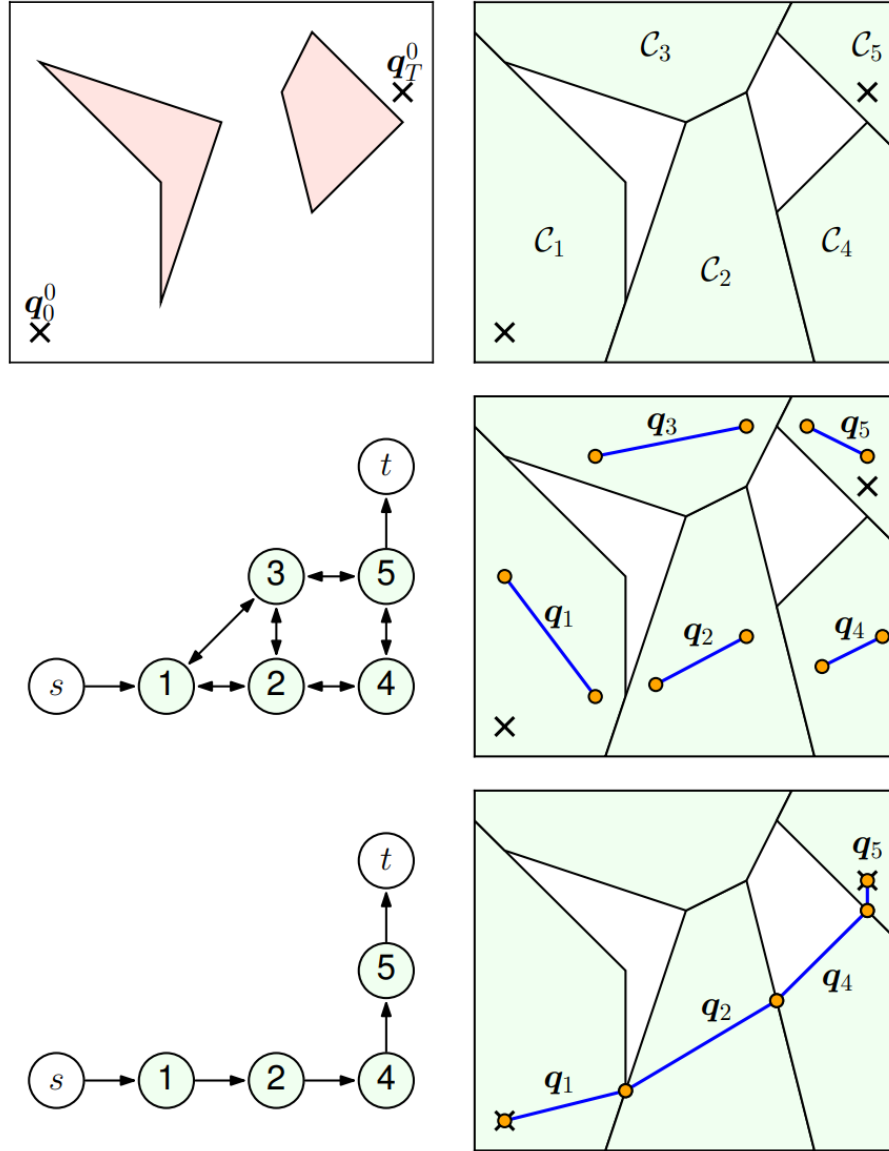
4.3 Hướng tiếp cận 1: Hoạch định chuyển động bằng Graph of Convex Sets (GCS)

4.3.1 Đồ thị của các Tập Lồi (Graphs of Convex Sets)

Graph of Convex Sets (GCS) Framework cung cấp một phương pháp luận thống nhất để giải quyết các bài toán tối ưu hóa lai ghép rời rạc-liên tục (hybrid discrete-continuous optimization). Nó đóng vai trò cầu nối hiệu quả giữa tìm kiếm đồ thị tổ hợp (combinatorial graph search) và quy hoạch lồi liên tục (continuous convex programming). Khác với lý thuyết đồ thị cổ điển, GCS tổng quát hóa Bài toán Đường đi Ngắn nhất (SPP) bằng cách liên kết trực tiếp các biến hình học liên tục với tô pô của đồ thị. Như được minh họa trong Hình 4.10, quá trình này bắt đầu bằng việc phân rã không gian an toàn thành các vùng lồi và xây dựng đồ thị giao. Một đường đi trên đồ thị này sẽ tương ứng với một chuỗi các phân đoạn quỹ đạo, và việc tối ưu hóa đường đi sẽ đồng thời xác định hình dạng hình học tối ưu của quỹ đạo đó.

4.3.1.1 Công thức Đường đi Ngắn nhất trong Graph of Convex Sets

Một GCS được định nghĩa là một đồ thị có hướng $G = (V, E)$, trong đó mỗi đỉnh $v \in V$ gắn liền với một biến quyết định liên tục x_v nằm trong một tập lồi compact, khác rỗng $X_v \subset \mathbb{R}^n$. Trong ngữ cảnh lập kế hoạch chuyển động (motion planning), các tập hợp này thường đại diện cho các vùng không va chạm đã được tính toán trước trong không gian cấu hình (configuration space). Các bước chuyển tiếp giữa các đỉnh được quy định bởi các cạnh $e = (u, v) \in E$, áp đặt các ràng buộc lồi phụ trợ $(x_u, x_v) \in X_e$ và được trọng số hóa bởi một hàm chi phí lồi $\ell_e(x_u, x_v)$. Hàm này bắt buộc phải là hàm thực sự (proper), đóng (closed) và lồi, liên kết tường minh các biến liên tục



Hình 4.10: Tối ưu hóa quỹ đạo có độ dài tối thiểu dưới dạng bài toán Tìm đường đi ngắn nhất (SPP) trong GCS. *Góc trên bên trái:* Môi trường với hai vật cản (màu hồng) cùng cấu hình khởi đầu q_0^0 và kết thúc q_T^0 . *Góc trên bên phải:* Phân rã vùng an toàn thành các vùng lõi C_i . *Giữa bên trái:* Đồ thị giao của các vùng phân rã, với các đỉnh nguồn s và đích t đại diện cho các điểm biên. *Giữa bên phải:* Các phân đoạn quỹ đạo q_i được gán ngẫu nhiên thỏa mãn ràng buộc tương ứng cho từng vùng lõi. *Dưới cùng:* Việc thiết lập một đường đi trong đồ thị giao sẽ kích hoạt các hàm chi phí và ràng buộc liên tục về mặt toán học lên hình dạng tổng thể của các phân đoạn quỹ đạo kết hợp [16].

x_u và x_v để mô hình hóa các thước đo chuyển tiếp như khoảng cách Euclid hoặc tiêu hao năng lượng. Cần lưu ý rằng, mặc dù sử dụng thuật ngữ này, chúng tôi không giả định ℓ_e là một metric hợp lệ; các tính chất như tính đối xứng hay bất đẳng thức tam giác không bắt buộc phải thỏa mãn [16]. Trong khi công thức luồng mạng (network flow) cổ điển giải quyết hiệu quả SPP trên các đồ thị rời rạc với chi phí cạnh tĩnh, khung GCS tổng quát hóa điều này bằng cách đưa vào các biến quyết định liên tục gắn liền với tô pô đồ thị. Trong GCS, chi phí di chuyển qua một cạnh $e = (u, v)$ không phải là một hằng số vô hướng c_{uv} , mà là một hàm lồi $\ell_e(x_u, x_v)$ phụ thuộc vào các trạng thái liên tục $x_u \in X_u$ và $x_v \in X_v$ tại các đỉnh liên thuộc. Mục tiêu của SPP trong GCS là xác định một đường đi p từ đỉnh nguồn s đến đỉnh đích t sao cho tối thiểu hóa tổng chi phí di chuyển qua các cạnh dọc theo đường đi, đồng thời lựa chọn các biến liên tục phù hợp tại mỗi đỉnh. Một cách hình thức, bài toán này được biểu diễn như sau:

$$\min_{p, \{x_v\}_{v \in p}} \sum_{(u,v) \in E_p} \ell_{(u,v)}(x_u, x_v) \quad (4.8)$$

$$\text{s.t. } p \in \mathcal{P}, \quad (4.9)$$

$$x_v \in X_v, \quad \forall v \in p, \quad (4.10)$$

$$(x_u, x_v) \in X_e, \quad \forall e = (u, v) \in E_p \quad (4.11)$$

trong đó $\mathcal{P}$ ký hiệu tập hợp tất cả các đường đi khả thi từ s đến t , và E_p đại diện cho các cạnh nằm trong đường đi p .

Bằng cách áp dụng công thức luồng mạng vào GCS, SPP có thể được biểu diễn dưới dạng bài toán Quy hoạch Phi tuyến Nguyên Hỗn hợp (Mixed-

Integer Nonlinear Program - MINLP) như sau:

$$\text{minimize} \quad \sum_{e=(u,v) \in \mathcal{E}} y_e \ell_e(x_u, x_v) \quad (4.12a)$$

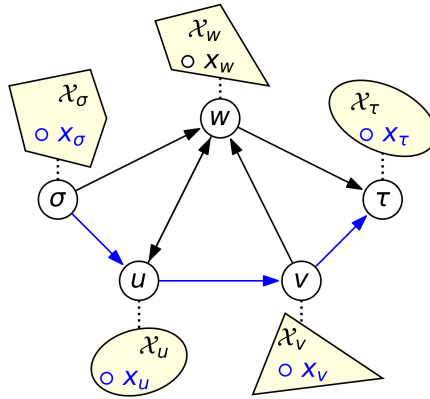
$$\text{subject to} \quad \sum_{e \in \mathcal{E}^+(u)} y_e - \sum_{e \in \mathcal{E}^-(u)} y_e = \delta_u, \quad \forall u \in \mathcal{V} \quad (4.12b)$$

$$(x_u, x_v) \in X_e, \quad \forall e = (u, v) \in \mathcal{E} \quad (4.12c)$$

$$x_v \in X_v, \quad \forall v \in \mathcal{V} \quad (4.12d)$$

$$y_e \in \{0, 1\}, \quad \forall e \in \mathcal{E} \quad (4.12e)$$

Trong công thức này, biến nhị phân y_e chỉ thị việc cạnh e có được đưa vào đường đi được chọn hay không ($y_e = 1$) hoặc không ($y_e = 0$). Ràng buộc bảo toàn luồng (4.12b) đảm bảo rằng các cạnh được chọn tạo thành một đường đi hợp lệ từ đỉnh nguồn s đến đỉnh đích t , trong đó δ_u được định nghĩa là 1 cho nguồn, -1 cho đích, và 0 cho tất cả các đỉnh khác. Các ràng buộc (4.12c) và (4.12d) bắt buộc các biến liên tục tại mỗi đỉnh và dọc theo mỗi cạnh phải tuân thủ các tập lỗi tương ứng của chúng.



Hình 4.11: Minh họa bài toán SPP in GCS. Mỗi đỉnh $i \in \{\sigma, u, v, w, \tau\}$ trong đồ thị được liên kết với một biến liên tục x_i nằm trong một tập lỗi tương ứng $\mathcal{X}_i$. Trọng số của mỗi cạnh là một hàm lỗi của các biến liên tục tại hai đỉnh mà cạnh đó kết nối. Trong hình, đường đi màu xanh $p = (\sigma, u, v, \tau)$ thể hiện một nghiệm khả thi của thành phần rời rạc, trong đó tổng chi phí phụ thuộc vào các giá trị tối ưu của $x_\sigma, x_u, x_v, x_\tau$ nhưng độc lập với x_w [16].

4.3.1.2 Phân tích Độ phức tạp

Độ phức tạp tính toán của SPP trong GCS khác biệt cơ bản so với Bài toán Đường đi Ngắn nhất cổ điển trên đồ thị rời rạc. Trong lý thuyết đồ thị cổ điển, SPP nói chung là NP-hard (ví dụ: bài toán Đường đi Dài nhất hoặc các biến thể liên quan đến chu trình âm). Tuy nhiên, nó trở nên giải được một cách hiệu quả trong thời gian đa thức (polynomial time) dưới hai điều kiện tiên quyết:

1. Đồ thị là **không chu trình** (acyclic) (cho phép giải bằng sắp xếp tô pô).
2. Tất cả chi phí cạnh và đỉnh là **không âm** (giải được bằng thuật toán Dijkstra).

Mệnh đề 1. *Trái ngược hoàn toàn với trường hợp trên, bài toán Đường đi ngắn nhất trong Graph of Convex Sets vẫn là **NP-hard** ngay cả khi đồng thời thỏa mãn hai điều kiện: đồ thị không chu trình và chi phí không âm.*

Độ khó này được thiết lập thông qua việc quy dẫn (reduction) từ bài toán **3SAT** (bài toán thỏa mãn Boolean với các mệnh đề gồm ba biến), vốn được biết đến là NP-đầy đủ (NP-complete). Như đã chứng minh trong Định lý 9.1 của [16], ta có thể xây dựng một thể hiện GCS phân lớp, không chu trình với chi phí không âm, trong đó việc xác định một đường đi hợp lệ tương đương với việc tìm một phép gán thỏa mãn cho công thức 3SAT. Điều này ngụ ý rằng chỉ riêng việc xác định tính *khả thi* (feasibility) của một SPP trong GCS đã là NP-hard. Các chứng minh khác về tính NP-hard cũng đã được đưa ra cho các trường hợp liên quan đến các tập lồi chiều thấp (1D và 2D), mặc dù các trường hợp này thường dựa trên các đồ thị có chứa chu trình [25].

4.3.1.3 Mô hình chi tiết các đỉnh và cạnh

Trong bối cảnh tối ưu quỹ đạo, các thành phần của GCS được định nghĩa một cách chặt chẽ nhằm đảm bảo cả tính an toàn hình học lẫn tính khả thi về động học.

Mỗi đỉnh $v \in V$ tương ứng với một vùng lõi, không va chạm Q_v trong không gian cấu hình (C-space). Biến quyết định liên tục $x_v \in \mathbb{R}^{n_v}$ gắn với đỉnh v biểu diễn một đoạn quỹ đạo cục bộ hoặc một cấu hình trạng thái. Cần nhấn mạnh rằng chiều n_v là cục bộ theo từng đỉnh, cho phép đồ thị có các đỉnh với số chiều biến khác nhau.

Biến x_v bị ràng buộc trong một tập lõi compact $\mathcal{X}_v \subset \mathbb{R}^{n_v}$. Đối với bài toán sinh quỹ đạo sử dụng các tham số đa thức từng đoạn (ví dụ như đường cong Bézier), tập $\mathcal{X}_v$ được xây dựng một cách tường minh nhằm đảm bảo quỹ đạo nằm hoàn toàn trong vùng an toàn Q_v và thỏa mãn các giới hạn động lực học.

Các ràng buộc xác định $\mathcal{X}_v$ thường có dạng:

$$r_{i,k} \in Q_i, \quad k = 0, \dots, d, \quad (4.13)$$

$$\dot{h}_{i,k} \geq \dot{h}_{\min} = 10^{-6}, \quad k = 0, \dots, d-1, \quad (4.14)$$

$$\dot{r}_{i,k} \in \dot{h}_{i,k} D, \quad k = 0, \dots, d-1, \quad (4.15)$$

$$h_{i,0} \geq 0, \quad h_{i,d} \leq T_{\max}. \quad (4.16)$$

Trong đó, (4.13) đảm bảo các điểm điều khiển không gian $r_{i,k}$ nằm trong vùng an toàn. Phương trình (4.14) đảm bảo tính đơn điệu chặt của hàm co giãn thời gian nhằm duy trì tính nhân quả. Phương trình (4.15) liên kết đạo hàm không gian và thời gian để thỏa mãn giới hạn vận tốc được xác định bởi tập D , và (4.16) áp đặt các giới hạn lên thời gian thực hiện quỹ đạo.

Một cạnh $e = (u, v) \in E$ xác định sự chuyển tiếp giữa hai đỉnh. Chuyển tiếp

này được điều khiển bởi một *tập ràng buộc liên kết* $\mathcal{X}_e \subseteq \mathbb{R}^{n_u+n_v}$. Tập này áp đặt các ràng buộc lên vector ghép (x_u, x_v) , đảm bảo các cặp cấu hình (x_u, x_v) là tương thích với nhau, chẳng hạn như thỏa mãn tính liên tục C^0 , C^1 hoặc bậc cao hơn giữa các đoạn quỹ đạo.

Gắn với mỗi cạnh là một *hàm chi phí liên kết* $f_e : \mathbb{R}^{n_u+n_v} \rightarrow \mathbb{R}$. Đây là một hàm lỗi giá trị vô hướng, đánh giá chi phí dựa trên việc lựa chọn đồng thời x_u và x_v , nhằm mô hình hóa các đại lượng như độ dài đường đi hoặc năng lượng tiêu thụ.

4.3.1.4 Đặc tả hàm chi phí

Việc lựa chọn hàm chi phí cạnh $\ell_e(x_u, x_v)$ mang tính then chốt, vì nó quy định ý nghĩa vật lý của quỹ đạo tối ưu. Khung làm việc của chúng tôi phân biệt giữa hai mô hình chính: lập kế hoạch đường đi hình học (LinearGCS) và tối ưu hóa quỹ đạo động học (BezierGCS), mỗi mô hình sử dụng các cấu trúc lõi cụ thể để đảm bảo hiệu quả tính toán.

4.3.1.4.a Độ dài Đường đi Hình học

Đối với các bài toán được xác định chặt chẽ bởi các ràng buộc hình học, chẳng hạn như tìm đường đi ngắn nhất qua các vùng lỗi, hàm chi phí được đặc trưng bởi khoảng cách Euclid có trọng số. Bình phương khoảng cách Euclid,

$$\ell_e(x_u, x_v) = \|x_v - x_u\|_2^2,$$

là một hàm bậc hai, phù hợp tự nhiên với các bộ giải Quadratic Programming (QP). Ngoài ra, chuẩn L_2 tiêu chuẩn

$$\ell_e(x_u, x_v) = \|x_v - x_u\|_2$$

có thể được mô hình hóa bằng Quy hoạch Nón Bậc hai (Second-Order Cone Programming - SOCP), giúp bảo toàn tính lồi. Các công thức này đảm bảo

rằng chi phí tỷ lệ tuyến tính với khoảng cách Euclid, mang lại một thước đo chính xác về mặt hình học cho độ dài đường đi.

4.3.1.4.b Chi phí Quỹ đạo Động lực học

Trong khung lập kế hoạch chuyển động đề xuất sử dụng đường cong Bézier, biến quyết định liên tục x_v được xây dựng để bao hàm cả các điểm điều khiển không gian và thời gian thực hiện của đoạn quỹ đạo. Để quản lý hiệu quả sự đánh đổi giữa tốc độ thực thi và nỗ lực điều khiển (control effort), hàm chi phí được xây dựng như một mục tiêu tổng hợp bao gồm ba thành phần riêng biệt.

Đầu tiên, để ưu tiên tính tối ưu về thời gian, một chi phí tuyến tính được áp dụng cho các biến co giãn thời gian h . Công thức này đảm bảo rằng việc tối thiểu hóa tổng thời gian quỹ đạo vẫn bảo toàn tính lồi của hàm mục tiêu. Chi phí thời gian được biểu diễn là:

$$J_t = w_t T_e, \quad (4.17)$$

trong đó w_t biểu thị hệ số trọng số được gán cho thời gian thực hiện đoạn T_e .

Đồng thời, để phạt nỗ lực điều khiển—cụ thể là tối thiểu hóa tích phân của bình phương vận tốc hoặc gia tốc—khung làm việc tận dụng các tính chất toán học của hàm phối cảnh (perspective functions). Đối với một đoạn quỹ đạo có thời gian T_e , chi phí năng lượng này được định nghĩa là:

$$J_e = \int_0^{T_e} |\dot{r}(t)|^2 dt. \quad (4.18)$$

Số hạng này hoạt động như một hàm phối cảnh có dạng bậc hai trên tuyến tính (quadratic-over-linear) $(x^T x/y)$. Một ưu điểm quan trọng của công thức này là nó lồi đồng thời (jointly convex) đối với cả các điểm điều khiển không gian và biến thời gian. Do đó, điều này cho phép bộ giải

tối ưu hóa đường đi hình học và phân bổ thời gian một cách đồng thời trong một khung Quy hoạch Lỗi Nguyên Hỗn hợp (Mixed-Integer Convex Programming - MICP) thống nhất.

Cuối cùng, để đảm bảo độ trơn bậc cao, chẳng hạn như tối thiểu hóa độ giật (jerk), hàm chi phí kết hợp điều chuẩn đạo hàm (derivative regularization). Khác với chi phí năng lượng, điều này đạt được bằng cách áp dụng các chi phí bậc hai tiêu chuẩn trực tiếp lên các hệ số của các đạo hàm bậc cao của quỹ đạo. Cách tiếp cận này tránh được các yêu cầu nghiêm ngặt của các công thức phối cảnh trong khi vẫn duy trì thành công cấu trúc lỗi tổng thể của bài toán tối ưu hóa.

4.3.1.5 Xây dựng Bài toán Quy hoạch Lỗi Nguyên Hỗn hợp thông qua Nới lỏng Phối cảnh

Việc giải quyết trực tiếp bài toán dưới dạng Quy hoạch Phi tuyến Nguyên Hỗn hợp (Mixed-Integer Nonlinear Programming - MINLP) đặt ra những thách thức tính toán đáng kể do sự hiện diện của các ràng buộc không lồi (non-convex constraints), đặc biệt là mối quan hệ song tuyến tính (bilinear relationship) giữa các biến quyết định nhị phân y_e và các biến trạng thái liên tục x_u . Để khắc phục điều này và chuyển đổi bài toán sang dạng Quy hoạch Lỗi Nguyên Hỗn hợp (Mixed-Integer Convex Programming - MICP) có thể giải quyết được, chúng tôi áp dụng kỹ thuật nới lỏng phối cảnh (perspective relaxation). Động lực chính của phương pháp này là cô lập tính không lồi bằng cách “nâng” (lifting) các biến liên tục vào một không gian có chiều cao hơn. Thay vì sử dụng các biến toàn cục x_u , chúng tôi giới thiệu các biến đại diện cục bộ (local proxy variables) $z_{e,u}, z_{e,v} \in \mathbb{R}^n$ cho mỗi cạnh $e = (u, v)$, đại diện cho sự đóng góp trạng thái của các đỉnh tương ứng khi cạnh đó được kích hoạt.

Đối với mỗi cạnh $e = (u, v)$, các biến đại diện cục bộ $z_{e,u}$ và $z_{e,v}$ mô hình hóa cụ thể sự đóng góp của trạng thái tại các đỉnh u và v vào việc di chuyển

qua cạnh e . Nếu cạnh không được kích hoạt (tức là $y_e = 0$), các đóng góp này bị buộc về không. Mỗi quan hệ này được biểu diễn như sau:

$$z_{e,u} \in y_e X_u \quad \text{và} \quad z_{e,v} \in y_e X_v$$

điều này ngụ ý:

$$\begin{cases} z_{e,u} \in X_u & \text{nếu } y_e = 1 \\ z_{e,u} = 0 & \text{nếu } y_e = 0 \end{cases}$$

Logic này được hình thức hóa bằng cách sử dụng hàm phối cảnh (perspective function) $\tilde{\ell}_e((z_{e,u}, z_{e,v}), y_e)$, là phối cảnh của hàm chi phí gốc ℓ_e . Một tính chất cơ bản trong giải tích lồi (convex analysis) phát biểu rằng nếu ℓ_e là lồi, thì phối cảnh $\tilde{\ell}_e$ của nó là lồi đồng thời (jointly convex) theo tất cả các đối số. Tính chất này rất quan trọng vì nó cho phép sử dụng các bộ giải lồi (convex solvers) hiệu quả. Hàm phối cảnh được định nghĩa là:

$$\tilde{\ell}_e(z_{e,u}, z_{e,v}, y_e) = \begin{cases} y_e \ell_e\left(\frac{z_{e,u}}{y_e}, \frac{z_{e,v}}{y_e}\right) & \text{nếu } y_e > 0 \\ 0 & \text{nếu } y_e = 0 \text{ và } z_{e,u} = z_{e,v} = 0 \\ +\infty & \text{trường hợp khác} \end{cases}$$

Quan trọng hơn, thay vì áp đặt các ràng buộc đẳng thức tường minh như $z_{e,u} = y_e x_u$, vốn không lồi, chúng tôi mở rộng luật bảo toàn luồng (flow conservation law) chuẩn (4.12b) sang các biến liên tục. Điều này bắt buộc rằng đối với bất kỳ nút u nào (ngoại trừ nguồn và đích), tổng các trạng thái liên tục đi vào nút thông qua các cạnh đến phải bằng tổng các trạng thái liên tục rời khỏi nút thông qua các cạnh đi.

Công thức MICP kết quả cho Bài toán Đường đi Ngắn nhất trong GCS được định nghĩa như sau:

$$\text{minimize} \quad \sum_{e=(u,v) \in \mathcal{E}} \tilde{\ell}_e((z_{e,u}, z_{e,v}), y_e) \quad (4.19a)$$

$$\text{subject to} \quad \sum_{e \in \mathcal{E}^+(u)} y_e - \sum_{e \in \mathcal{E}^-(u)} y_e = \delta_u, \quad \forall u \in \mathcal{V} \quad (4.19b)$$

$$\sum_{e \in \mathcal{E}^+(u)} z_{e,u} - \sum_{e \in \mathcal{E}^-(u)} z_{e,v} = \begin{cases} x_s & \text{nếu } u = s \\ -x_t & \text{nếu } u = t \\ 0 & \text{trường hợp khác} \end{cases}, \quad \forall u \in \mathcal{V} \quad (4.19c)$$

$$(z_{e,u}, z_{e,v}) \in y_e \mathcal{X}_e, \quad \forall e \in \mathcal{E} \quad (4.19d)$$

$$z_{e,u} \in y_e X_u, \quad z_{e,v} \in y_e X_v, \quad \forall e = (u, v) \in \mathcal{E} \quad (4.19e)$$

$$y_e \in \{0, 1\}, \quad \forall e \in \mathcal{E} \quad (4.19f)$$

Trong mô hình này, Phương trình (4.19b) đảm bảo tính liên thông tô pô (topological connectivity), trong đó δ_u được định nghĩa là 1 cho nguồn, -1 cho đích, và 0 cho các trường hợp khác. Phương trình (4.19c) đảm bảo rằng quỹ đạo liên tục là liên mạch trên cấu trúc đồ thị. Các phương trình (4.19d) và (4.19e) thực thi sự bao hàm hình học (geometric containment) bên trong các tập lỗi đã được tỉ lệ hóa; cụ thể, nếu $y_e = 0$, các biến liên tục liên quan bị ràng buộc về gốc tọa độ, phát sinh chi phí bằng không. Công thức này cung cấp một sự nói lỏng chặt chẽ và hiệu quả về mặt tính toán, cho phép bộ giải xác định các quỹ đạo tối ưu gần toàn cục (near-global optimal) thỏa mãn cả tô pô đồ thị rời rạc và các ràng buộc liên tục phức tạp vốn có trong lập kế hoạch chuyển động.

4.3.1.6 Lỗi hóa Bài toán GCS thông qua Kỹ thuật RLT

Để giải quyết tính không lồi vốn có của Chương trình Phi lồi Nguyên Hỗn hợp (Mixed-Integer Non-Convex Program - MINCP) ban đầu—cụ

thể là các ràng buộc song tuyến tính liên kết các biến quyết định nhị phân với các biến trạng thái liên tục—chúng tôi sử dụng chiến lược lồi hóa (convexification strategy) dựa trên Kỹ thuật Cải biến-Tuyến tính hóa (Reformulation-Linearization Technique - RLT) cấp độ một. Mục tiêu cơ bản của phương pháp này là thay thế các ràng buộc khó giải bằng các bao lồi (convex envelopes) chặt chẽ. Điều này đảm bảo rằng Chương trình Lồi Nguyên Hỗn hợp (MICP) kết quả bảo toàn tập khả thi nguyên của bài toán gốc đồng thời cung cấp các cận dưới mạnh (strong lower bounds) khi các ràng buộc nguyên được nối lỏng.

4.3.1.6.a Cơ sở Lý thuyết: Tính Đối ngẫu giữa Nón Phối cảnh và Bất đẳng thức Hợp lệ

Trước khi đi vào chi tiết mô hình tối ưu hóa, chúng ta phải thiết lập mối liên hệ toán học giữa các nón phối cảnh (perspective cones) và các bất đẳng thức tuyến tính. Xét nón đối ngẫu (dual cone) của nón phối cảnh $\tilde{X}$, ký hiệu là $(\tilde{X})^*$. Nón này được định nghĩa là tập hợp các vectơ (a, b) trong không gian mở rộng sao cho tích vô hướng của chúng với mọi phần tử trong nón $\tilde{X}$ là không âm:

$$(\tilde{X})^* := \{(a, b) \in \mathbb{R}^{n+1} \mid (a, b)^\top k \geq 0, \quad \forall k \in \tilde{X}\}. \quad (4.20)$$

Một kết quả quan trọng được sử dụng trong công thức này là sự tương đương giữa nón đối ngẫu $(\tilde{X})^*$ và tập hợp các bất đẳng thức tuyến tính hợp lệ (valid linear inequalities) cho tập X . Về mặt hình học, điều này đại diện cho giao của tất cả các nửa không gian đóng (closed half-spaces) chứa X . Chúng tôi hình thức hóa điều này trong mệnh đề sau:

Mệnh đề. Nón đối ngẫu $(\tilde{X})^*$ chính xác là tập hợp các hệ số (a, b) xác định các bất đẳng thức tuyến tính hợp lệ trên tập X ; nghĩa là, $(\tilde{X})^* \equiv X^\circ$.

Chứng minh. Để chứng minh điều này, xét một phần tử tùy ý $k \in \tilde{X}$. Theo định nghĩa của nón phối cảnh, k có thể được biểu diễn dưới dạng $k = (\lambda x, \lambda)$

với một $x \in X$ nào đó và một vô hướng $\lambda \geq 0$. Điều kiện để một cặp vectơ (a, b) thuộc về nón đối ngẫu $(\tilde{X})^*$ có thể được khai triển như sau:

$$\begin{aligned} (a, b) \in (\tilde{X})^* &\iff (a, b)^\top (\lambda x, \lambda) \geq 0, \quad \forall x \in X, \forall \lambda \geq 0 \\ &\iff \lambda(a^\top x + b) \geq 0, \quad \forall x \in X, \forall \lambda \geq 0. \end{aligned} \quad (4.21)$$

Quan sát thấy rằng bất đẳng thức cuối cùng phải thỏa mãn với mọi giá trị không âm của λ . Trong trường hợp $\lambda > 0$, ta có thể chia cả hai vế cho λ mà không làm thay đổi dấu bất đẳng thức, thu được điều kiện $a^\top x + b \geq 0$. Trường hợp $\lambda = 0$ được thỏa mãn một cách hiển nhiên. Do đó, điều kiện cần và đủ để $(a, b) \in (\tilde{X})^*$ là biểu thức tuyến tính $a^\top x + b$ vẫn không âm trên toàn bộ tập X . Điều này xác nhận rằng $(\tilde{X})^*$ cấu thành tập hợp các hệ số cho tất cả các bất đẳng thức hợp lệ trên X , hoàn tất chứng minh.

4.3.1.6.b Xây dựng các Ràng buộc Lỗi

Tận dụng mệnh đề trên, chúng ta có thể “nâng” bất kỳ ràng buộc tuyến tính hợp lệ nào từ không gian biến luồng (flow variable space) vào không gian hỗn hợp trạng thái-luồng (mixed state-flow space). Xét một đỉnh v tùy ý và một bất đẳng thức tuyến tính hợp lệ liên quan đến các biến luồng y_e kề với v , được cho bởi:

$$\sum_{e \in E_v} c_e y_e + d \geq 0, \quad (4.22)$$

trong đó E_v biểu thị tập hợp các cạnh kề với v , và d là một hằng số phụ thuộc vào bài toán. Bằng cách áp dụng nguyên lý phối cảnh, chúng tôi chuyển đổi bất đẳng thức (4.22) thành một ràng buộc bên trong nón lỗi $\tilde{X}_v$. Để tạo thuận lợi cho việc này, chúng tôi giới thiệu các biến đại diện z_e và z'_e . Biến z_e đại diện cho sự đóng góp trạng thái tại đỉnh nguồn của cạnh e khi được kích hoạt, trong khi z'_e đại diện cho sự đóng góp trạng thái tại đỉnh đích.

Sự chuyển đổi này mang lại ràng buộc nón lồi sau:

$$\left(\sum_{e \in E_{\text{in}}(v)} c_e z'_e + \sum_{e \in E_{\text{out}}(v)} c_e z_e + dx_v, \sum_{e \in E_v} c_e y_e + d \right) \in \tilde{X}_v. \quad (4.23)$$

Phương trình (4.23) ràng buộc mối quan hệ giữa các biến trạng thái trung gian (z, z') và các biến luồng y , đảm bảo chúng nằm trong nón phối cảnh của tập khả thi X_v .

Một ứng dụng cụ thể nhưng quan trọng của Bổ đề này liên quan đến ràng buộc không âm của luồng, tức là $y_e \geq 0$. Đối với một cạnh $e = (u, v)$:

- Tại đỉnh nguồn u , áp dụng việc nâng với $c_e = 1$ mang lại ràng buộc $(z_e, y_e) \in \tilde{X}_u$.
- Tại đỉnh đích v , áp dụng việc nâng với $c_e = 1$ mang lại ràng buộc $(z'_e, y_e) \in \tilde{X}_v$.

Các ràng buộc này hoạt động hiệu quả như một cơ chế “bật/tắt”: nếu $y_e = 1$, biến đại diện z_e bị ràng buộc vào tập X_u ; ngược lại, nếu $y_e = 0$, z_e bị buộc về gốc tọa độ.

4.3.1.6.c Công thức MICP Hoàn chỉnh

Tổng hợp các phân tích trước đó, chúng tôi đề xuất công thức Quy hoạch Lồi Nguyên Hỗn hợp (MICP) cho Bài toán Đường đi Ngắn nhất trên GCS, như được mô tả trong Định lý 5.7. Mô hình này loại bỏ hoàn toàn các thành phần song tuyến tính, thay thế chúng bằng các ràng buộc nón lồi:

$$\underset{y, z, z'}{\text{minimize}} \quad \sum_{e \in E} \tilde{\ell}_e((z_e, z'_e), y_e) \quad (5.5a)$$

$$\text{subject to} \quad \sum_{e \in E_{\text{out}}(s)} y_e = 1, \quad \sum_{e \in E_{\text{in}}(t)} y_e = 1, \quad (5.5b)$$

$$\sum_{e \in E_{\text{out}}(v)} y_e \leq 1, \quad \forall v \in V \setminus \{s, t\}, \quad (5.5c)$$

$$\sum_{e \in E_{\text{in}}(v)} (z'_e, y_e) = \sum_{e \in E_{\text{out}}(v)} (z_e, y_e), \quad \forall v \in V \setminus \{s, t\}, \quad (5.5d)$$

$$(z_e, y_e) \in \tilde{X}_u, \quad (z'_e, y_e) \in \tilde{X}_v, \quad \forall e = (u, v) \in E, \quad (5.5e)$$

$$y_e \in \{0, 1\}, \quad \forall e \in E. \quad (5.5f)$$

Công thức (4.24) bao gồm bốn thành phần chính đảm bảo tính liên thông đồ thị và tính nhất quán hình học.

Thứ nhất, hàm mục tiêu (5.5a) đại diện cho tổng chi phí trên tất cả các cạnh, trong đó $\tilde{\ell}_e$ là hàm phối cảnh của thước đo chi phí gốc. Vì các phép toán phối cảnh bảo toàn tính lồi, bài toán tối ưu hóa duy trì được cấu trúc lồi có thể giải quyết được.

Thứ hai, các ràng buộc (5.5b) và (5.5c) thực thi bảo toàn luồng cổ điển, đảm bảo sự tồn tại của một đường đi từ nguồn s đến đích t trong khi hạn chế mỗi đỉnh trung gian chỉ được ghé thăm tối đa một lần.

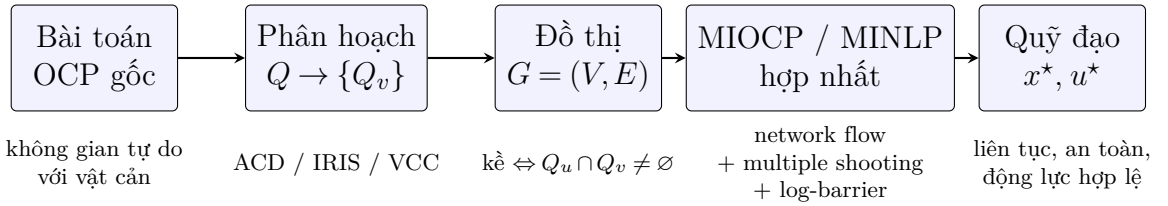
Thứ ba, ràng buộc (5.5d) đại diện cho bảo toàn luồng không gian (spatial flow conservation), một đặc điểm khác biệt so với các bài toán đường đi ngắn nhất tiêu chuẩn. Ràng buộc này được suy ra bằng cách nhân phương trình bảo toàn luồng tại đỉnh v với biến trạng thái x_v . Về mặt vật lý, nó đảm bảo tính liên tục của quỹ đạo: trạng thái tổng hợp “đi vào” đỉnh v (thông qua z'_e) phải bằng trạng thái tổng hợp “rời khỏi” đỉnh v (thông qua z_e).

Cuối cùng, các ràng buộc (5.5e) là kết quả của quá trình lồi hóa đã thảo luận trước đó. Chúng đóng vai trò là các bao lồi cho các biến đại diện z_e

và z'_e , đảm bảo rằng khi một cạnh $e = (u, v)$ được chọn ($y_e = 1$), các biến này tương ứng với các điểm hợp lệ nằm trong các tập X_u và X_v .

4.4 Hướng tiếp cận 2: Hoạch định chuyển động sử dụng Optimal Control và Multiple Shooting

Để khắc phục các hạn chế của hướng tiếp cận hình học thuần túy, chúng tôi đề xuất mô hình hóa bài toán hoạch định chuyển động thành một bài toán Điều khiển Tối ưu (OCP) trên đồ thị các vùng lỗi, sử dụng phương pháp direct multiple shooting. Mô hình này ghép hai khối lý thuyết: (i) Graphs of Convex Sets / network flow để chọn đường đi rời rạc qua các vùng an toàn lỗi, và (ii) direct multiple shooting để mô hình hóa quỹ đạo liên tục trong mỗi vùng lỗi. Phần ràng buộc an toàn hình học được hiện thực bằng cả hard constraint trên lưới hữu hạn lẫn hàm phạt log-barrier. Cấu trúc của mô hình hợp nhất được phân tích thành bốn lớp có thể xử lý tương đối độc lập (xem mục 4.4.10), tạo nền tảng cho chiến lược giải sẽ được trình bày ở mục A.1.



Hình 4.12: Quy trình tổng quan của hướng tiếp cận OCP+MMS. Bắt đầu từ bài toán điều khiển tối ưu liên tục với ràng buộc hình học phi lỗi, không gian tự do được phân hoạch thành một họ vùng lỗi; cấu trúc kề giữa các vùng được mã hóa thành đồ thị có hướng; việc chọn đường đi rời rạc và tối ưu hóa quỹ đạo cục bộ được hợp nhất thành một bài toán MIOCP/MINLP duy nhất; nghiệm thu được là cặp quỹ đạo trạng thái và điều khiển tối ưu thỏa mãn đồng thời ràng buộc động lực học và an toàn.

4.4.1 Bài toán gốc cần giải

Điểm xuất phát của phương pháp đề xuất là bài toán điều khiển tối ưu (optimal control problem, OCP) liên tục theo thời gian. Xét một hệ động lực học có phương trình trạng thái

$$\dot{x}(t) = f(x(t), u(t)), \quad t \in [0, T], \quad (4.25)$$

trong đó $x(t) \in \mathbb{R}^{n_x}$ là vector trạng thái, $u(t) \in \mathbb{R}^{n_u}$ là tín hiệu điều khiển, và $T > 0$ là chân trời thời gian — có thể được ấn định trước hoặc đưa vào như một biến quyết định tự do.

Cho trạng thái khởi đầu $x(0) = x^s$, trạng thái đích $x(T) = x^g$, không gian tự do $Q \subset \mathbb{R}^{n_x}$ (tức tập hợp các trạng thái không va chạm với bất kỳ vật cản nào), và tập điều khiển chấp nhận được $\mathcal{U} \subseteq \mathbb{R}^{n_u}$. Bài toán điều khiển tối ưu cần giải là

$$\min_{T, x(\cdot), u(\cdot)} \quad \Phi(x(T), T) + \int_0^T \ell(x(t), u(t)) dt \quad (4.26)$$

$$\text{s.t.} \quad \dot{x}(t) = f(x(t), u(t)), \quad t \in [0, T], \quad (4.27)$$

$$x(0) = x^s, \quad x(T) = x^g, \quad (4.28)$$

$$x(t) \in Q, \quad u(t) \in \mathcal{U}, \quad \forall t \in [0, T]. \quad (4.29)$$

Trong hàm mục tiêu (4.26), số hạng $\Phi(x(T), T)$ là chi phí cuối (terminal cost) phản ánh chất lượng trạng thái tại thời điểm kết thúc, còn $\ell(x(t), u(t))$ là chi phí tức thời (running cost) được tích lũy dọc theo toàn bộ quỹ đạo. Ràng buộc (4.27) mô tả phương trình động lực học của hệ thống, (4.28) là điều kiện biên tại hai đầu mút quỹ đạo, và (4.29) yêu cầu quỹ đạo phải nằm trong không gian tự do ở mọi thời điểm.

Thách thức chính khi giải bài toán (4.26)–(4.29) nằm ở ràng buộc $x(t) \in Q$: vì sự hiện diện của vật cản khiến Q là tập phi lồi, bài toán trở thành phi lồi và NP-hard nói chung. Ý tưởng cốt lõi của phương pháp đề xuất là *Phân*

hoạch Q thành một họ hữu hạn các vùng lỗi, sau đó mã hóa bài toán chọn chuỗi vùng cần đi qua dưới dạng luồng mạng nhị phân (binary network flow) trên một đồ thị có hướng, đồng thời giải bài toán động lực học liên tục cục bộ trong từng vùng bằng kỹ thuật direct multiple shooting.

4.4.2 Phân hoạch không gian tự do và xây dựng đồ thị

Giả sử không gian tự do Q đã được Phân hoạch thành một họ hữu hạn các vùng lỗi

$$\mathcal{Q} := \{Q_v\}_{v \in V_r}, \quad Q_v \subseteq Q \subset \mathbb{R}^{n_x},$$

trong đó mỗi Q_v là một tập lỗi đóng, bị chặn, và không giao với bất kỳ vật cản nào. Bước Phân hoạch này có thể thực hiện bằng các kỹ thuật như ACD, VCC, hay IRIS như đã trình bày ở chương trước.

Từ họ vùng lỗi $\mathcal{Q}$, ta xây dựng một đồ thị có hướng $G = (V, E)$ để mã hóa cấu trúc liên thông giữa các vùng. Tập đỉnh $V = V_r \cup \{\sigma, \tau\}$ gồm một đỉnh $v \in V_r$ tương ứng với mỗi vùng lỗi Q_v , cùng một đỉnh nguồn σ (đại diện cho trạng thái xuất phát x^s) và một đỉnh đích τ (đại diện cho trạng thái kết thúc x^g). Tập cạnh $E = E_{rr} \cup E_s \cup E_t$ được phân thành ba loại.

Hai vùng Q_u và Q_v được nối với nhau khi và chỉ khi chúng có phần giao không rỗng, tức là quỹ đạo có thể chuyển tiếp liên tục từ vùng này sang vùng kia:

$$E_{rr} := \{(u, v) \in V_r \times V_r : Q_u \cap Q_v \neq \emptyset\}. \quad (4.30)$$

Điều kiện này đảm bảo tại điểm chuyển tiếp giữa hai đoạn quỹ đạo kề nhau, trạng thái của rô-bốt có thể thuộc về cả hai vùng cùng lúc — tức là không có bước nhảy gián đoạn. Các cạnh nối với nguồn và đích được thêm riêng dựa trên vị trí của trạng thái đầu và cuối:

$$E_s := \{(\sigma, v) : x^s \in Q_v\}, \quad E_t := \{(v, \tau) : x^g \in Q_v\}. \quad (4.31)$$

Mỗi đường đi từ σ đến τ trên đồ thị G tương ứng với một chuỗi vùng lồi có thể ghép nối liên tục mà quỹ đạo đi qua. Bài toán điều khiển tối ưu vì vậy quy về việc đồng thời chọn đường đi tốt nhất trên đồ thị và tìm quỹ đạo tối ưu trong từng vùng dọc theo đường đi đó.

4.4.3 Biến quyết định của mô hình

Tương ứng với cấu trúc hỗn hợp rời rạc–liên tục của bài toán, ta chia biến quyết định thành ba lớp: biến nhị phân chọn đường đi, biến liên tục mô tả động lực học cục bộ, và biến liên kết tại điểm chuyển tiếp giữa các vùng.

4.4.3.1 Biến rời rạc: chọn đường đi trên đồ thị

Với mỗi cạnh $e = (u, v) \in E$, ta đặt biến nhị phân $y_{uv} \in \{0, 1\}$, trong đó $y_{uv} = 1$ có nghĩa là cạnh (u, v) thuộc đường đi được chọn, và $y_{uv} = 0$ nếu ngược lại. Với mỗi đỉnh vùng $v \in V_r$, ta đặt biến kích hoạt $p_v \in \{0, 1\}$, trong đó $p_v = 1$ khi và chỉ khi vùng Q_v thực sự nằm trên đường đi được chọn, tức là quỹ đạo đi qua Q_v .

Biến p_v không độc lập: nó được xác định hoàn toàn bởi các biến cạnh thông qua điều kiện bảo toàn luồng

$$p_v = \sum_{(u,v) \in E} y_{uv} = \sum_{(v,w) \in E} y_{vw}, \quad v \in V_r.$$

Đẳng thức này cho thấy $p_v = 1$ khi và chỉ khi có đúng một cạnh đi vào v và đúng một cạnh đi ra khỏi v được chọn — tức là v nằm trên đường đi.

4.4.3.2 Biến liên tục cục bộ trên mỗi vùng

Với mỗi vùng $v \in V_r$, ta gán một đoạn quỹ đạo cục bộ được tham số hóa trên miền thời gian chuẩn hóa $\tau \in [0, 1]$. Bộ biến liên tục mô tả đoạn này

gồm:

- $s_v^- \in \mathbb{R}^{n_x}$: trạng thái tại thời điểm vào vùng v (điểm bắt đầu),
- $s_v^+ \in \mathbb{R}^{n_x}$: trạng thái tại thời điểm rời vùng v (điểm cuối đoạn),
- $\Delta_v \geq 0$: thời lượng vật lý của đoạn quỹ đạo trong vùng v ,
- $w_v \in \mathbb{R}^{m_v}$: tham số hóa tín hiệu điều khiển trên vùng v .

Trong kỹ thuật direct multiple shooting, mỗi đoạn có một giá trị ban đầu riêng s_v^- được tối ưu hóa đồng thời với các biến còn lại, thay vì tích phân tuần tự từ trạng thái ban đầu qua toàn bộ chân trời thời gian. Điều này giúp cho bài toán được song song hóa theo từng vùng và cải thiện tính ổn định số của quá trình tối ưu hóa. Tính liên tục của quỹ đạo tại điểm nối giữa các đoạn sẽ được đảm bảo thông qua các ràng buộc ghép nối ở mục sau.

4.4.3.3 Biến liên kết tại điểm chuyển tiếp giữa hai vùng

Với mỗi cạnh nội bộ $(u, v) \in E_{rr}$, ta đặt biến interface $z_{uv} \in \mathbb{R}^{n_x}$ đại diện cho trạng thái vật lý của rô-bốt tại khoảnh khắc chuyển từ vùng Q_u sang vùng Q_v . Khi cạnh (u, v) được chọn ($y_{uv} = 1$), trạng thái này phải nằm trong phần giao $Q_u \cap Q_v$ — đảm bảo tính hợp lệ của điểm nối: z_{uv} vừa là điểm cuối của đoạn trong Q_u , vừa là điểm đầu của đoạn trong Q_v , và đồng thời thuộc cả hai vùng an toàn.

4.4.4 Ràng buộc network flow cho phần rời rạc

Các ràng buộc network flow đảm bảo rằng tập hợp các cạnh được chọn tạo thành đúng một đường đi đơn giản (không có chu trình, không phân nhánh) từ đỉnh nguồn σ đến đỉnh đích τ .

Đầu tiên, ta yêu cầu đúng một cạnh đi ra khỏi nguồn và đúng một cạnh đi

vào đích:

$$\sum_{(\sigma,v) \in E} y_{\sigma v} = 1, \quad \sum_{(v,\tau) \in E} y_{v\tau} = 1. \quad (4.32)$$

Tiếp theo, bảo toàn luồng tại mỗi đỉnh vùng $v \in V_r$ đảm bảo số cạnh đi vào bằng số cạnh đi ra, đồng thời liên kết biến p_v với các biến cạnh:

$$\sum_{(u,v) \in E} y_{uv} = \sum_{(v,w) \in E} y_{vw} = p_v. \quad (4.33)$$

Cuối cùng, ràng buộc loại chu trình ngăn đường đi quay lại một vùng đã đi qua, tức là mỗi vùng xuất hiện tối đa một lần:

$$\sum_{e \in I_v^{\text{out}}} y_e \leq 1, \quad \forall v \in V_r. \quad (4.34)$$

Ký hiệu I_v^{out} là tập tất cả các cạnh đi ra khỏi đỉnh v . Ràng buộc (4.34) đóng vai trò then chốt về mặt tính toán: nó loại trừ các vòng lặp vô tận trong không gian quyết định, khiến bài toán network flow có nghiệm hữu hạn.

4.4.5 Mô hình động lực học cục bộ theo direct multiple shooting

4.4.5.1 Tham số hóa tín hiệu điều khiển trên từng vùng

Để đưa bài toán điều khiển tối ưu về dạng chương trình phi tuyến hữu hạn chiều (nonlinear program, NLP) có thể giải bằng các bộ giải số, tín hiệu điều khiển liên tục $u(\cdot)$ trên mỗi vùng v được tham số hóa bởi một vector hữu hạn chiều w_v :

$$u_v(\tau) = \mathcal{U}_v(\tau; w_v), \quad \tau \in [0, 1]. \quad (4.35)$$

Lớp hàm $\mathcal{U}_v$ có thể là điều khiển hằng, piecewise constant, hay spline bậc thấp, tùy theo yêu cầu về tính mượt của quỹ đạo. Với cách tham số hóa này, bài toán tối ưu hóa hàm $u(\cdot)$ trong không gian hàm vô hạn chiều được rút gọn về bài toán tối ưu hóa vector w_v hữu hạn chiều.

4.4.5.2 Bài toán giá trị đầu cực bộ trên từng vùng

Mỗi vùng $v \in V_r$ được gán với một đoạn quỹ đạo tham số hóa trên miền chuẩn hóa $\tau \in [0, 1]$. Việc chuẩn hóa thời gian này cho phép thời lượng Δ_v trở thành biến quyết định: thay vì tích phân trên khoảng thời gian vật lý $[0, \Delta_v]$ (biến động theo từng vùng), ta luôn tích phân trên $[0, 1]$ cố định và đưa Δ_v vào như một hệ số tỉ lệ. Phương trình động lực học sau chuẩn hóa thành

$$\begin{cases} \frac{dx_v}{d\tau}(\tau) = \Delta_v f(x_v(\tau), u_v(\tau)), & \tau \in [0, 1], \\ x_v(0) = s_v^-, \\ u_v(\tau) = \mathcal{U}_v(\tau; w_v). \end{cases} \quad (4.36)$$

Hệ (4.36) là bài toán giá trị đầu (initial value problem, IVP) với điều kiện ban đầu là biến bản s_v^- . Nghiệm $x_v(\tau)$ của hệ này, khi được tích phân đến $\tau = 1$, cho trạng thái cuối của đoạn trong vùng v ; ta định nghĩa ánh xạ endpoint

$$F_v(s_v^-, w_v, \Delta_v) := x_v(1). \quad (4.37)$$

4.4.5.3 Ràng buộc defect của multiple shooting

Trong phương pháp direct multiple shooting, s_v^+ là biến độc lập đại diện cho trạng thái cuối đoạn trong vùng v , và không nhất thiết trùng với giá trị $F_v(s_v^-, w_v, \Delta_v)$ trong quá trình tối ưu hóa. Tính liên tục của quỹ đạo được áp đặt thông qua *ràng buộc defect*:

$$s_v^+ - F_v(s_v^-, w_v, \Delta_v) = 0, \quad \forall v \in V_r. \quad (4.38)$$

Ràng buộc này yêu cầu điểm cuối của IVP tích phân từ s_v^- phải khớp chính xác với biến đầu ra s_v^+ . Tại nghiệm tối ưu, tất cả các ràng buộc defect đều bằng 0, và quỹ đạo tổng thể là liên tục hoàn toàn.

4.4.6 Ràng buộc an toàn hình học

Khi vùng v được kích hoạt ($p_v = 1$), đoạn quỹ đạo cục bộ trong vùng đó phải nằm hoàn toàn trong vùng an toàn Q_v :

$$x_v(\tau) \in Q_v, \quad \forall \tau \in [0, 1], \quad \text{khi } p_v = 1. \quad (4.39)$$

Đây là ràng buộc continuous-time vô hạn chiều — nó phải thỏa mãn tại mọi $\tau \in [0, 1]$, không chỉ tại hữu hạn điểm rời rạc. Để đưa vào bộ giải số, ta cần xấp xỉ hoặc thay thế nó bằng một dạng hữu hạn chiều. Hai cách tiếp cận chính được trình bày dưới đây.

4.4.6.1 Cách 1: Xấp xỉ qua lưới hữu hạn điểm (hard constraint)

Cách tiếp cận trực tiếp nhất là kiểm tra ràng buộc an toàn tại $M_v + 1$ điểm lưới trên $[0, 1]$:

$$0 = \tau_{v,0} < \tau_{v,1} < \dots < \tau_{v,M_v} = 1.$$

Giả sử Q_v là polytope $Q_v = \{x \in \mathbb{R}^{n_x} : A_v x \leq b_v\}$, điều kiện an toàn rời rạc hóa thành

$$A_v x_v(\tau_{v,k}) \leq b_v - \varepsilon_v, \quad k = 0, \dots, M_v,$$

với $\varepsilon_v > 0$ là biên an toàn (safety margin). Cách này đơn giản và không thay đổi cấu trúc hàm mục tiêu, nhưng chỉ đảm bảo an toàn tại các điểm lưới; vi phạm giữa các điểm vẫn có thể xảy ra với bước lưới đủ thô.

4.4.6.2 Cách 2: Hàm cản logarit (log-barrier)

Một hướng tiếp cận có nền tảng lý thuyết chắc hơn là sử dụng hàm cản logarit (log-barrier) — một kỹ thuật cổ điển trong tối ưu hóa điểm trong (interior-point optimization). Ý tưởng là thay ràng buộc cứng $A_v x \leq b_v$ bằng một số hạng phạt trong hàm mục tiêu, số hạng này tiến về $+\infty$ khi quỹ đạo tiếp cận biên vùng lồi. Với polytope $Q_v = \{x : A_v x \leq b_v\}$ có m_v bất đẳng thức, ta định nghĩa hàm cản

$$\phi_v(x) = - \sum_{j=1}^{m_v} \log((b_v)_j - (A_v x)_j). \quad (4.40)$$

Hàm ϕ_v xác định trên phần nội của Q_v (nơi mọi $(b_v)_j - (A_v x)_j > 0$) và thỏa mãn $\phi_v(x) \rightarrow +\infty$ khi x tiến đến bất kỳ mặt biên nào của Q_v . Do đó, khi cộng ϕ_v vào hàm mục tiêu với trọng số $\mu_v > 0$, bộ giải bị “đẩy” tự nhiên ra xa biên, giúp quỹ đạo nằm sâu trong vùng an toàn mà không cần ràng buộc tường minh. Bài toán OCP cục bộ trên vùng v khi đó có dạng:

$$\begin{aligned} \min_{x(\cdot), u(\cdot)} \quad & \int_0^{T_v} [\ell(x(t), u(t)) + \mu_v \phi_v(x(t))] dt \\ \text{s.t.} \quad & \dot{x}(t) = f(x(t), u(t)), \quad x(0) = x_v^{\text{in}}, \quad x(T_v) = x_v^{\text{out}}. \end{aligned}$$

Tham số μ_v điều chỉnh mức độ “chặt” của rào cản: khi μ_v lớn, quỹ đạo bị ép về gần tâm giải tích (analytic center) của Q_v ; khi giảm μ_v dần về 0, quỹ đạo xấp xỉ nghiệm của bài toán gốc có ràng buộc cứng. Trong thực tế, ta giải một chuỗi bài toán với μ_v giảm dần (thường từ 10^{-2} đến 10^{-5}), dùng nghiệm bài toán trước khởi tạo cho bài toán sau.

Sau khi rời rạc hóa bằng multiple shooting với N_v nút và bước thời gian

$\Delta t = \Delta_v / N_v$, bài toán NLP trên một vùng trở thành:

$$\begin{aligned} \min_{\{x_k, u_k\}_{k=0}^{N_v-1}} \quad & \sum_{k=0}^{N_v-1} \left[\ell(x_k, u_k) \Delta t - \mu_v \sum_{j=1}^{m_v} \log((b_v)_j - (A_v x_k)_j) \right] \\ \text{s.t.} \quad & x_{k+1} = \Phi(x_k, u_k), \\ & x_0 = x_v^{\text{in}}, \quad x_{N_v} = x_v^{\text{out}}, \end{aligned} \tag{4.41}$$

trong đó $\Phi(x_k, u_k)$ là bản đồ chuyển trạng thái thu được bằng cách tích phân phương trình (4.25) từ x_k với điều khiển u_k trong khoảng thời gian Δt . Lưu ý rằng điều kiện biên $x_{N_v} = x_v^{\text{out}}$ được giữ nguyên là ràng buộc đẳng thức, không bị hàm cản thay thế — hàm cản chỉ tác động lên các ràng buộc bất đẳng thức xác định biên vùng lỗi.

4.4.7 Ràng buộc ghép nối

Các ràng buộc ghép nối liên kết phần rời rạc (đường đi trên đồ thị) với phần liên tục (động lực học cục bộ), đảm bảo rằng khi cạnh (u, v) được chọn, đoạn quỹ đạo trong Q_u và đoạn trong Q_v khớp nhau tại điểm chuyển tiếp.

4.4.7.1 Ghép nối tại điểm chuyển tiếp giữa hai vùng kề nhau

Với mỗi cạnh nội bộ $(u, v) \in E_{rr}$, khi cạnh này được chọn ($y_{uv} = 1$), biến interface z_{uv} phải đồng thời là điểm cuối của đoạn trong Q_u và điểm đầu của đoạn trong Q_v , đồng thời thuộc phần giao của hai vùng:

$$y_{uv} = 1 \implies \begin{cases} z_{uv} \in Q_u \cap Q_v, \\ s_u^+ = z_{uv}, \\ s_v^- = z_{uv}. \end{cases} \tag{4.42}$$

Điều kiện $z_{uv} \in Q_u \cap Q_v$ đảm bảo điểm nối nằm trong vùng an toàn của cả hai vùng liền kề, trong khi hai đẳng thức $s_u^+ = z_{uv}$ và $s_v^- = z_{uv}$ thiết lập tính liên tục của quỹ đạo tại điểm này.

4.4.7.2 Điều kiện biên tại nguồn và đích

Khi cạnh từ nguồn $(\sigma, v) \in E_s$ được chọn, điểm vào của đoạn quỹ đạo đầu tiên phải khớp với trạng thái xuất phát:

$$y_{\sigma v} = 1 \implies s_v^- = x^s, \quad \forall (\sigma, v) \in E_s.$$

Tương tự, khi cạnh đến đích $(v, \tau) \in E_t$ được chọn, điểm ra của đoạn quỹ đạo cuối cùng phải khớp với trạng thái kết thúc:

$$y_{v\tau} = 1 \implies s_v^+ = x^g, \quad \forall (v, \tau) \in E_t.$$

Hai nhóm ràng buộc trên, kết hợp với (4.42), đảm bảo rằng chuỗi đoạn quỹ đạo cục bộ tạo thành một quỹ đạo liên tục hoàn chỉnh từ x^s đến x^g .

4.4.8 Hàm mục tiêu

Ta chọn chi phí tức thời có dạng

$$\ell(x(t), u(t)) := w_L \|\dot{q}(t)\|^2 + w_E \|u(t)\|^2, \quad q(t) := \pi(x(t)), \quad (4.43)$$

trong đó $\pi : \mathbb{R}^{n_x} \rightarrow \mathbb{R}^{n_q}$ là phép chiếu lấy phần cấu hình hình học của trạng thái, $\dot{q}(t)$ là vận tốc hình học, và $w_L, w_E > 0$ là các trọng số. Số hạng $w_L \|\dot{q}\|^2$ phạt vận tốc lớn (khuyến khích quỹ đạo đi chậm, mượt), còn $w_E \|u\|^2$ phạt năng lượng điều khiển (khuyến khích tín hiệu điều khiển nhỏ).

Tổng chi phí cục bộ trên vùng v trong miền thời gian chuẩn hóa $\tau \in [0, 1]$

là

$$J_v(s_v^-, w_v, \Delta_v) := a \Delta_v + \int_0^1 \Delta_v \left(w_L \left\| \frac{1}{\Delta_v} \frac{dq_v}{d\tau}(\tau) \right\|^2 + w_E \|u_v(\tau)\|^2 \right) d\tau, \quad (4.44)$$

trong đó số hạng $a \Delta_v$ (với $a > 0$) phạt thời lượng của đoạn, khuyến khích robot đi qua vùng v nhanh hơn. Phép đổi biến $\frac{dq_v}{d\tau} = \Delta_v \dot{q}_v$ cho thấy $\frac{1}{\Delta_v} \frac{dq_v}{d\tau} = \dot{q}_v$ chính là vận tốc vật lý; tích phân $\Delta_v d\tau = dt$, nên J_v tương đương với chi phí tức thời (4.43) tích phân trên thời gian vật lý $[0, \Delta_v]$.

Khi kết hợp với hàm cản log-barrier để đảm bảo an toàn hình học (xem mục 4.4.6.2), hàm mục tiêu toàn cục trở thành:

$$\min \sum_{v \in V_r} p_v \left[J_v(s_v^-, w_v, \Delta_v) + \int_0^1 \Delta_v \mu_v \phi_v(x_v(\tau)) d\tau \right]. \quad (4.45)$$

Nhân tử p_v đảm bảo rằng chỉ các vùng thuộc đường đi được chọn mới đóng góp vào chi phí tổng.

4.4.9 Mô hình tổng hợp

Tập hợp tất cả các thành phần đã trình bày, bài toán tối ưu hóa tổng hợp được viết đầy đủ như sau:

$$\min_{\substack{y, p, z, \\ \{s_v^-, s_v^+, w_v, \Delta_v\}_{v \in V_r}}} \sum_{v \in V_r} p_v \left[J_v(s_v^-, w_v, \Delta_v) + \int_0^1 \Delta_v \mu_v \phi_v(x_v(\tau)) d\tau \right] \quad (4.46)$$

$$\text{s.t.} \quad \sum_{(\sigma, v) \in E} y_{\sigma v} = 1, \quad \sum_{(v, \tau) \in E} y_{v\tau} = 1, \quad (4.47)$$

$$\sum_{(u, v) \in E} y_{uv} = \sum_{(v, w) \in E} y_{vw} = p_v, \quad \forall v \in V_r, \quad (4.48)$$

$$\sum_{e \in I_v^{\text{out}}} y_e \leq 1, \quad \forall v \in V_r, \quad (4.49)$$

$$x'_v(\tau) = \Delta_v f(x_v(\tau), u_v(\tau)), \quad \tau \in [0, 1], \quad \forall v \in V_r, \quad (4.50)$$

$$x_v(0) = s_v^-, \quad u_v(\tau) = \mathcal{U}_v(\tau; w_v), \quad \forall v \in V_r, \quad (4.51)$$

$$s_v^+ - F_v(s_v^-, w_v, \Delta_v) = 0, \quad \forall v \in V_r, \quad (4.52)$$

$$x_v(\tau) \in Q_v, \quad \forall \tau \in [0, 1], \quad \forall v \in V_r \text{ với } p_v = 1, \quad (4.53)$$

$$y_{uv} = 1 \implies \begin{cases} z_{uv} \in Q_u \cap Q_v, \\ s_u^+ = z_{uv}, \\ s_v^- = z_{uv}, \end{cases} \quad \forall (u, v) \in E_{rr}, \quad (4.54)$$

$$y_{\sigma v} = 1 \implies s_v^- = x^s, \quad \forall (\sigma, v) \in E_s, \quad (4.55)$$

$$y_{v\tau} = 1 \implies s_v^+ = x^g, \quad \forall (v, \tau) \in E_t, \quad (4.56)$$

$$y_{uv} \in \{0, 1\}, \quad p_v \in \{0, 1\}, \quad \forall (u, v) \in E, \quad \forall v \in V_r. \quad (4.57)$$

Mỗi nhóm ràng buộc đảm nhận một vai trò riêng biệt:

- (4.47)–(4.49): ràng buộc network flow, đảm bảo tập cạnh được chọn

tạo thành đúng một đường đi đơn giản từ nguồn đến đích, không có nhánh phụ hay chu trình;

- (4.50)–(4.52): khối động lực học cục bộ, bao gồm phương trình IVP sau chuẩn hóa thời gian và ràng buộc defect của multiple shooting đảm bảo tính liên tục nội bộ trong từng vùng;
- (4.53): ràng buộc an toàn hình học continuous-time, được hiện thực hóa bằng sampling lưới hoặc log-barrier như đã trình bày trong mục 4.4.6;
- (4.54)–(4.56): ràng buộc ghép nối, liên kết phần rời rạc với phần liên tục để đảm bảo tính liên tục toàn cục của quỹ đạo qua các điểm chuyển tiếp;
- (4.57): ràng buộc nhị phân trên các biến chọn đường đi.

4.4.10 Phân tách cấu trúc bài toán thành bốn lớp

Mặc dù bài toán (4.46)–(4.57) ghép phần rời rạc và phần liên tục thành một chương trình duy nhất, cấu trúc bên trong của nó cho phép tách thành bốn lớp có thể xử lý tương đối độc lập. Phép phân hoạch này không thay đổi nghiệm tối ưu mà cung cấp khung nhìn để thiết kế chiến lược giải (xem mục A.1):

1. **Lớp discrete path:** ràng buộc network flow (4.47), (4.48), (4.49) — hoàn toàn tuyến tính nguyên, có thể giải bằng các bộ giải ILP;
2. **Lớp convex/on–off geometry:** các ràng buộc membership được homogenization/perspective để xử lý điều kiện bật/tắt theo biến nhị phân:

$$\begin{aligned} (s_v^-, p_v), (s_v^+, p_v) &\in \text{cl}(\widetilde{Q}_v), & \forall v \in V_r, \\ (z_{uv}, y_{uv}) &\in \text{cl}(\widetilde{Q_u \cap Q_v}), & \forall (u, v) \in E_{rr}; \end{aligned}$$

3. **Lớp ghép nối on-off**: ràng buộc (4.54)–(4.56), bật/tắt các đẳng thức ghép nối đồng bộ với biến nhị phân tương ứng;
4. **Lớp nonlinear multiple-shooting dynamics**: với mỗi vùng $v \in V_r$,

$$\begin{aligned}
p_v = 1 &\implies \begin{cases} \Delta_v \geq \underline{\Delta}_v > 0, \\ \text{IVP cục bộ (4.36) được kích hoạt,} \\ s_v^+ - F_v(s_v^-, w_v, \Delta_v) = 0, \\ \text{ràng buộc an toàn (sampling hoặc log-barrier),} \\ \rho_v \geq \hat{J}_v(s_v^-, w_v, \Delta_v), \end{cases} \\
p_v = 0 &\implies \begin{cases} \Delta_v = 0, \quad \rho_v = 0, \\ s_v^- = s_v^+ = 0, \quad w_v = \bar{w}_v. \end{cases}
\end{aligned}$$

Nhờ vậy, hàm mục tiêu toàn cục được viết gọn thành $\min \sum_{v \in V_r} \rho_v$, trong đó ρ_v là biến epigraph cho chi phí dynamic cục bộ trên vùng v , được bật/tắt đồng bộ với p_v . Khi $p_v = 0$, vùng v không thuộc đường đi và tất cả các biến liên tục gắn với nó được đặt về 0 — điều này tránh đưa vào bài toán các IVP thừa không cần thiết.

4.4.11 Phân loại bài toán

Từ mô hình tổng hợp (4.46)–(4.57), có thể thấy bài toán đề xuất là một bài toán tối ưu lai ghép rời rạc–liên tục. Tính hỗn hợp nguyên xuất hiện từ các biến nhị phân y_{uv} và p_v , dùng để quyết định đường đi trên đồ thị vùng lỗi. Tính phi tuyến xuất hiện từ khối điều khiển tối ưu liên tục: trạng thái $x_v(\cdot)$ phải thỏa phương trình động lực học (4.50), tín hiệu điều khiển được tham số hóa bởi w_v , thời lượng đoạn Δ_v là biến quyết định, và các đoạn quỹ đạo được nối bằng ràng buộc defect của multiple shooting.

Vì vậy, nếu xét ở mức liên tục theo thời gian, trước khi thay thế các hàm

trạng thái và điều khiển bằng hữu hạn biến số, bài toán vẫn chứa đồng thời biến nhị phân và các ràng buộc vi phân phi tuyến. Đây chính là dạng *Mixed-Integer Nonlinear Optimal Control Problem* (MIOCP). Sau khi áp dụng direct multiple shooting, các hàm $x_v(\cdot)$ và $u_v(\cdot)$ được thay bằng các biến hữu hạn chiều $\{s_v^-, s_v^+, w_v, \Delta_v\}$ cùng các ràng buộc defect. Khi đó bài toán trở thành một chương trình tối ưu hữu hạn chiều có cả biến nguyên và ràng buộc phi tuyến, tức *Mixed-Integer Nonlinear Program* (MINLP). Nói cách khác, mô hình được phân loại là MIOCP ở dạng liên tục theo thời gian và là MINLP sau khi rời rạc hóa bằng multiple shooting.

Điểm quan trọng là bài toán này **không thể quy trực tiếp về Mixed-Integer Second Order Cone Problem** (MISOCP) như trong GCS-Bézier của Marcucci và cộng sự [16], [25]. Một MISOCP yêu cầu các ràng buộc liên tục có dạng tuyến tính hoặc nón bậc hai lồi. Trong GCS-Bézier, điều này đạt được nhờ tham số hóa quỹ đạo bằng các điểm điều khiển Bézier: ràng buộc nằm trong vùng lồi có thể áp lên các điểm điều khiển thông qua tính chất convex hull, còn các ràng buộc độ dài, vận tốc hoặc gia tốc thường được viết dưới dạng chuẩn Euclid và đưa về SOCP.

Ngược lại, trong mô hình OCP+MMS, ràng buộc defect

$$s_v^+ - F_v(s_v^-, w_v, \Delta_v) = 0$$

phụ thuộc vào endpoint map F_v , tức nghiệm tại $\tau = 1$ của IVP phi tuyến (4.36). Nói cách khác, F_v không phải là một biểu thức affine, đa thức lồi, hay nón bậc hai; nó là một hàm ẩn sinh ra bởi quá trình tích phân động lực học. Ngay cả khi f trơn, ánh xạ này nói chung vẫn phi tuyến và không lồi theo (s_v^-, w_v, Δ_v) . Ngoài ra, việc đưa Δ_v vào phương trình chuẩn hóa thời gian qua tích $\Delta_v f(x_v, u_v)$ cũng tạo thêm quan hệ phi tuyến giữa thời lượng và động lực học. Nếu dùng log-barrier, các số hạng $-\log((b_v)_j - (A_v x)_j)$ trong hàm mục tiêu tiếp tục làm mất cấu trúc conic lồi.

Do đó, chỉ trong các trường hợp rất đặc biệt, chẳng hạn động lực học tuyến

tính đơn giản, thời lượng cố định, và ràng buộc hình học được viết hoàn toàn bằng các chuẩn lỗi, bài toán mới có thể được xấp xỉ hoặc biến đổi về một lớp lỗi hỗn hợp nguyên. Với hệ động lực học tổng quát đang xét, phân loại đúng của mô hình là MIOCP ở cấp liên tục và MINLP sau rời rạc hóa. Hệ quả trực tiếp là ta không còn thừa hưởng bảo đảm convex relaxation chặt và hiệu năng giải của các bộ giải MISOCP như Mosek/Gurobi; thay vào đó, cần dùng chiến lược giải phù hợp với MINLP phi lỗi, như tách pha chọn đường đi và pha tối ưu quỹ đạo liên tục được trình bày ở mục A.1.

4.4.11.1 So sánh với GCS-Bézier

Sự khác biệt giữa mô hình MIOCP+MMS đề xuất và GCS-Bézier dựa trên MISOCP thể hiện rõ ở mức độ biểu diễn động lực học, cấu trúc tối ưu hóa và khả năng giải số.

Ưu điểm. Thứ nhất, quỹ đạo thu được thỏa mãn trực tiếp phương trình động lực học $\dot{x} = f(x, u)$ của hệ, thay vì chỉ được xấp xỉ ở mức động học bằng đa thức Bézier. Điều này đặc biệt quan trọng đối với các hệ rô-bốt có động lực học phi tuyến rõ rệt, chẳng hạn tay máy với động lực Lagrange, hệ underactuated, hoặc rô-bốt di động chịu ràng buộc nonholonomic. Thứ hai, tín hiệu điều khiển $u(\cdot)$ xuất hiện tường minh như một biến quyết định, nhờ đó các ràng buộc giới hạn điều khiển $u \in \mathcal{U}$ và chi phí năng lượng $\|u\|^2$ có thể được đưa vào mô hình một cách tự nhiên. Thứ ba, khi sử dụng hàm cản log-barrier (mục 4.4.6.2), bài toán khuyến khích quỹ đạo nằm trong miền trong của các vùng an toàn, thay vì chỉ kiểm tra điều kiện an toàn tại một tập hữu hạn các điểm lưới.

Hạn chế. Đổi lại, mô hình không còn giữ được cấu trúc lỗi của GCS-Bézier. Trong GCS-Bézier, relaxation lỗi của phần chọn đường đi thường có gap rất nhỏ, và trong nhiều trường hợp gap bằng không [16]. Với MIOCP/MINLP, khối động lực học multiple shooting là phi tuyến và nói chung không lỗi, nên không có bảo đảm tương tự về độ chặt của relaxation. Ngoài ra, bài

toán thuộc lớp MINLP phi lồi, vốn NP-hard trong trường hợp tổng quát, nên chi phí tính toán dự kiến cao hơn đáng kể so với MISOCP. Về mặt công cụ, các bộ giải thương mại như Mosek hoặc Gurobi xử lý rất hiệu quả các bài toán MISOCP, nhưng không trực tiếp giải được MINLP phi lồi tổng quát; do đó cần đến các bộ giải chuyên dụng dựa trên branch-and-bound kết hợp với lời giải NLP tại các nút tìm kiếm.

Tóm lại, OCP+MMS phù hợp hơn trong các bài toán mà tính khả thi động lực học là yêu cầu trung tâm và quỹ đạo Bézier động học không đủ đại diện cho hệ thực. Ngược lại, khi ràng buộc động lực học không phải yếu tố quyết định, GCS-Bézier vẫn là lựa chọn có lợi thế hơn về tốc độ giải và khả năng tận dụng relaxation lồi.

Chương 5

Thực nghiệm và mô phỏng

Trong chương này, chúng tôi tiến hành đánh giá hiệu năng và tính khả thi của hai hướng tiếp cận đã đề xuất: (1) phương pháp dựa trên hình học nguyên bản (GCS) và (2) phương pháp Điều khiển tối ưu kết hợp Multiple Shooting và hàm cản log-barrier (GCS+MMS). Mục tiêu là so sánh một cách toàn diện khả năng sinh quỹ đạo an toàn, chất lượng động lực học và thời gian tính toán của hai phương pháp trên nhiều kịch bản thử nghiệm từ đơn giản đến phức tạp.

5.1 Thiết lập thực nghiệm

Mục đích của các thực nghiệm là kiểm chứng và so sánh hai hướng tiếp cận đã đề xuất ở Chương 4: hoạch định bằng **Graph of Convex Sets (GCS-Bezier)** và hoạch định bằng **OCP+Multiple Shooting kết hợp log-barrier (OCP+MMS)**. Chúng tôi tập trung đánh giá ba khía cạnh: (i) chất lượng phân hoạch không gian tự do và tác động của nó lên cả hai phương pháp, (ii) khả năng mở rộng tổ hợp khi số vùng lỗi lớn, và (iii) vai trò của ràng buộc động lực phi tuyến — nơi GCS-Bezier với mô hình kinematic xấp xỉ bị hạn chế nhưng OCP+MMS xử lý trực tiếp.

5.1.1 Môi trường phần cứng và phần mềm

Toàn bộ thực nghiệm được thực hiện trên máy tính cá nhân với cấu hình: CPU AMD Ryzen 7 4800H @ 2.9 GHz, RAM 16 GB, hệ điều hành Ubuntu 24.04. Stack phần mềm cho từng pha của bài toán được liệt kê trong Bảng 5.1.

Bảng 5.1: Stack phần mềm cho hai phương pháp.

Pha	GCS / GCS-Bezier	OCP+MMS
Phân hoạch không gian	ACD, IRIS, VCC (Drake / cài đặt riêng)	
Pha rời rạc	Mosek (SOCP relaxation)	Gurobi (MILP top- K)
Pha liên tục	Mosek	CasADi + IPOPT (NLP)
Ngôn ngữ	Python (Drake)	Python

5.1.2 Chỉ số đánh giá

Để đảm bảo so sánh công bằng giữa hai phương pháp, chúng tôi sử dụng bốn nhóm chỉ số sau, được áp dụng đồng nhất trên tất cả các kịch bản.

5.1.2.0.1 (A) Chi phí tính toán.

- t_{decomp} : thời gian phân hoạch không gian tự do thành các vùng lõi.
- t_{discrete} : thời gian giải pha rời rạc — relaxation SOCP + làm tròn đối với GCS, MILP top- K đối với OCP+MMS.
- $t_{\text{continuous}}$: thời gian giải pha liên tục — SOCP cuối cùng đối với GCS, dãy NLP multiple shooting đối với OCP+MMS.
- $t_{\text{total}} = t_{\text{decomp}} + t_{\text{discrete}} + t_{\text{continuous}}$.

5.1.2.0.2 (B) Chất lượng quỹ đạo.

- T_{exec} : thời gian thực thi quỹ đạo.
- $L = \int_0^{T_{\text{exec}}} \|\dot{q}(t)\| dt$: độ dài hình học.

- $E_u = \int_0^{T_{\text{exec}}} \|u(t)\|^2 dt$: chi phí điều khiển (chỉ khả dụng cho OCP+MMS, vì GCS-Bezier không có u tường minh).
- $J_{\text{jerk}} = \int_0^{T_{\text{exec}}} \|\ddot{q}(t)\|^2 dt$: tích phân jerk, phản ánh độ trơn.

5.1.2.0.3 (C) An toàn.

- $d_{\min} = \min_t \min_v d(q(t), \partial Q_v)$ với v là vùng đang chứa $q(t)$: khoảng cách tối thiểu giữa quỹ đạo và biên các vùng an toàn, lấy mẫu trên lưới mịn $10 \times$ lưới rời rạc hóa.
- ρ_{vio} : tỉ lệ thời gian quỹ đạo nằm ngoài hợp các vùng an toàn (đánh giá ex-post bằng hàm membership).

5.1.2.0.4 (D) Tính khả thi và tối ưu.

- Cho GCS: optimality gap của relaxation $\delta_{\text{relax}} = (C_{\text{round}} - C_{\text{relax}})/C_{\text{relax}}$.
- Cho OCP+MMS: tỉ lệ NLP hội tụ trên K đường đi ứng viên, và residual của ràng buộc defect cuối cùng $\|s_v^+ - F_v\|_{\infty}$.

5.1.3 Các kịch bản thử nghiệm

Chúng tôi thiết kế ba kịch bản với độ phức tạp tăng dần, mỗi kịch bản nhằm soi sáng một khía cạnh khác nhau của hai hướng tiếp cận:

- **Kịch bản A — Môi trường 2D đơn giản** (mục 5.1.3.1): tác động của thuật toán phân hoạch lên hiệu suất cuối cùng;
- **Kịch bản B — Mê cung** (mục 5.1.3.2): khả năng mở rộng tổ hợp khi số vùng lỗi lên đến hàng trăm;
- **Kịch bản C — Xe nonholonomic kiểu Dubins** (mục 5.1.3.3): vai trò của ràng buộc động lực phi tuyến và giới hạn của xấp xỉ Bezier kinematic.

5.1.3.1 Kịch bản A — Môi trường 2D đơn giản

Mục tiêu. Định lượng tác động của các thuật toán phân hoạch lỗi (ACD, VCC) so với phân hoạch thủ công¹ (Manual) lên cả bộ lập kế hoạch GCS-Bezier và OCP+MMS.

Mô tả kịch bản. Một không gian làm việc 2D với các vật cản đa giác lỗi được bố trí cố định, cấu hình bắt đầu q_0 và cấu hình kết thúc q_T được xác định trước. Không gian tự do $\mathcal{X}_{\text{free}}$ được phân rã thành tập các vùng lỗi an toàn $\{Q_i\}_{i \in I}$ thông qua ba phương pháp:

1. **Phân hoạch thủ công:** tạo số vùng lỗi tối thiểu bằng can thiệp của con người, làm chuẩn đối sánh.
2. **Phân hoạch lỗi xấp xỉ (ACD):** dung sai $\tau = 0$ (các vùng lỗi hoàn toàn).
3. **Lớp phủ clique khả kiến (VCC):** mức độ phủ 96%.

Trên mỗi phân hoạch, chúng tôi giải bài toán tối ưu hóa thời gian thực hiện kết hợp với điều chuẩn đạo hàm bậc hai theo cả hai phương pháp đề xuất.

5.1.3.2 Kịch bản B — Mê cung

Mục tiêu. Đánh giá khả năng mở rộng tổ hợp của hai phương pháp khi số vùng lỗi tăng cao, đồng thời kiểm chứng tính ổn định thống kê qua nhiều lần sinh môi trường ngẫu nhiên.

Mô tả kịch bản. Mê cung kích thước $25 \times 25 = 625$ ô được sinh bằng thuật toán DFS ngẫu nhiên, sau đó loại bỏ ngẫu nhiên 100 bức tường để tạo ra các lộ trình đa dạng. Mỗi ô được mô hình hóa thành một tập an toàn lỗi Q_i với các cạnh hai chiều giữa các ô kề. Thực nghiệm được lặp lại trên 10 mê cung khác nhau để đảm bảo tính khách quan thống kê.

¹Tức người dùng tự phân hoạch không gian tự do; ở đây chúng tôi coi kết quả thủ công làm chuẩn đối sánh về số vùng tối thiểu.

5.1.3.3 Kịch bản C — Xe nonholonomic kiểu Dubins

Mục tiêu. Minh chứng vai trò của ràng buộc động lực phi tuyến trong OCP+MMS và làm rõ giới hạn của xấp xỉ Bezier kinematic dùng trong GCS-Bezier — đây là kịch bản trọng tâm để khẳng định đóng góp của hướng tiếp cận thứ hai.

Mô hình động lực. Xét một xe Dubins di chuyển trong mặt phẳng, có trạng thái $x = (p_x, p_y, \theta)^\top \in \mathbb{R}^2 \times S^1$ và điều khiển $u = (v, \omega)^\top$, với phương trình:

$$\dot{p}_x = v \cos \theta, \quad \dot{p}_y = v \sin \theta, \quad \dot{\theta} = \omega. \quad (5.1)$$

Vận tốc thẳng bị giới hạn $v \in [v_{\min}, v_{\max}]$ với $v_{\min} > 0$ (xe không lùi), và tốc độ góc lái $|\omega| \leq \omega_{\max}$, mô hình hóa giới hạn vật lý của bánh trước.

Mô tả kịch bản. Không gian làm việc 2D kích thước 20×20 với 6–10 vật cản hình chữ nhật bố trí ngẫu nhiên trên 10 seed. Cấu hình khởi đầu và đích được chọn sao cho yêu cầu xe phải đổi hướng tổng cộng ít nhất 90° , đảm bảo ràng buộc $\omega_{\max}$ thực sự kích hoạt. Không gian tự do được phân hoạch bằng VCC trên không gian vị trí (p_x, p_y) ; chiều θ không tham gia phân hoạch mà chỉ xuất hiện trong động lực nội bộ của OCP+MMS.

Đánh giá đặc thù cho GCS-Bezier. Vì GCS-Bezier tham số hóa quỹ đạo bằng đa thức Bezier trên (p_x, p_y) mà không mô hình hóa θ , chúng tôi suy ngược tốc độ góc từ nghiệm thu được bằng $\omega(t) = \dot{\theta}(t)$ với $\theta(t) = \arctan 2(\dot{p}_y(t), \dot{p}_x(t))$, sau đó đo tỉ lệ thời gian vi phạm $|\omega(t)| > \omega_{\max}$. Đây là chỉ số định lượng cho thấy quỹ đạo Bezier có thể đẹp về hình học nhưng không khả thi về động lực, trong khi OCP+MMS đảm bảo $|\omega(t)| \leq \omega_{\max}$ trực tiếp như một ràng buộc của bài toán.

5.1.4 Tham số bài toán

Bảng 5.2 tóm tắt các tham số chính của hai phương pháp trên ba kịch bản. Trên cùng một kịch bản, các tham số chung của hàm mục tiêu được giữ

giống nhau giữa hai phương pháp để đảm bảo so sánh công bằng.

Bảng 5.2: Tham số chính của các thực nghiệm.

Tham số	Kịch bản A	Kịch bản B	Kịch bản C
Số chiều cấu hình	2	2	3
Bậc Bezier (GCS) / liên tục C^k	6 / C^2	6 / C^2	6 / C^2
M_v — số nút lưới mỗi vùng (OCP+MMS)	10	5	15
K — số đường đi ứng viên (OCP+MMS)	5	20	10
μ_v ban đầu / cuối (log-barrier)	10^{-2} / 10^{-5}	10^{-2} / 10^{-5}	10^{-2} / 10^{-5}
Trọng số w_t / w_E	1 / 0.1	1 / 0.1	1 / 0.1
Vận tốc tối đa $v_{\max}$	1	1	1
Tốc độ góc lái $\omega_{\max}$	—	—	$\pi/2$ rad/s
Số seed	1 (cố định)	10	10

5.1.5 Phương pháp đối sánh

Mỗi kịch bản được giải bằng các baseline sau:

- **Manual + GCS-Bezier** (chỉ Kịch bản A): chuẩn đối sánh về số vùng tối thiểu.
- **Linear-GCS**: chỉ tối thiểu hóa độ dài hình học, dùng làm cận dưới về thời gian giải.
- **Bezier-GCS**: phương pháp đề xuất hướng 1 của báo cáo này (mục 4.3).
- **OCP+MMS**: phương pháp đề xuất hướng 2 của báo cáo này (mục 4.4).

Trong Kịch bản C, do GCS-Bezier không mô hình hóa được ràng buộc $\omega_{\max}$ tường minh, baseline GCS-Bezier được giải với mô hình kinematic thuần và đánh giá vi phạm động lực ex-post như đã mô tả ở mục 5.1.3.3.

5.2 Kết quả thực nghiệm

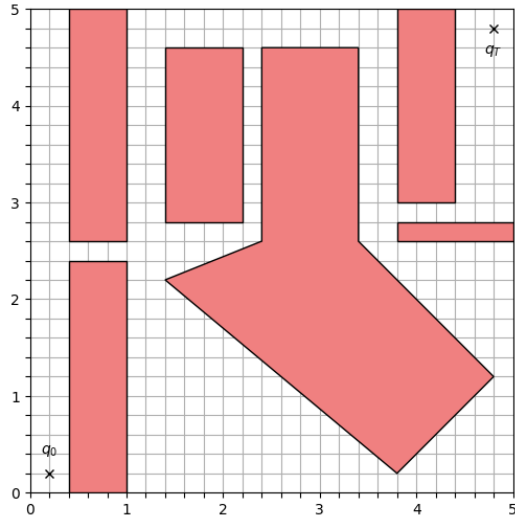
5.3 Kết quả và Phân tích

Trong chương này, chúng tôi trình bày các kết quả định lượng thu được từ các thực nghiệm đã thiết lập ở phần trước. Mục tiêu là phân tích sự đánh đổi giữa độ phức tạp của việc phân hoạch không gian và tính tối ưu của quỹ đạo đầu ra. Toàn bộ các thực nghiệm được thực hiện trên máy tính cá nhân với cấu hình: CPU AMD Ryzen 7 4800H 2.9 GHz, RAM 16GB, hệ điều hành Ubuntu 24.04.

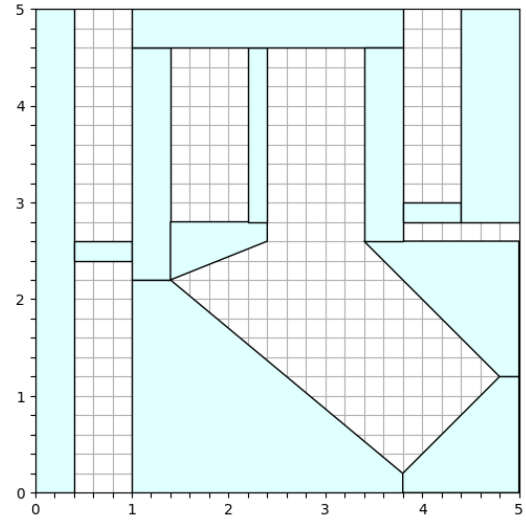
5.3.1 Phân tích Môi trường 2D đơn giản: Tác động của Phân hoạch lỗi

Kết quả thực nghiệm đối với các giải thuật phân hoạch tự động cho thấy sự tương phản rõ rệt về chiến lược biểu diễn không gian cấu hình cũng như chi phí tính toán giữa hai phương pháp tiếp cận chủ đạo là ACD và VCC. Đối với giải thuật Phân hoạch Lỗi Xấp xỉ (ACD), kết quả đạt được thể hiện như hình 5.1c ưu thế vượt trội trong môi trường hình học 2D khi chỉ tạo ra 15 vùng lỗi, một con số tiệm cận sát với mức tối ưu của phương pháp phân hoạch thủ công là 12 vùng. Hiệu suất này được củng cố bởi thời gian xử lý nhanh, chỉ xấp xỉ 0,01 giây, hoàn toàn phù hợp với độ phức tạp lý thuyết $O(nr^2)$ [8] của thuật toán khi xử lý các đa giác có số lượng đỉnh lồi không quá lớn. Việc ACD có khả năng kiểm soát mức độ chi tiết thông qua tham số lồi τ cho phép hệ thống duy trì được sự cân bằng giữa độ chính xác hình học và số lượng vùng lỗi tối giản, từ đó giảm thiểu số lượng biến nhị phân trong bài toán quy hoạch nguyên-hỗn hợp (MICP) của GCS framework.

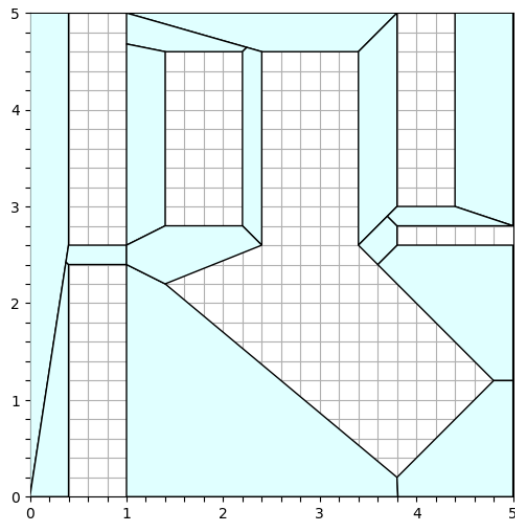
Ngược lại, giải thuật Visibility Clique Cover (VCC) như được thể hiện thông qua hình 5.1d lại mang đến một cấu trúc phân hoạch dày đặc và phức tạp hơn với số lượng vùng lỗi dao động từ 33 đến 35 vùng. Sự thiếu ổn định về số lượng vùng phân hoạch giữa các lần thực thi xuất phát trực



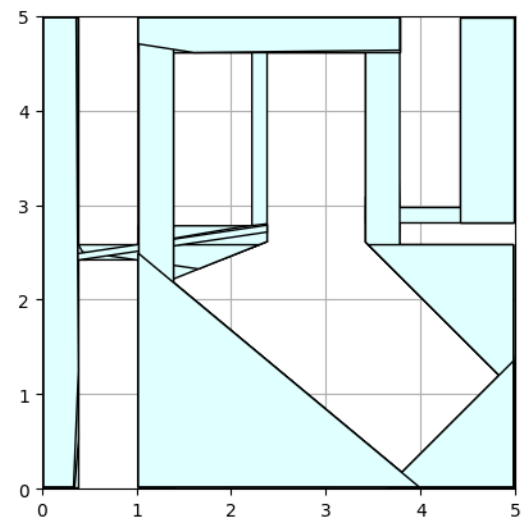
(a) Không gian 2D với vật cản (vùng màu đỏ) ban đầu



(b) Kết quả Phân hoạch thủ công



(c) Kết quả Phân hoạch ACD



(d) Kết quả Phân hoạch VCC

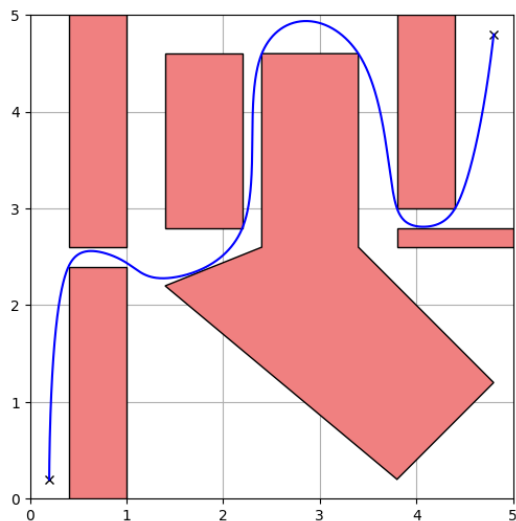
Hình 5.1: So sánh chất lượng các vùng lồi được phân hoạch bởi các giải thuật khác nhau. Số lượng tập lồi tăng dần từ (b) đến (d), cho thấy sự cải thiện về độ chính xác nhưng tăng độ phức tạp tính toán.

tiếp từ tính chất ngẫu nhiên của bước lấy mẫu n điểm trong không gian cấu hình tự do C_{free} để xây dựng đồ thị khả kiến đã được nêu ở phần 4.2.3. Mặc dù sự chênh lệch khoảng 1-3 vùng lỗi tại các khu vực hành lang hẹp là mức sai số có thể chấp nhận được, nhưng tổng số lượng vùng lỗi lớn đã tạo ra một gánh nặng tính toán đáng kể cho bộ giải Mosek do số lượng biến nhị phân y_e tăng lên theo tỉ lệ xấp xỉ $2|I|$. Thời gian hội tụ của VCC trong môi trường 2D thực tế không mang lại hiệu quả cao cho các ứng dụng yêu cầu phản hồi tức thời so với các kỹ thuật phân hoạch đa giác trực tiếp như ACD. Tuy nhiên, cần phải nhìn nhận rằng VCC được thiết kế như một phương pháp lai ghép mạnh mẽ để xử lý các không gian cấu hình cao chiều, nơi các vật cản không được xác định tường minh bằng các đa giác mà thông qua các bộ kiểm tra va chạm chung. Trong các hệ thống có số chiều lớn như cánh tay robot 7 bậc tự do (7-DoF), việc lấy mẫu và tìm lớp phủ clique giúp VCC tự động hóa quá trình đặt các điểm mào chiến lược cho thuật toán IRIS, điều mà các phương pháp phân hoạch hình học 2D đơn giản không thể thực hiện được. Do đó, việc áp dụng VCC vào môi trường 2D trong thực nghiệm này chủ yếu đóng vai trò kiểm chứng khả năng bao phủ toàn cục và tính an toàn của quỹ đạo.

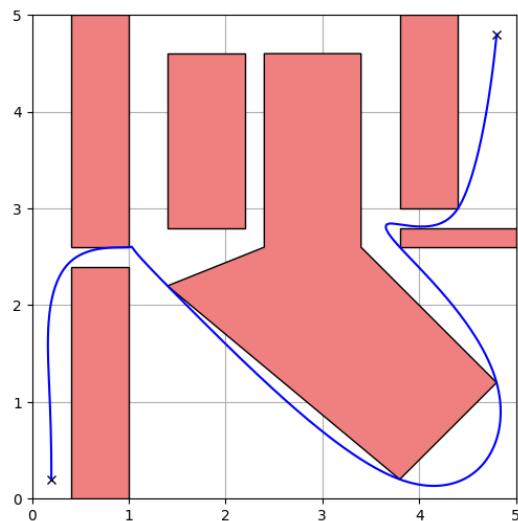
Bảng 5.3: So sánh hiệu năng giữa các phương pháp lập kế hoạch quỹ đạo

Phương pháp	Thời gian phân hoạch (s)	Số vùng	Thời gian giải (s)	Time cost (s)	Path cost (rounded)	Tổng thời gian (s)
VCC	5.7807	33	269.9363	13.2217	24.4365	275.7170
ACD	0.0109	15	4.3105	12.7875	24.6286	4.3214
Phân hoạch thủ công	—	12	3.9005	13.3565	27.8805	3.9005

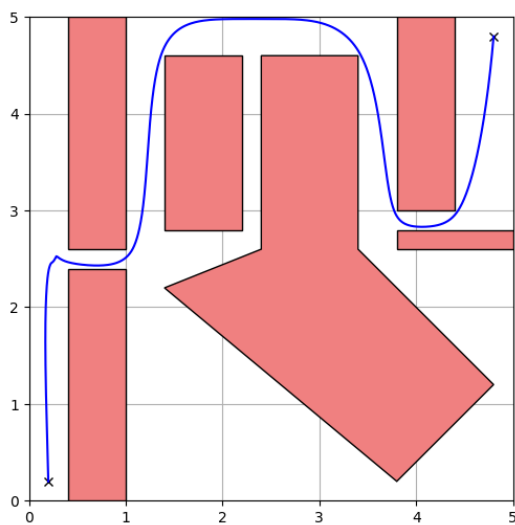
Sau khi hoàn tất giai đoạn phân hoạch không gian tự do thành các tập hợp vùng lỗi, chúng tôi tiến hành thực hiện bài toán hoạch định quỹ đạo dựa trên khung làm việc Đồ thị các Tập lỗi (Graphs of Convex Sets - GCS). Mục tiêu của giai đoạn này là tìm kiếm một lộ trình tối ưu không chỉ về mặt hình học mà còn phải đáp ứng các ràng buộc về động lực học và độ trơn của quỹ đạo. Để đạt được sự cân bằng giữa thời gian thực thi vật lý ($T_{duration}$) và nỗ lực điều khiển (Control effort), hàm mục tiêu tổng hợp



(a) Quỹ đạo chuyển động với phân hoạch thủ công



(b) Quỹ đạo chuyển động với phân hoạch ACD



(c) Quỹ đạo chuyển động với phân hoạch VCC

Hình 5.2: So sánh chất lượng các quỹ đạo được hoạch định cho tập các vùng lỗi được tạo bởi các giải thuật ở trên.

(Rounded Cost) được thiết lập như sau:

$$RoundedCost \approx (1 \times T_{duration}) + \sum \left(0.1 \times \int_0^{T_e} \|a(t)\|^2 dt \right)$$

Trong đó, trọng số cho thời gian thực thi được đặt là $w_t = 1$ và hệ số điều chuẩn đạo hàm bậc hai (gia tốc) là $\epsilon = 0.1$ nhằm giảm thiểu hiện tượng rung động (jerk) khi robot di chuyển qua các ranh giới vùng lỗi. Các kết quả định lượng về hiệu suất tính toán và chất lượng quỹ đạo được phân tích chi tiết dựa trên dữ liệu thu được từ bộ giải Mosek.

5.3.1.1 Hiệu suất tính toán và Khả năng đáp ứng thời gian thực

Kết quả thực nghiệm tại bảng 5.3 cho thấy một sự tương quan phi tuyến tính chặt chẽ giữa số lượng vùng lỗi được phân hoạch và thời gian hội tụ của bộ giải. Phương pháp **VCC** cho thấy hiệu năng kém nhất trong môi trường thực nghiệm 2D. Mặc dù số lượng vùng lỗi được tạo ra (33 vùng) chỉ cao hơn gấp đôi so với ACD, nhưng thời gian giải bài toán tối ưu hóa lại rất lớn lên tới xấp xỉ 269,94 giây. Hiện tượng này xuất phát từ việc gia tăng số lượng vùng lỗi dẫn đến sự bùng nổ của các biến quyết định nhị phân và các ràng buộc nón phối cảnh liên kết, khiến không gian tìm kiếm tổ hợp vượt quá khả năng xử lý hiệu quả của bộ giải trong thời gian thực. Ngược lại, phương pháp **ACD** thể hiện tính thực tiễn vượt trội khi tổng thời gian thực thi (bao gồm cả phân hoạch và giải quỹ đạo) chỉ mất 4,32 giây, nhanh gấp khoảng 64 lần so với VCC. Kết quả này khẳng định rằng đối với các ứng dụng di động trong môi trường 2D, một cấu trúc phân hoạch tối giản (15 vùng) là yếu tố tiên quyết để duy trì khả năng phản hồi nhanh của hệ thống. Trong khi đó, phương pháp **Phân hoạch thủ công** mặc dù đạt thời gian giải nhanh nhất (3,90 giây) do cấu trúc đồ thị đơn giản (12 vùng), nhưng không được xem là giải pháp khả thi cho các hệ thống tự hành do đòi hỏi sự can thiệp trực tiếp từ con người, vốn là một rào cản lớn đối với tính tự chủ của robot.

5.3.1.2 Chất lượng Quỹ đạo và Sự đánh đổi giữa các Mục tiêu Tối ưu

Chất lượng của quỹ đạo được đánh giá thông qua sự đánh đổi giữa thời gian di chuyển thực tế và giá trị tối ưu của hàm mục tiêu (Path cost).

Phương pháp VCC: Đạt giá trị Path cost thấp nhất (24,44), đại diện cho nghiệm tối ưu nhất về mặt toán học. Việc chia nhỏ không gian thành nhiều vùng lỗi diện tích nhỏ cho phép bộ giải có nhiều "không gian tự do" hơn để tinh chỉnh đường cong Bézier, từ đó tối ưu hóa nỗ lực điều khiển và giảm thiểu tối đa các pha phạt gia tốc. Tuy nhiên, thời gian thực thi vật lý (13,22 giây) lại chậm hơn ACD, cho thấy hệ thống chấp nhận di chuyển với vận tốc thấp hơn để đổi lấy độ trơn tuyệt đối.

Phương pháp ACD: Mang lại kết quả thực dụng nhất khi robot hoàn thành nhiệm vụ với thời gian ngắn nhất (12,79 giây). Mặc dù Path cost (24,63) cao hơn nhẹ so với VCC, nhưng sự chênh lệch này không đáng kể trong vận hành thực tế. Điều này chứng tỏ ACD đã tạo ra một quỹ đạo "thực dụng": chạy nhanh, đủ mượt mà và duy trì được tính toán hiệu quả.

Phân hoạch thủ công: Thể hiện hiệu năng kém nhất về mặt chất lượng quỹ đạo với Path cost cao nhất (27,88) và thời gian di chuyển lâu nhất (13,36 giây). Nguyên nhân chủ yếu nằm ở việc phân chia vùng dựa trên trực giác con người thường tạo ra các vùng lỗi thô, các vùng có thể to nhưng hình dạng không tối ưu cho robot lách qua, hoặc các điểm nối giữa các vùng bị đặt ở vị trí xấu, khiến robot phải đi đường vòng hoặc phanh gấp tại các giao điểm.

Tóm lại, thông qua các chỉ số đo lường thực nghiệm, chúng tôi xác định **ACD** là giải pháp tối ưu cho các bài toán hoạch định quỹ đạo 2D, đảm bảo được sự cân bằng giữa chi phí tính toán và hiệu suất vận hành thực tế. Trong khi đó, phương pháp VCC dù cho thấy tiềm năng về tính tối ưu toán học nhưng lại gặp rào cản lớn về thời gian giải, khiến nó phù hợp hơn cho các ứng dụng ngoại tuyến hoặc trong các không gian cấu hình cao

chiều phức tạp.

5.3.2 Phân tích Thực nghiệm trên Môi trường Mê cung Phức tạp

Trong phần thực nghiệm này, chúng tôi mở rộng quy mô kiểm chứng sang môi trường mê cung có độ phức tạp tổ hợp lớn nhằm đánh giá khả năng của khung làm việc GCS trong việc kết hợp hài hòa giữa tối ưu hóa rời rạc và liên tục. Để đảm bảo tính khách quan và độ tin cậy về mặt thống kê, các thực nghiệm được thực hiện trên 10 cấu trúc mê cung khác nhau, được sinh ngẫu nhiên bằng thuật toán DFS với kích thước $25 \times 25 = 625$ ô. Trong mỗi mê cung, chúng tôi loại bỏ ngẫu nhiên 100 bức tường để tạo ra các lộ trình đa dạng, buộc bộ lập kế hoạch phải tìm kiếm nghiệm tối ưu trên một đồ thị có cấu trúc liên thông phức tạp.

Các tham số tối ưu hóa được thiết lập đồng nhất cho tất cả các mẫu thử nghiệm với đường cong Bézier bậc $d = 6$, đảm bảo độ trơn liên tục cấp C^2 . Hàm mục tiêu tập trung tối thiểu hóa thời gian thực thi kết hợp với điều chuẩn đạo hàm bậc hai ($\epsilon = 10^{-1}$) nhằm phạt các gia tốc quá lớn. Bộ giải *Mosek* được sử dụng để xử lý bài toán SOCP thu được từ kỹ thuật nói lỏng lỗi.

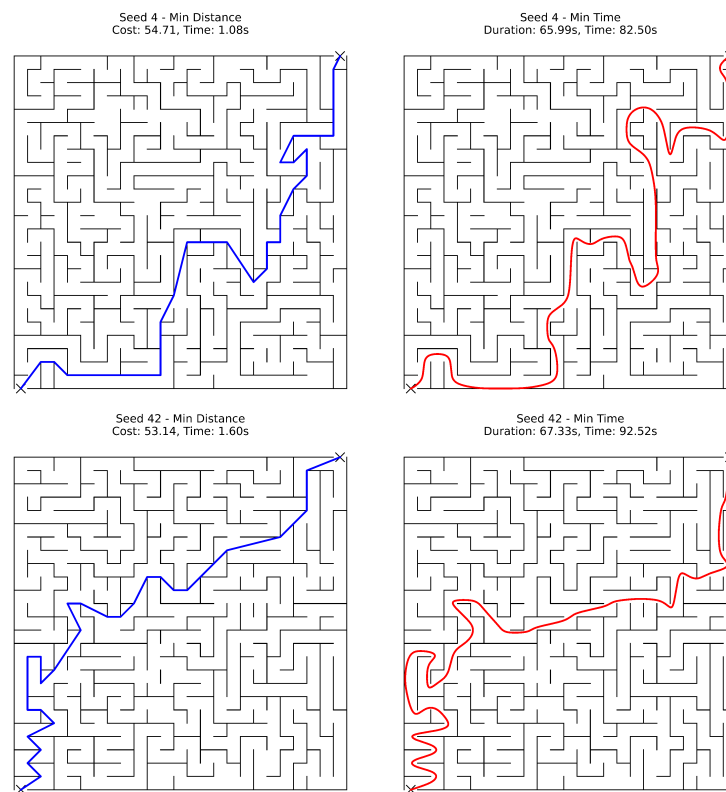
5.3.2.1 Phân tích Hiệu suất Tính toán và Chi phí Tối ưu

Dựa trên dữ liệu thu thập từ 10 mẫu thử nghiệm tại bảng 5.4, chúng tôi tiến hành so sánh đối chiếu giữa hai phương pháp:

- **Linear GCS:** tối thiểu hóa độ dài hình học.
- **Bezier GCS:** tối thiểu hóa thời gian với ràng buộc động lực học.

Kết quả thống kê cho thấy sự khác biệt rõ rệt về hiệu suất:

Về thời gian tính toán: Linear GCS cho thấy khả năng phản hồi cực nhanh với thời gian giải trung bình chỉ 1,28 giây. Trong khi đó, Bezier GCS



Hình 5.3: Quỹ đạo chuyển động cho 1 số mê cung

Bảng 5.4: Thống kê kết quả thực nghiệm trên 10 mẫu mê cung ngẫu nhiên

Chỉ số (Metric)	Linear GCS	Bezier GCS	Đơn vị
Thời gian giải trung bình (Mean)	1,28	85,22	s
Thời gian giải trung vị (Median)	1,16	85,10	s
Chi phí trung bình (Mean Cost)	71,88	88,03	unit
Sai số chi phí (Cost Ratio TB)	0,82	1,00	x

yêu cầu trung bình 85,22 giây để hội tụ. Sự gia tăng này là do Bezier GCS phải giải quyết các ràng buộc vi phân bậc cao (C^2) và tối ưu hóa đồng thời biến thời gian h , khiến bài toán SOCP có quy mô lớn hơn đáng kể.

Về chi phí tối ưu: Chi phí trung bình của Bezier GCS cao hơn khoảng 22% so với Linear GCS. Tuy nhiên, chi phí của Bezier GCS bao gồm cả thành phần điều chuẩn gia tốc (*Regularization Cost*), phản ánh sự đánh đổi cần thiết để đạt được quỹ đạo mượt mà và khả thi về mặt vật lý.

5.3.2.2 Phân tích Đặc điểm Quỹ đạo và Tính Tối ưu Toàn cục

Quan sát các quỹ đạo tại Hình 5.3 (tương ứng với các *Seeds* khác nhau), chúng tôi rút ra các nhận định quan trọng về hành vi của bộ lập kế hoạch:

1. **Sự khác biệt về lộ trình (Homotopy class):** Trong nhiều trường hợp (ví dụ *Seed 4* và *Seed 42*), quỹ đạo Bezier GCS (màu đỏ) lựa chọn một lộ trình hoàn toàn khác so với quỹ đạo Linear GCS (màu xanh). Trong khi quỹ đạo hình học ưu tiên đường ngắn nhất với nhiều khúc cua gắt, quỹ đạo Bezier chấp nhận đi quãng đường dài hơn nhưng ít góc rẽ gắt hơn nhằm duy trì vận tốc cao và giảm chi phí gia tốc.
2. **Hành vi tại các góc cua:** Quỹ đạo Bezier thể hiện xu hướng “ôm cua” rộng tại các giao lộ để đảm bảo tính liên tục của gia tốc (C^2). Các thành phần vận tốc được duy trì trong giới hạn $[-1, 1]$ nhờ vào cấu trúc nón phối cảnh trong mô hình GCS.
3. **Tính tối ưu toàn cục:** Đối với cả 10 mẫu mê cung, kỹ thuật nói lỏng lỗi đều trả về các xác suất nhị phân ϕ_e hội tụ về $\{0, 1\}$, giúp nghiệm thu được tự động đạt tối ưu toàn cục mà không cần qua bước làm tròn (*Rounding*). Điều này chứng minh GCS vượt trội hơn các phương pháp MICP truyền thống khi giảm số lượng biến nhị phân từ mức $|I|^2$ xuống chỉ còn $2|I|$, cho phép giải quyết các mê cung lớn mà các bộ giải cũ thường thất bại.

Kết luận, việc thực hiện trên 10 mê cung khác nhau đã khẳng định tính ổn định của framework GCS. Mặc dù chi phí tính toán cho quỹ đạo *kinodynamic* cao hơn, GCS cung cấp một giải pháp nhất quán để tìm ra lộ trình trơn tru, an toàn tuyệt đối và tối ưu toàn cục trong các môi trường có độ phức tạp tổ hợp cực cao.

5.4 Mô phỏng

Chương 6

Kết luận

6.1 Kết luận

Luận văn này đã tập trung nghiên cứu và triển khai khung lý thuyết Đồ thị các Tập lồi (Graphs of Convex Sets - GCS) để giải quyết bài toán hoạch định chuyển động cho hệ thống robot trong các môi trường có cấu trúc hình học phức tạp. Về mặt lý thuyết, chúng tôi đã xây dựng một quy trình hoàn chỉnh từ giai đoạn phân hoạch không gian tự do thành hợp của các tập lồi, đến việc tham số hóa quỹ đạo bằng đường cong Bézier nhằm đảm bảo tính liên tục của các trạng thái động học cấp cao như vận tốc và gia tốc. Điểm cốt lõi của phương pháp nằm ở việc vận dụng kỹ thuật nói lỏng phối cảnh để chuyển đổi bài toán tối ưu hóa lồi nguyên hỗn hợp (MICP) vốn có độ phức tạp tính toán rất lớn thành bài toán Lập trình nón bậc hai (SOCP) có thể giải quyết hiệu quả trong thời gian đa thức.

Thông qua các kết quả thực nghiệm định lượng trên môi trường 2D và hệ thống mô phỏng đa tầng, luận văn đã chứng minh được tính khả thi và hiệu quả vận hành của khung làm việc GCS. Trong các kịch bản môi trường 2D, việc phân tích đối chiếu giữa các kỹ thuật phân hoạch không gian cho thấy thuật toán Phân hoạch lồi xấp xỉ (ACD) mang lại sự cân bằng tối ưu giữa số lượng vùng lồi và thời gian hội tụ của bộ giải so với phương pháp Lóp

phủ Clique Khả kiến (VCC). Đặc biệt, kết quả tính toán trên môi trường mê cung có độ phức tạp tổ hợp cao đã làm nổi bật sự đánh đổi giữa hai mô hình: Linear GCS cho phép phản hồi cực nhanh về mặt hình học, trong khi Bezier GCS mặc dù yêu cầu chi phí tính toán cao hơn nhưng lại tạo ra các quỹ đạo mượt mà, đảm bảo các ràng buộc vi phân liên tục C^2 và khả thi về mặt vật lý cho robot.

Một đóng góp quan trọng của nghiên cứu này là việc đề xuất và triển khai thành công mô hình bài toán Điều khiển Tối ưu (OCP) kết hợp Multiple Shooting và hàm cản log-barrier trên nền tảng của GCS. Thay vì chỉ tạo ra quỹ đạo hình học như GCS nguyên bản, hướng tiếp cận đề xuất đã giải quyết triệt để vấn đề vi phạm biên (hiện tượng đi xuyên vùng) thông qua áp dụng hàm phạt logarit. Đồng thời, kỹ thuật Multiple Shooting cho phép chia nhỏ và tối ưu hóa động lực học hệ thống một cách trơn tru, đảm bảo quỹ đạo thu được vừa tối ưu về mặt hình học, vừa khả thi tuyệt đối về mặt vật lý. Mặc dù trong phạm vi của báo cáo này, chúng tôi tập trung chủ yếu vào hệ thống robot trong môi trường tĩnh, nhưng các dữ liệu thực nghiệm định lượng thu được đã minh chứng tính hiệu quả vượt trội của phương pháp đề xuất.

6.2 Hướng nghiên cứu trong tương lai

Dựa trên những kết quả thực nghiệm đạt được, nghiên cứu đề xuất các hướng phát triển tiếp theo nhằm tối ưu hóa hiệu năng và mở rộng khả năng ứng dụng của đề án như sau:

- **Tối ưu hóa hiệu suất tính toán thời gian thực (Real-time MPC):** Nghiên cứu sẽ tập trung vào việc áp dụng phương pháp của đề án trong mô hình Điều khiển Dự báo Mô hình (Model Predictive Control - MPC). Việc khai thác khả năng warm-start của Multiple Shooting sẽ giúp hệ thống đáp ứng tốt hơn các yêu cầu hoạch định

quỹ đạo trực tuyến (online planning) cho robot.

- **Mở rộng cho môi trường động (Dynamic Environments):** Khung làm việc hiện tại chủ yếu giải quyết bài toán tĩnh. Hướng đi tiếp theo là mở rộng đồ thị tập lời theo cả trục thời gian (Spatiotemporal GCS) để nhận diện và lẩn tránh các vật cản di động.
- **Nâng cao thuật toán phân hoạch không gian an toàn:** Để xử lý các môi trường có vật thể phi tuyến tính phức tạp trong không gian đa chiều, nghiên cứu dự kiến sẽ tích hợp các kỹ thuật phân hoạch trực tiếp trong không gian cấu hình (C -space) như C-IRIS. Điều này cho phép mở rộng áp dụng hệ thống cho các cánh tay robot công nghiệp có số bậc tự do (DOF) lớn.
- **Cải tiến hàm phạt Log-barrier:** Đánh giá thêm các dạng hàm phạt hoặc các phương pháp interior-point nâng cao nhằm giảm số lần lặp (iterations) cần thiết khi điều chỉnh trọng số μ , qua đó tối ưu thêm chi phí tính toán của bài toán NLP.

Để cụ thể hóa các hướng nghiên cứu trên, bảng 6.1 trình bày kế hoạch chi tiết trong 15 tuần tới nhằm hoàn thiện đồ án tốt nghiệp, tập trung vào việc kiểm thử và đánh giá các cải tiến đề xuất.

Bảng 6.1: Kế hoạch chi tiết Đồ án Tốt nghiệp (15 tuần)

Giai đoạn	Nhiệm vụ chi tiết	W1– 3	W4– 6	W7– 9	W10– 12	W13– 15
Tối ưu thuật toán	(1) Kiểm thử và tích hợp kỹ thuật Multiple Shooting vào GCS Framework	X				
	(2) Triển khai phân hoạch C -space bằng C-IRIS/IRIS-NP để xử lý vật cản phi tuyến	X	X			
Hiện thực mô hình	(3) Thiết kế hệ thống điều khiển robot tích hợp khung BezierGCS		X	X		
	(4) Xây dựng thuật toán nhận diện và ánh xạ mê cung thực tế sang đồ thị các tập lỗi			X	X	
Thực nghiệm	(5) Thử nghiệm robot giải mê cung, đo lường thời gian giải và sai số quỹ đạo				X	X
	(6) Đánh giá đối chứng hiệu năng với các thuật toán lấy mẫu RRT* và PRM					X
Hoàn thiện	(7) Phân tích dữ liệu, đánh giá tính hội tụ của nới lỏng phối cảnh và viết luận văn					X

Tài liệu tham khảo

- [1] R. Deits and R. Tedrake, “Motion planning around obstacles with convex optimization,” *arXiv preprint*, vol. arXiv:2205.04422, 2022. [Online]. Available: <https://arxiv.org/abs/2205.04422>.
- [2] Boston Dynamics, *Leaps, bounds, and backflips*, Blog post, 2021. [Online]. Available: <https://bostondynamics.com/blog/leaps-bounds-and-backflips/>.
- [3] Amazon Robotics, *Amazon.com announces first quarter results*, Press release, 2023. [Online]. Available: <https://ir.aboutamazon.com/news-release/news-release-details/2023/Amazon.com-Announces-First-Quarter-Results/default.aspx>.
- [4] M. Diehl, H. G. Bock, H. Diedam, and P.-B. Wieber, “Fast direct multiple shooting algorithms for optimal robot control,” pp. 65–93, 2006. DOI: [10.1007/978-3-540-36119-0_4](https://doi.org/10.1007/978-3-540-36119-0_4).
- [5] L. E. Kavraki, P. Svestka, J.-C. Latombe, and M. H. Overmars, “Probabilistic roadmaps for path planning in high-dimensional configuration spaces,” *IEEE Transactions on Robotics and Automation*, vol. 12, no. 4, pp. 566–580, 1996. DOI: [10.1109/70.508439](https://doi.org/10.1109/70.508439).
- [6] S. Boyd and L. Vandenberghe, *Convex Optimization*. Cambridge University Press, 2004. [Online]. Available: <https://web.stanford.edu/~boyd/cvxbook/>.

- [7] R. K. Ahuja, T. L. Magnanti, and J. B. Orlin, *Network Flows: Theory, Algorithms, and Applications*. Prentice Hall, 1993. [Online]. Available: https://web.mit.edu/dimitrib/www/NETWORK_FLOW.pdf.
- [8] J. M. Lien and N. M. Amato, “Approximate convex decomposition of polygons,” *Computational Geometry: Theory and Applications*, 2006. [Online]. Available: https://cs.gmu.edu/~jmlien/masc/uploads/Main/cd2d_CGTA.pdf.
- [9] R. Deits and R. Tedrake, “Approximating robot configuration spaces with few convex sets using clique covers of visibility graphs,” *arXiv preprint*, vol. arXiv:2310.02875, 2023. [Online]. Available: <https://arxiv.org/pdf/2310.02875>.
- [10] S. M. LaValle, *Planning Algorithms*. Cambridge, U.K.: Cambridge University Press, 2006, ISBN: 9780521862059. [Online]. Available: <http://planning.cs.uiuc.edu/>.
- [11] A. Tandon, J. Stührenberg, K. Dragos, and K. Smarsly, “Autonomous navigation of quadruped robots for monitoring and inspection of civil infrastructure,” *Automation in Construction*, vol. 160, p. 105313, 2024. DOI: [10.1016/j.autcon.2024.105313](https://doi.org/10.1016/j.autcon.2024.105313).
- [12] S. M. LaValle, “Rapidly-exploring random trees: A new tool for path planning,” Iowa State University, Tech. Rep. TR 98-11, 1998.
- [13] J. D. Gammell, S. S. Srinivasa, and T. D. Barfoot, “Informed RRT*: Optimal sampling-based path planning focused via direct sampling of an admissible ellipsoidal heuristic,” in *2014 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2014, pp. 2997–3004. DOI: [10.1109/IROS.2014.6942976](https://doi.org/10.1109/IROS.2014.6942976). [Online]. Available: <https://doi.org/10.1109/IROS.2014.6942976>.
- [14] J. T. Betts, *Practical Methods for Optimal Control and Estimation Using Nonlinear Programming*, 2nd. Philadelphia, PA: Society for

- Industrial and Applied Mathematics (SIAM), 2010. DOI: [10.1137/1.9780898717822](https://doi.org/10.1137/1.9780898717822).
- [15] H. G. Bock and K. J. Plitt, “A multiple shooting algorithm for direct solution of optimal control problems,” in *Proceedings of the 9th IFAC World Congress*, Budapest, Hungary, 1984, pp. 1603–1608.
 - [16] T. Marcucci, J. Umenberger, P. A. Parrilo, and R. Tedrake, “Shortest paths in graphs of convex sets,” *arXiv preprint*, vol. arXiv:2101.11565, 2023. [Online]. Available: <https://arxiv.org/pdf/2101.11565>.
 - [17] M. Zucker et al., “CHOMP: Covariant Hamiltonian optimization for motion planning,” *The International Journal of Robotics Research*, vol. 32, no. 9–10, pp. 1164–1193, 2013. DOI: [10.1177/0278364913488805](https://doi.org/10.1177/0278364913488805).
 - [18] J. Schulman et al., “Motion planning with sequential convex optimization and convex collision checking,” *The International Journal of Robotics Research*, vol. 33, no. 9, pp. 1251–1270, 2014. DOI: [10.1177/0278364914528132](https://doi.org/10.1177/0278364914528132).
 - [19] J. E. Bobrow, S. Dubowsky, and J. S. Gibson, “Time-optimal control of robotic manipulators along specified paths,” *The International Journal of Robotics Research*, vol. 4, no. 3, pp. 3–17, 1985. DOI: [10.1177/027836498500400301](https://doi.org/10.1177/027836498500400301).
 - [20] K. G. Shin and N. D. McKay, “Minimum-time control of robotic manipulators with geometric path constraints,” *IEEE Transactions on Automatic Control*, vol. 30, no. 6, pp. 531–541, 1985. DOI: [10.1109/TAC.1985.1103986](https://doi.org/10.1109/TAC.1985.1103986).
 - [21] D. Verschueren, B. Demeulenaere, J. Swevers, J. De Schutter, and M. Diehl, “Time-optimal path tracking for robots: A convex optimization approach,” *IEEE Transactions on Robotics*, vol. 25, no. 2, pp. 358–368, 2009. DOI: [10.1109/TR0.2009.2013492](https://doi.org/10.1109/TR0.2009.2013492).

- [22] R. Deits and R. Tedrake, “Computing large convex regions of obstacle-free space through semidefinite programming,” in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2014. [Online]. Available: https://groups.csail.mit.edu/robotics-center/public_papers/Deits14.pdf.
- [23] R. Deits and R. Tedrake, “Growing convex collision-free regions in configuration space using nonlinear programming,” *arXiv preprint*, vol. arXiv:2303.14737, 2023. [Online]. Available: <https://arxiv.org/abs/2303.14737>.
- [24] R. Deits and R. Tedrake, “Finding and optimizing certified, collision-free regions in configuration space for robot manipulators,” *arXiv preprint*, vol. arXiv:2205.03690, 2022. [Online]. Available: <https://arxiv.org/abs/2205.03690>.
- [25] T. Marcucci, “Graphs of convex sets with applications to optimal control and motion planning,” Ph.D. dissertation, Massachusetts Institute of Technology, 2024. [Online]. Available: <https://dspace.mit.edu/handle/1721.1/156046>.

Chương A

Phụ lục: Chi tiết chiến lược giải bài toán MIOCP/MINLP

A.1 Chiến lược giải bài toán MIOCP/MINLP

Bài toán MINLP (4.46)–(4.57) thuộc lớp NP-hard và không có bộ giải thương mại hiệu suất cao tương đương Mosek/Gurobi cho lớp MISOCP. Trong khuôn khổ này, chúng tôi sử dụng một chiến lược heuristic gồm ba bước: giải relaxation liên tục tích hợp của MINLP sau rời rạc hóa, trích xuất đường đi rời rạc từ nghiệm relaxation, rồi tinh chỉnh quỹ đạo bằng một NLP trên đường đi cố định. Các bài toán NLP trong pipeline được xây dựng bằng CasADi và giải bằng IPOPT; các thao tác trên đồ thị như tìm đường ngắn nhất hoặc liệt kê đường ứng viên được xử lý riêng trên đồ thị vùng.

Pha 1 — Relaxation liên tục tích hợp của MINLP. Tất cả biến của bài toán (4.46)–(4.57) được gom vào một NLP duy nhất sau khi thả lỏng các biến nhị phân $y_{uv}, p_v \in \{0, 1\}$ thành các biến liên tục trong đoạn $[0, 1]$. Vector biến quyết định gồm luồng cạnh y_{uv} , biến kích hoạt vùng p_v , trạng thái biên s_v^-, s_v^+ , tham số điều khiển w_v , thời lượng vùng Δ_v , trạng

thái giao diện z_{uv} , và biến epigraph chi phí cục bộ ρ_v .

Cơ chế bật/tắt vùng được áp đặt bằng các ràng buộc Big-M dựa trên p_v .

Cụ thể, các biến trạng thái và thời lượng cục bộ được chặn bởi

$$-M_s p_v \leq s_v^\pm \leq M_s p_v, \quad 0 \leq \Delta_v \leq M_\Delta p_v, \quad 0 \leq \rho_v \leq M_\rho p_v.$$

Với tham số điều khiển, ta dùng dạng

$$-M_w p_v \leq w_v - \bar{w}_v \leq M_w p_v,$$

trong đó $\bar{w}_v$ là điều khiển danh nghĩa khi vùng không được kích hoạt. Khi $p_v = 0$, các biến động lực học của vùng bị tắt được đưa về giá trị danh nghĩa và $\Delta_v = 0$; do đó endpoint map thỏa $F_v(0, \bar{w}_v, 0) = 0$, khiến ràng buộc defect (4.52) trở thành đẳng thức tầm thường mà không cần gắn thêm Big-M trực tiếp vào defect. Trong cài đặt số, F_v được xấp xỉ bằng tích phân RK4 trên N_{int} bước. Hàm mục tiêu được viết dưới dạng $\min \sum_v \rho_v$ thông qua epigraph hóa chi phí cục bộ. Các hằng số Big-M được chọn từ miền chặn vật lý của trạng thái, điều khiển, thời gian và chi phí; giá trị quá lớn có thể làm bài toán NLP kém điều kiện số.

Ràng buộc hình học bật/tắt hỗ trợ hai chế độ:

- *Mesh-point hard constraint*: với polytope $Q_v = \{q : A_v q \leq b_v\}$, ràng buộc $A_v q \leq b_v + M_Q(1 - p_v)$ được áp tại hai trạng thái biên s_v^-, s_v^+ và tại các điểm mesh nội tại của quỹ đạo RK4. Có thể thêm dung sai biên ϵ_b và khoảng dự phòng η_{safe} để tránh nghiệm nằm quá sát biên vùng.
- *Log-barrier penalty*: containment cứng vẫn được giữ tại hai trạng thái biên, còn an toàn nội tại được đưa vào hàm mục tiêu bằng số hạng

$$-\mu \sum_v p_v \sum_j \log((b_v)_j + M_Q(1 - p_v) - (A_v q)_j).$$

Dịch slack $M_Q(1 - p_v)$ được đặt bên trong logarit để vô hiệu hóa vùng không kích hoạt và tránh đánh giá logarit ngoài miền xác định.

Các ràng buộc ghép nối giao diện $s_u^+ = z_{uv} = s_v^-$, điều kiện nguồn/dích, và nếu cần cả ràng buộc liên tục điều khiển giữa hai vùng, được bật/tắt bằng Big-M theo biến cạnh y_{uv} . Khi cạnh (u, v) được kích hoạt, z_{uv} đồng thời phải nằm trong cả hai polytope Q_u và Q_v .

Khởi tạo (warm start). Để cải thiện khả năng hội tụ của IPOPT, nghiệm khởi tạo được xây dựng từ một đường đi hình học π_0 trên đồ thị vùng. Cụ thể, π_0 là đường ngắn nhất từ σ đến τ theo trọng số khoảng cách giữa tâm các vùng. Với mỗi vùng $v \in \pi_0$, các trạng thái s_v^- và s_v^+ được đặt tại các điểm neo trên giao của hai vùng kề nhau; nếu giao bị thoái hóa, dùng trung điểm giữa hai tâm vùng làm xấp xỉ. Góc hướng θ được suy từ vector nối điểm vào và điểm ra, Δ_v được ước lượng từ chiều dài đoạn chia cho vận tốc tối đa, còn w_v được đặt bằng điều khiển danh nghĩa. Các biến luồng được khởi tạo bằng $y_{uv} = 1, p_v = 1$ trên π_0 và bằng 0 trên các cạnh, vùng còn lại.

Continuation trên tham số rào. Khi sử dụng log-barrier, chúng tôi áp dụng kỹ thuật continuation trên tham số μ [6]. Cụ thể, giải một dãy NLP với μ giảm dần từ $\mu_0 = 10^{-2}$ về $\mu_{\min} = 10^{-5}$, dùng nghiệm ở bước trước làm khởi tạo cho bước tiếp theo. Số vòng continuation và hệ số giảm μ là các tham số cấu hình. Với chế độ mesh-point hard constraint, bước continuation này được bỏ qua.

Pha 2 — Trích xuất đường đi rời rạc. Sau khi NLP relaxation hội tụ, nghiệm $y^* \in [0, 1]^{|E|}$ được chuyển thành một đường đi rời rạc π^* . Trước tiên, ta duyệt tham lam từ σ : tại mỗi đỉnh, chọn cạnh ra có y_{uv}^* lớn nhất nếu giá trị này vượt ngưỡng 0.5; nếu không có cạnh nào đạt ngưỡng, hạ ngưỡng xuống 10^{-3} để tránh dừng sớm. Nếu thủ tục tham lam không đến

được τ , hoặc độ lệch nguyên

$$\delta_{\text{int}} = \max_{e \in E} \min\{y_e^*, 1 - y_e^*\}$$

lớn hơn 10^{-3} , ta thay bằng đường đi ngắn nhất trên đồ thị con $\{e : y_e^* > 10^{-6}\}$ với trọng số $-\log y_e^*$. Cách làm này ưu tiên các cạnh có giá trị luồng lớn trong nghiệm relaxation, nhưng vẫn trả về một đường đi rời rạc hợp lệ.

Pha 3 — Polish quỹ đạo trên đường đi cố định. Nghiệm relaxation có thể còn phân số, nên các trạng thái nối s_u^+ và s_v^- trên hai vùng liên tiếp của π^* có thể chưa khớp hoàn toàn. Ta đo sai lệch này bằng *connection gap*. Nếu $\max\{\delta_{\text{int}}, \delta_{\text{conn}}\}$ vượt ngưỡng cấu hình, hoặc nếu muốn loại bỏ hoàn toàn ảnh hưởng của Big-M, ta cố định đường đi π^* và giải lại một NLP *fixed-path*. Bài toán này chỉ giữ các biến liên tục trên các vùng thuộc π^* ; các ràng buộc ghép nối giao diện trở thành đẳng thức cứng, kèm điều kiện biên start/goal, ràng buộc defect multiple shooting, và ràng buộc containment theo cùng chế độ ở Pha 1. Đây là dạng direct multiple shooting tiêu chuẩn trên một chuỗi vùng đã biết [4], [14], [15]. Nghiệm relaxation được dùng làm khởi tạo cho NLP fixed-path; nếu dùng log-barrier, continuation trên μ được tiếp tục áp dụng trong bước polish.